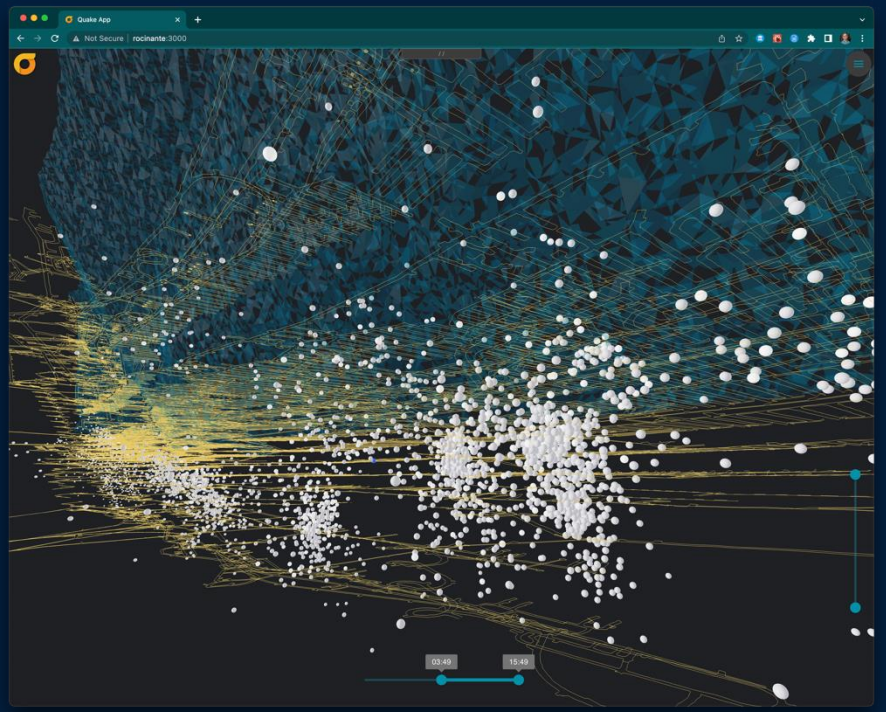
L8 · Orchestration & Scheduling

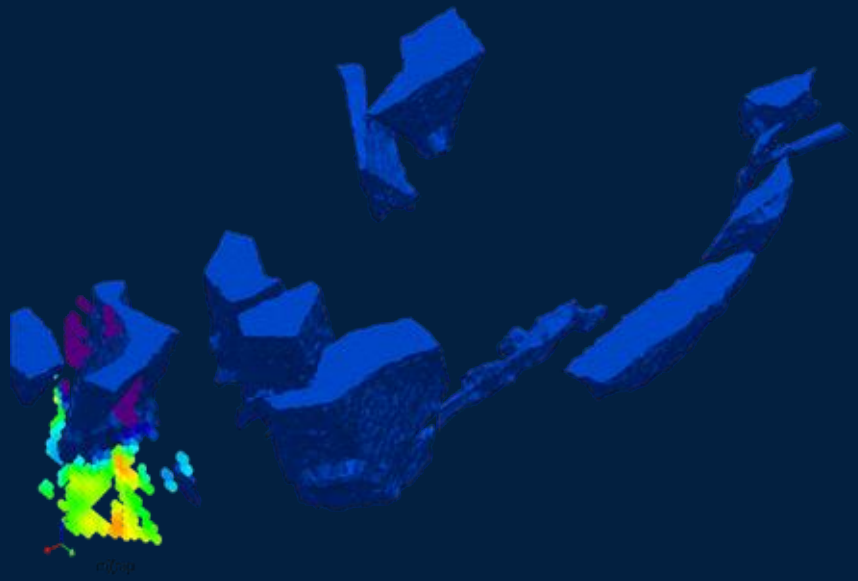
D7024E Mobile and Distributed Computing Systems · Johan Kristiansson · Autumn 2026



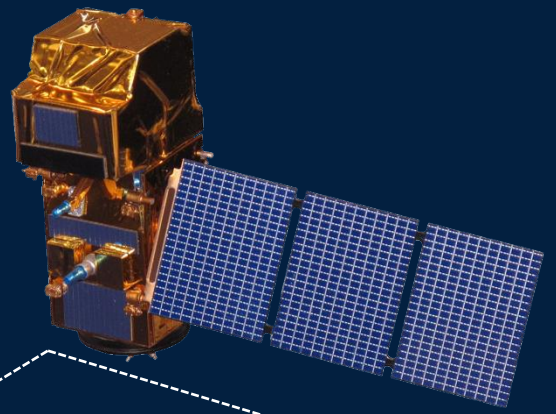
RockSigma AB

- Seismic processing underground mines
- Used by LKAB to analyze seismicity and process a massive amount of data from one the largest mines in the world (Kiruna/Malmberget)
- On-preem + cloud



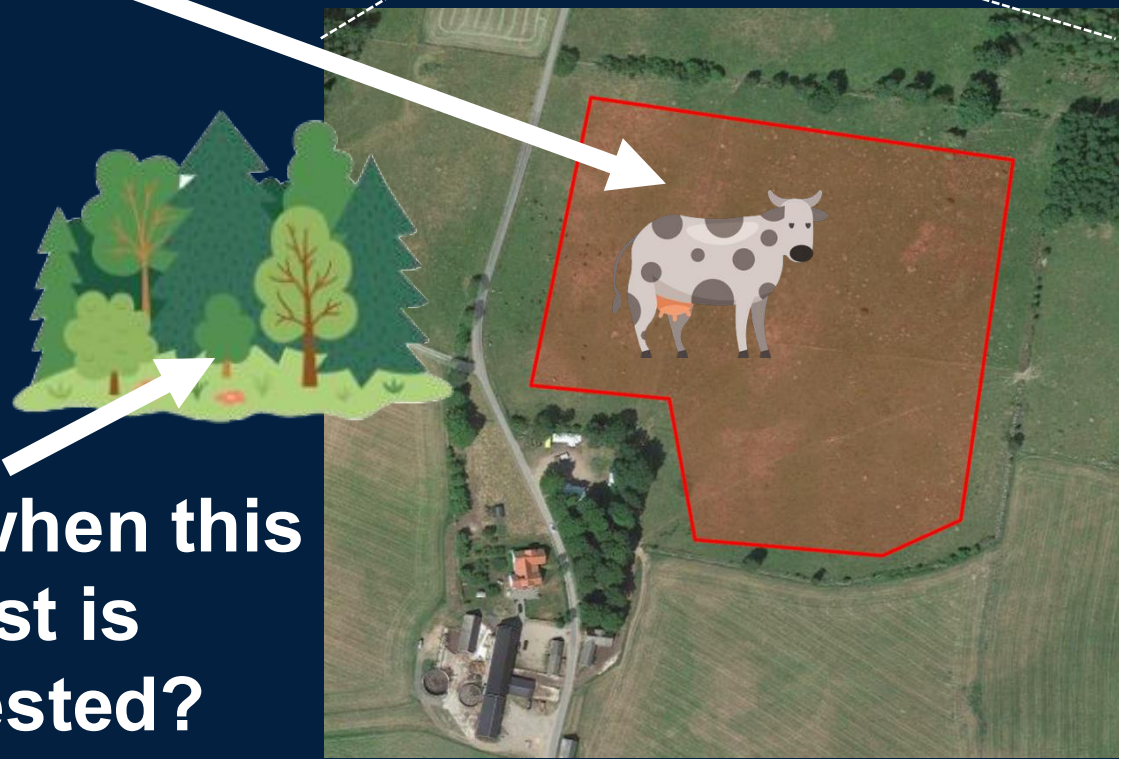


Earth Observation



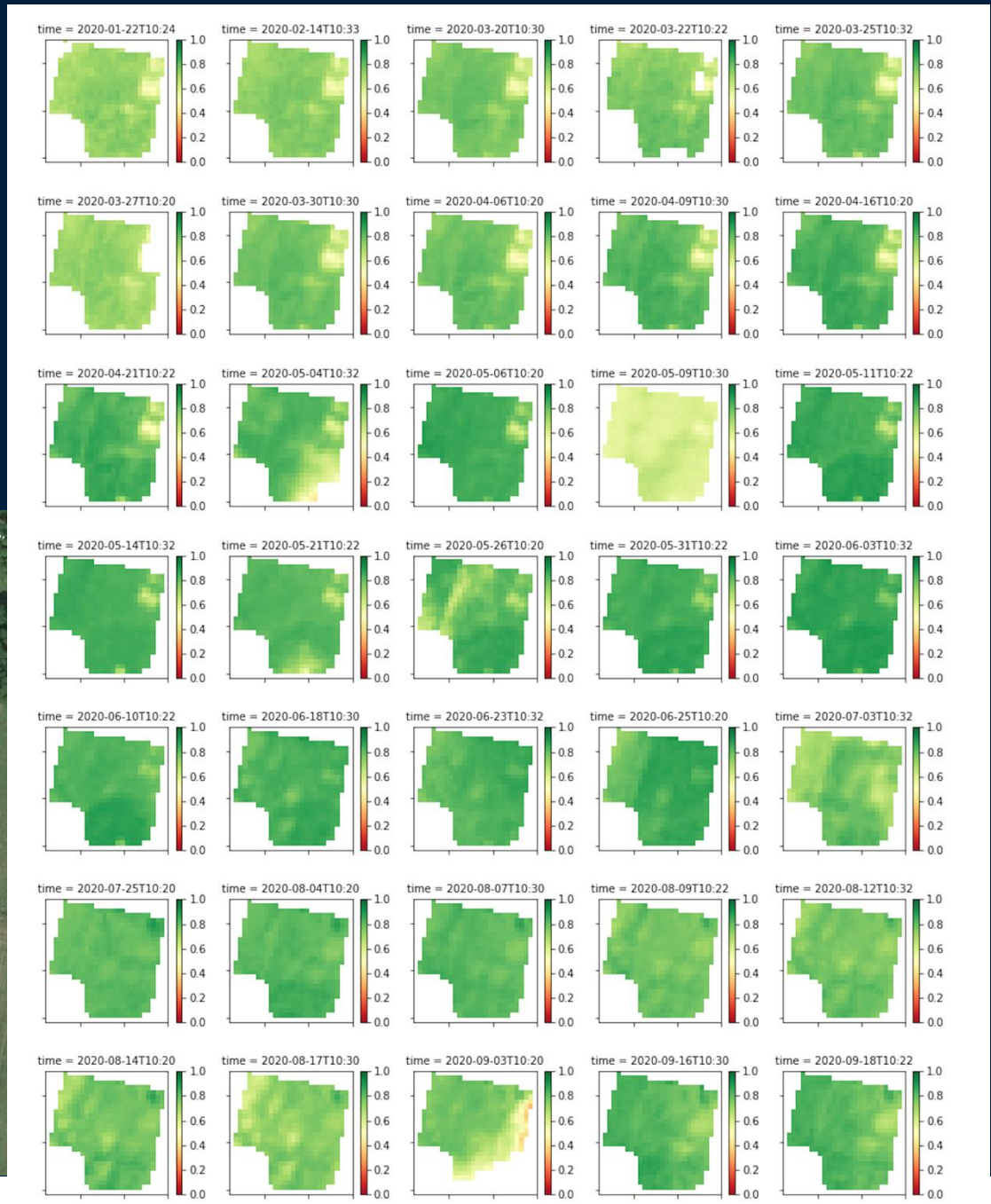
Did the cow graze?

Jordbruks verket



SKOGSSTYRELSEN

Tell me when this forest is harvested?



Workflow says what; scheduling decides where and when; orchestration makes it happen

WORKFLOW

the specification

A set of tasks and the dependencies between them — a DAG — with what each task consumes and produces. Written by the author; it says what must run and in which order, not where or when.

$P_1 \rightarrow \{P_2, P_3\} \rightarrow P_4$; P_3 needs a GPU and D7

SCHEDULING

the decision

Allocating resources to tasks over time to optimise an objective: which READY task goes to which machine, when, ahead of whom. Decided online, by a policy someone owns.

"P_3 runs on the cloud GPU at 10:42, before P_4's user's job"

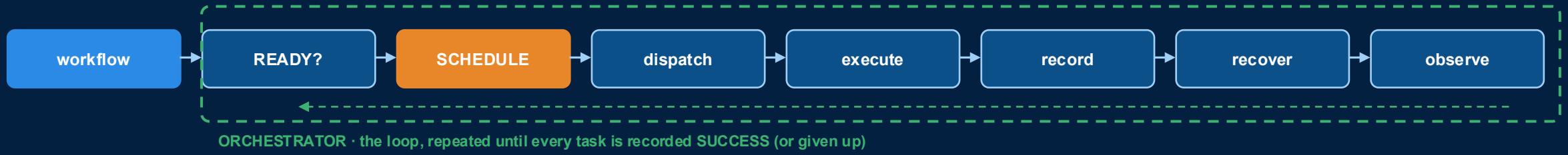
ORCHESTRATION

the control loop

Turning the specification into completed work on real machines: track readiness, schedule, dispatch, execute, record outcomes, recover from failure, expose what is happening.

gets P_3 there, notices when it dies, runs it again once

How they relate: the workflow is the input, the scheduler is one step of the loop, the orchestrator is the loop



Scheduling is a decision; orchestration is the discipline of making decisions, carrying them out on unreliable machines, and recording what happened. The stations of the map are that loop, taken apart.

The lecture in one map

1 Workflow

What may run next?

· DAG, readiness, execution
· Work and span; Graham's anomaly and its cure

2 Scheduling

Where and when?

· Kubernetes as the system model
· Bin packing: brute force, heuristics, a genetic algorithm
· Little's law and the utilisation curve

3 Data & communication

How do processes exchange information?

· Data must be fetched; the computation goes to the data
· MapReduce: model, shuffle, cluster, messages
· Blocked networks, a broker, a specification; push or pull

★ Case study · ColonyOS

Twelve failure modes — and the design they lead to

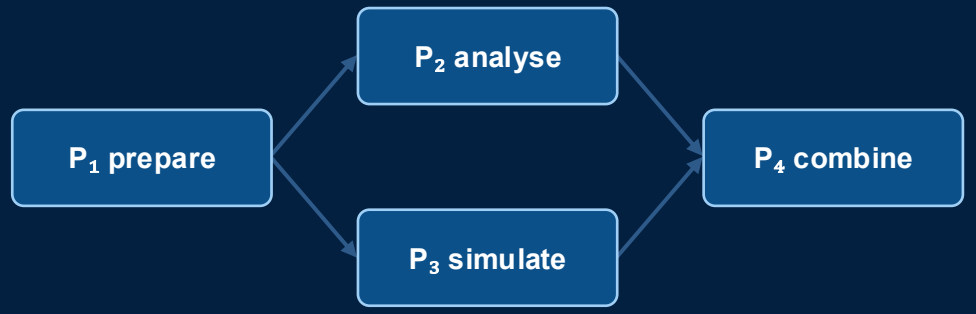
· Silence and time; lost messages; two owners; the record
· pull · match · claim · return; leases, retries
· Two queues, three clocks; diagnose a stall

1 · Workflow

What may run next?

A DAG states execution order — *and data dependencies*

Directed Acyclic Graph (DAG)



$V = \{P_1, P_2, P_3, P_4\}$ $E = \{(P_1,P_2), (P_1,P_3), (P_2,P_4), (P_3,P_4)\}$
acyclic ⇔ a topological order π exists: $(u, v) \in E \Rightarrow \pi(u) < \pi(v)$

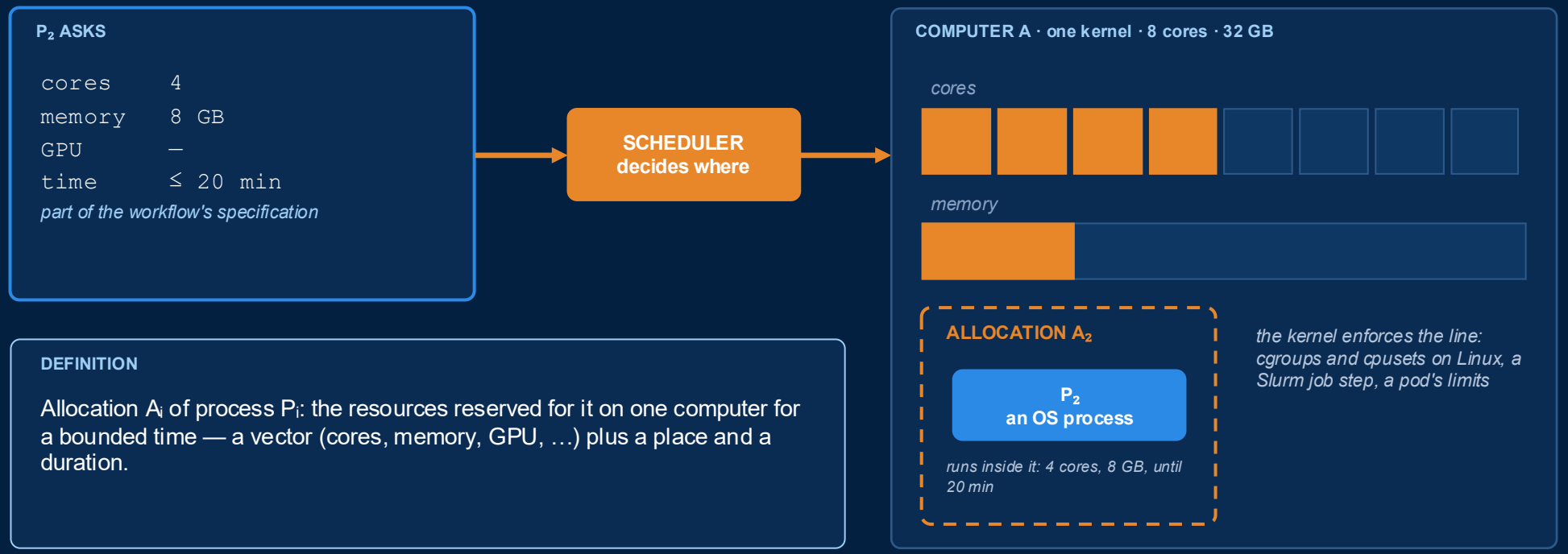
1 · ORDER — when P_2 may run

$ready(v) \Leftrightarrow \forall\ (u, v) \in E : state(u) = SUCCESS$
over recorded states — the coordinator's record, not a machine's belief

2 · DATA — what P_2 consumes

$input(v) \leftarrow output(u)$ for each $(u, v) \in E$
the graph declares it; the system must move or reference the bytes (station 3)

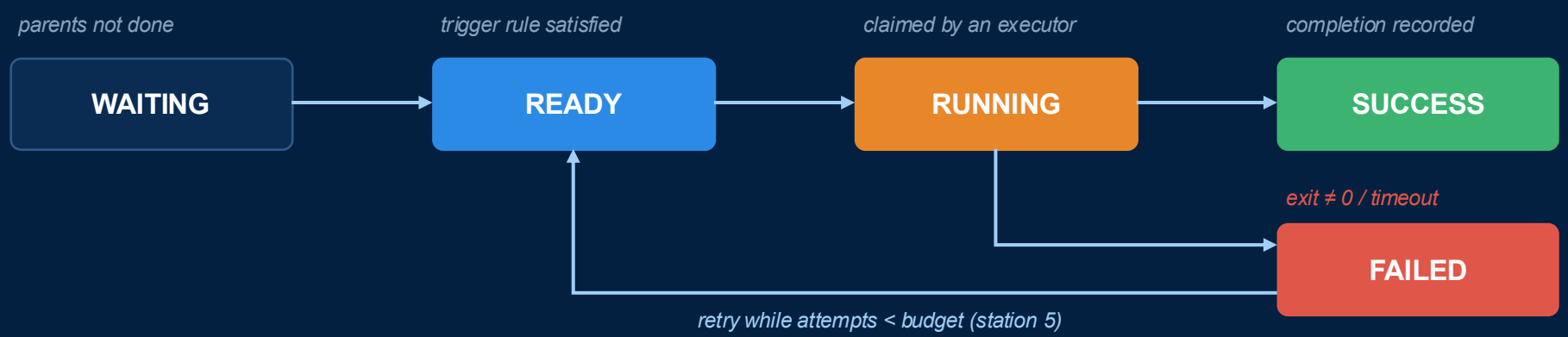
An allocation: what a process asks for, reserved on one computer for a time



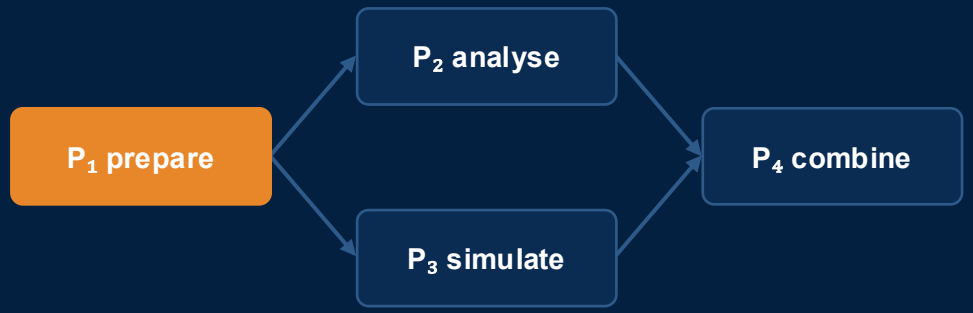
DEFINITION

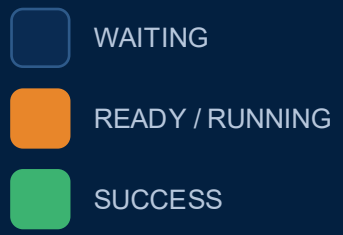
Allocation A_i of process P_i: the resources reserved for it on one computer for a bounded time — a vector (cores, memory, GPU, …) plus a place and a duration.

Readiness is a state machine over recorded states

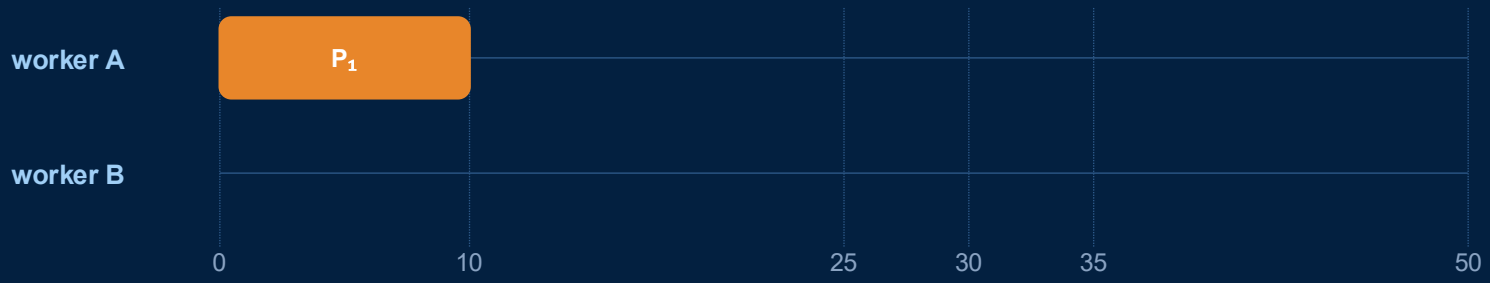


Executing the DAG, step by step





t = 0 · P_1 is the only vertex with in-degree 0: READY, claimed, RUNNING. The others wait.

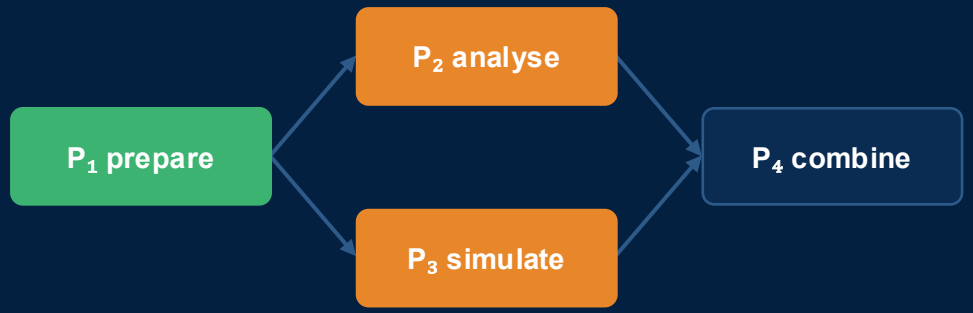


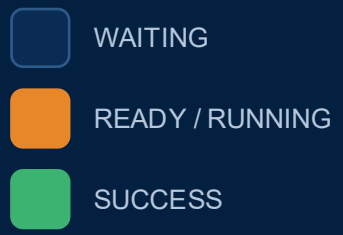
The coordinator's record at t = 0

process	recorded state	owner	started	finished
P_1	RUNNING	A	0	—
P_2	WAITING	—	—	—
P_3	WAITING	—	—	—
P_4	WAITING	—	—	—

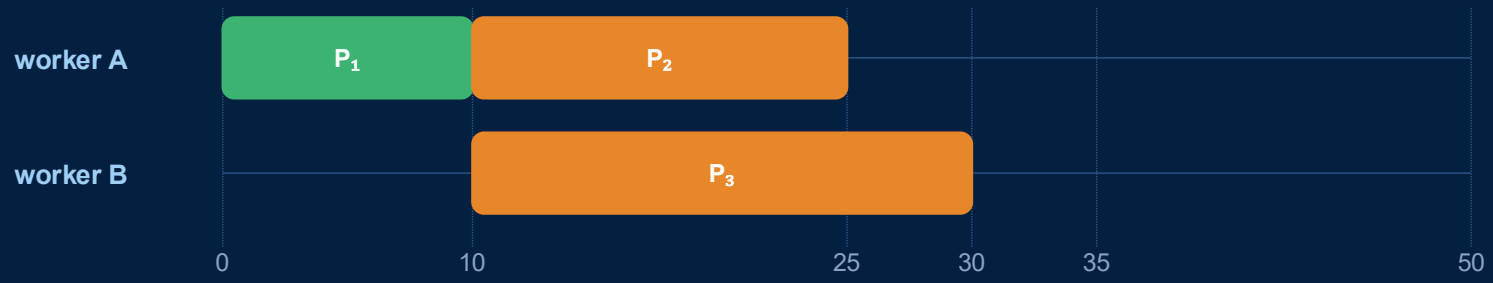
t = 10 min

Executing the DAG, step by step





t = 10 · P_1 SUCCESS is recorded → P_2 and P_3 both READY, both claimed. P_4 still waits for two parents.

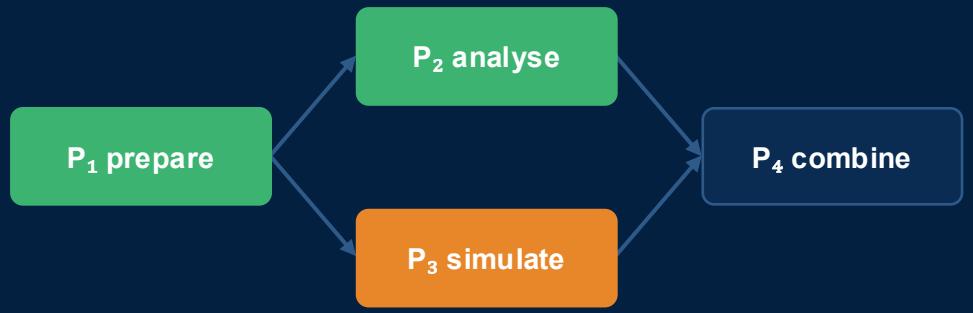


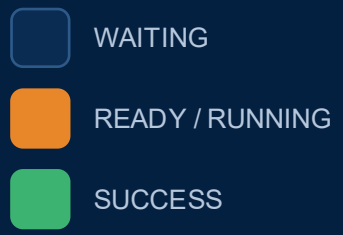
The coordinator's record at t = 10

process	recorded state	owner	started	finished
P_1	SUCCESS	A	0	10
P_2	RUNNING	A	10	—
P_3	RUNNING	B	10	—
P_4	WAITING	—	—	—

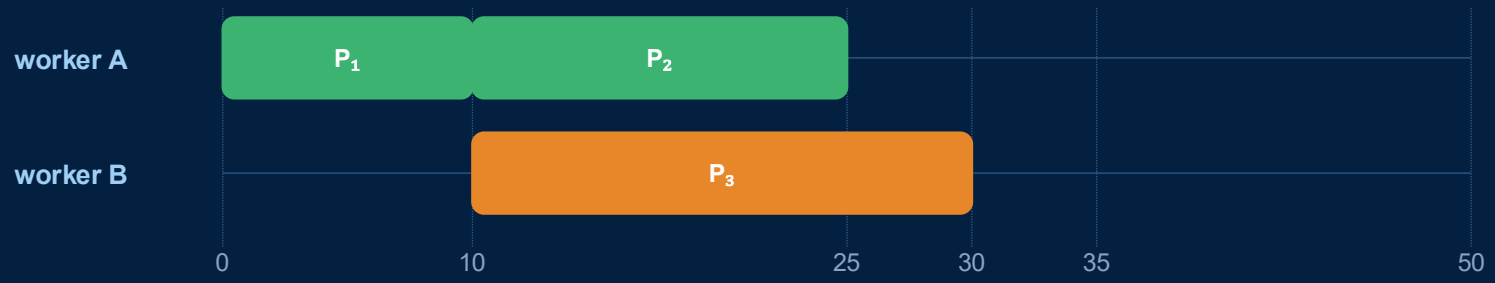
t = 25 min

Executing the DAG, step by step





t = 25 · P_2 SUCCESS. P_4 still WAITING (one parent RUNNING). Worker A idles — nothing is READY.

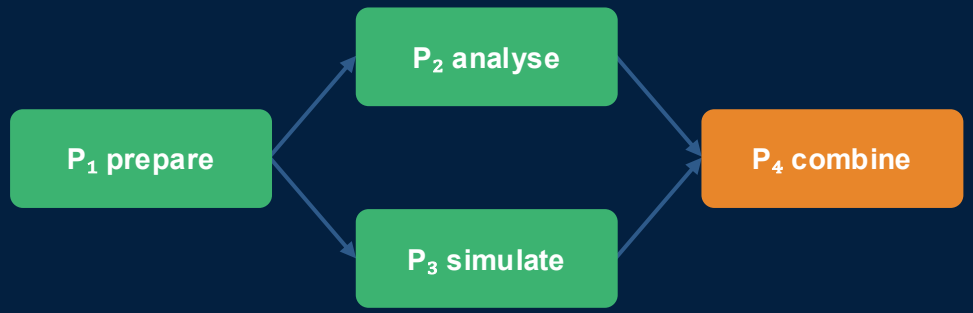


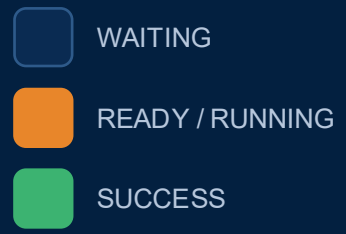
The coordinator's record at t = 25

process	recorded state	owner	started	finished
P_1	SUCCESS	A	0	10
P_2	SUCCESS	A	10	25
P_3	RUNNING	B	10	—
P_4	WAITING	—	—	—

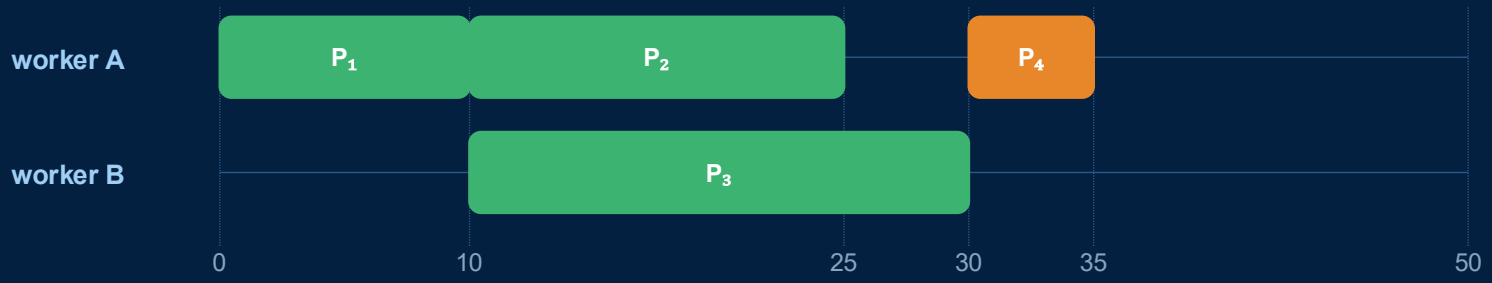
t = 30 min

Executing the DAG, step by step





t = 30 · P_3 SUCCESS → P_4 READY, claimed, RUNNING. Done at 35 = the longest path $P_1 \rightarrow P_3 \rightarrow P_4$.



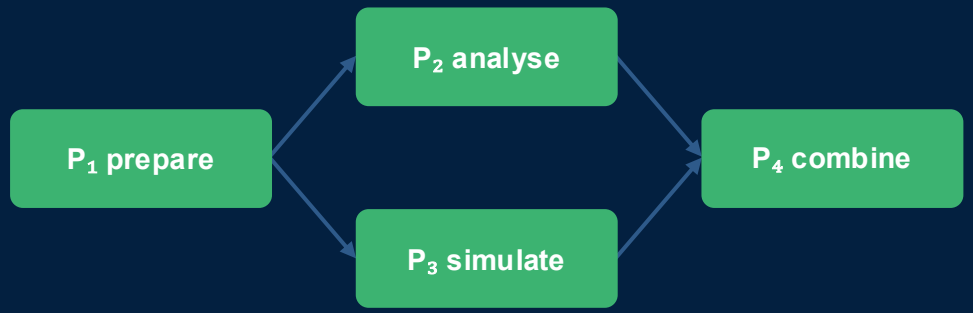
The coordinator's record at t = 30

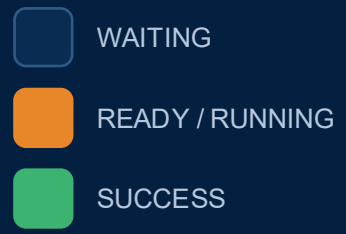
process	recorded state	owner	started	finished
P_1	SUCCESS	A	0	10
P_2	SUCCESS	A	10	25
P_3	SUCCESS	B	10	30
P_4	RUNNING	A	30	—

Finished at 35 min with two workers. Work: 50 min. Longest path: 35 min. Those two numbers are the whole story.

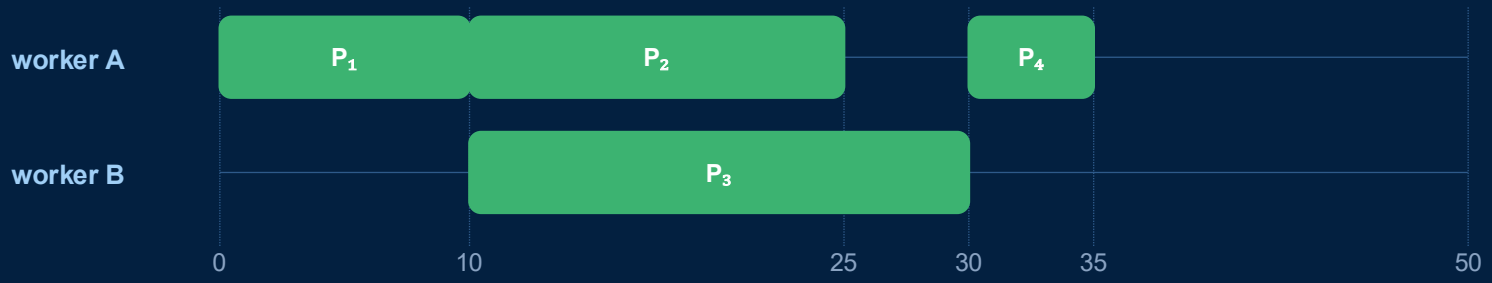
t = 30 min

Executing the DAG, step by step





t = 30 · P_3 SUCCESS → P_4 READY, claimed, RUNNING. Done at 35 = the longest path $P_1 \rightarrow P_3 \rightarrow P_4$.

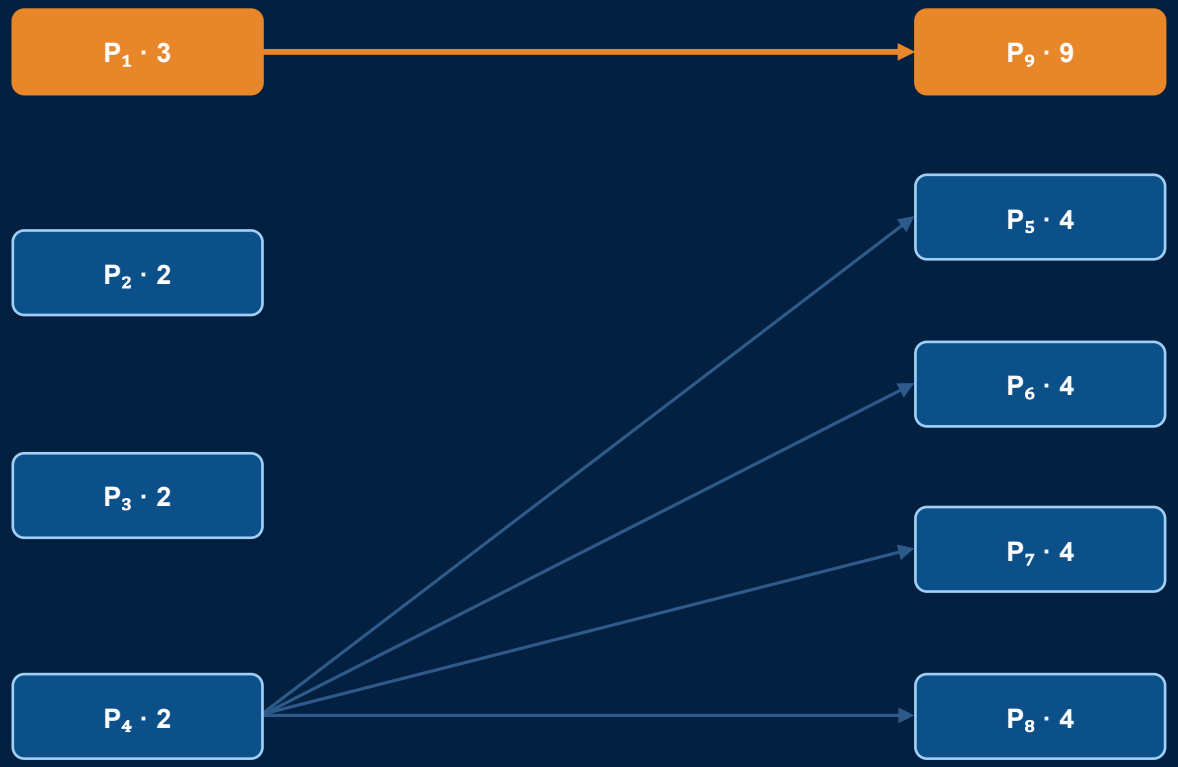


The coordinator's record at t = 30

process	recorded state	owner	started	finished
P_1	SUCCESS	A	0	10
P_2	SUCCESS	A	10	25
P_3	SUCCESS	B	10	30
P_4	SUCCESS	A	30	35

Finished at 35 min with two workers. Work: 50 min. Longest path: 35 min. Those two numbers are the whole story.

Graham's nine-process anomaly



durations in time units · P_2, P_3, P_4 have no parents; P_5–P_8 wait for P_4; P_9 waits for P_1

Work and span

$T_1 = 3+2+2+2+4 \cdot 4+9 = 34$. $T\infty = P_1 \rightarrow P_9 = 3 + 9 = 12$ (orange).

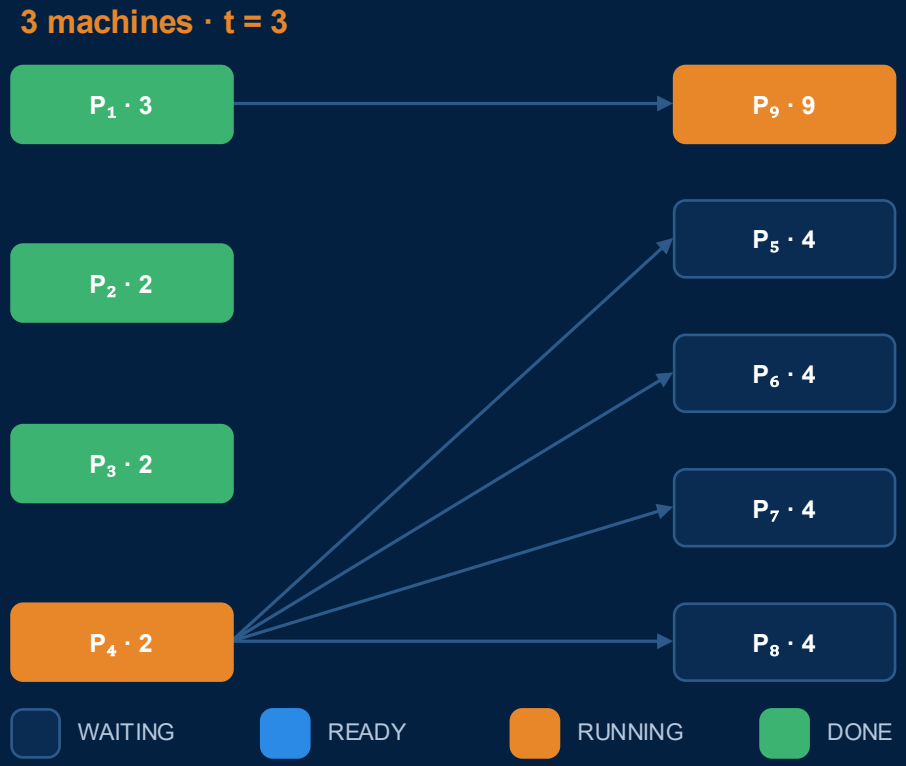
Lower bound

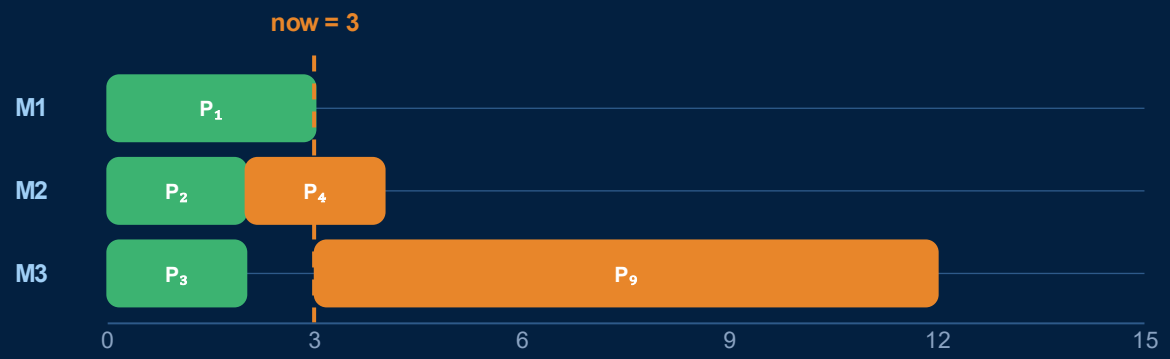
$\max(34/m, 12) = 12$ for both $m = 3$ and $m = 4$ — the span law binds; more machines cannot help.

The scheduler

List scheduling with the fixed priority list $P_1, P_2, \ldots, P_9$: whenever a machine is free, start the first READY process on the list.

Graham's nine-process anomaly





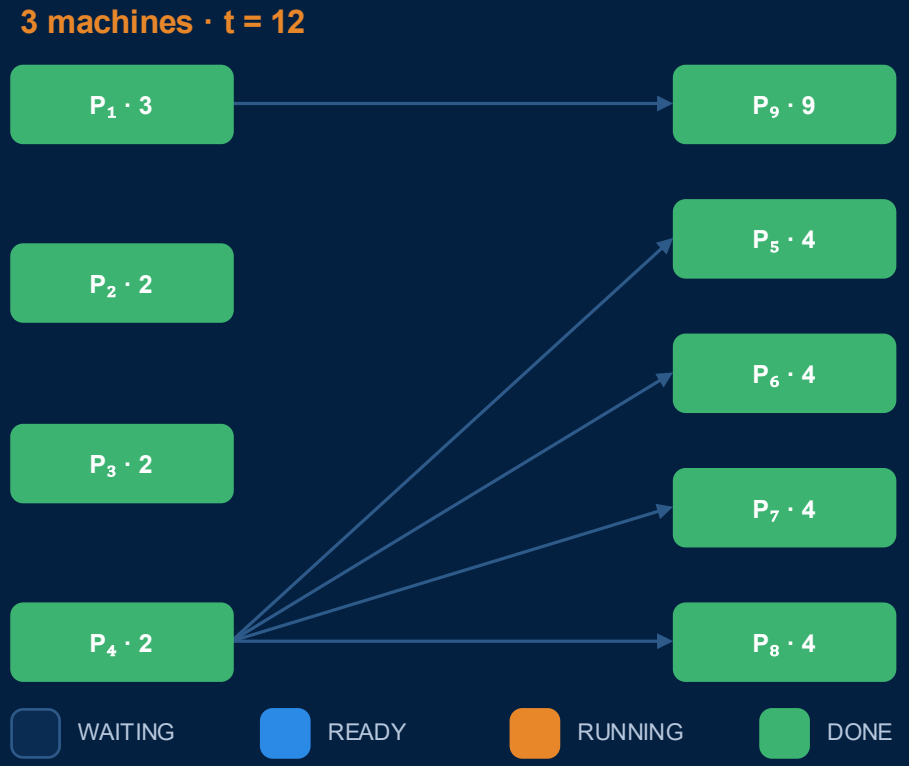
list: P_1 P_2 P_3 P_4 P_5 P_6 P_7 P_8 P_9 — a free machine takes the first READY process on it

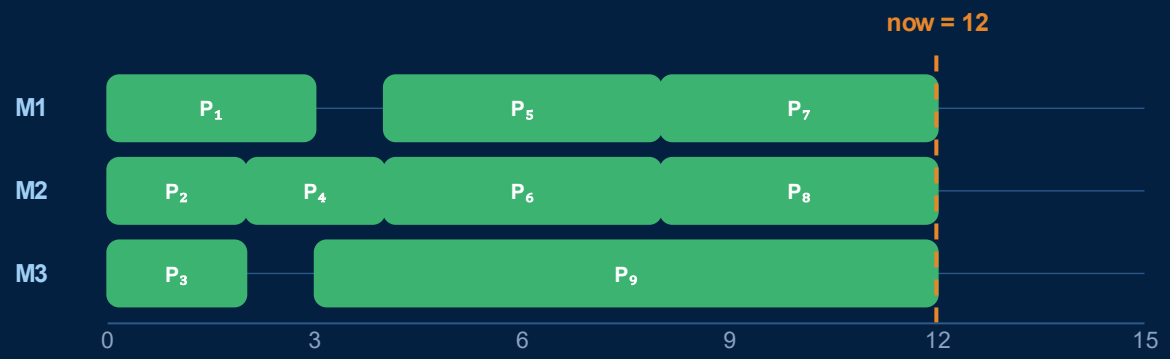
The scheduler's view at t = 3

- READY now: P_9 (P_1 just finished)
- free machines: M1 (now), M3 (idle since 2)
- decision: P_9 → M3 · M1 idles until P_4 finishes at 4

t = 3 · P_1 finishes and P_9 becomes READY. M3 has been idle since P_3 ended at 2 — it takes P_9 at once. P_5–P_8 still wait for P_4 (done at 4).

Graham's nine-process anomaly





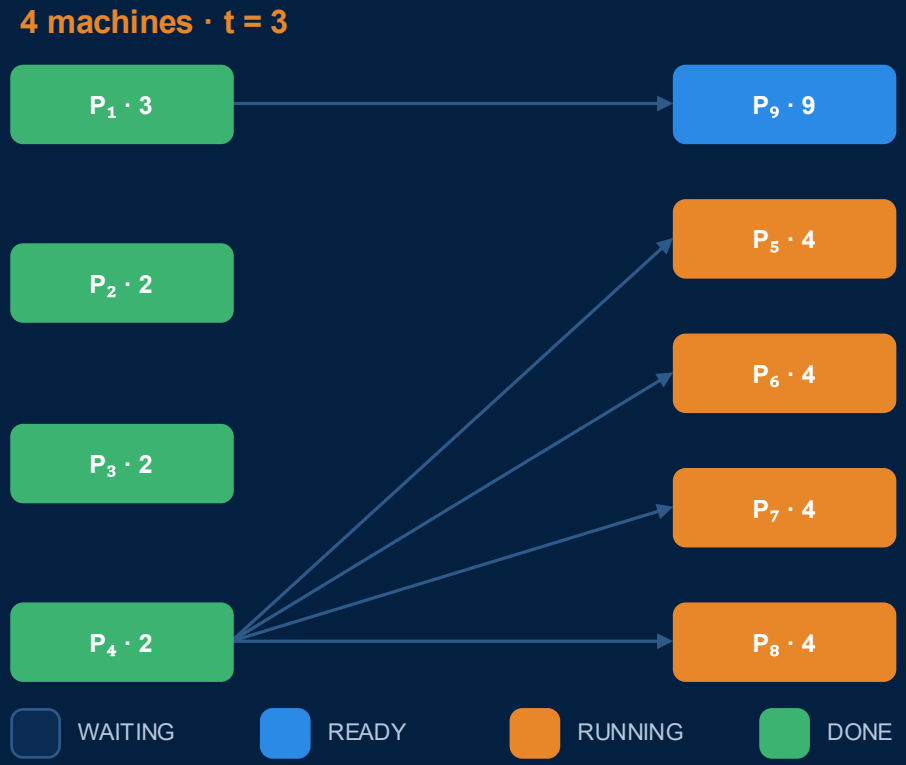
list: P_1 P_2 P_3 P_4 P_5 P_6 P_7 P_8 P_9 — a free machine takes the first READY process on it

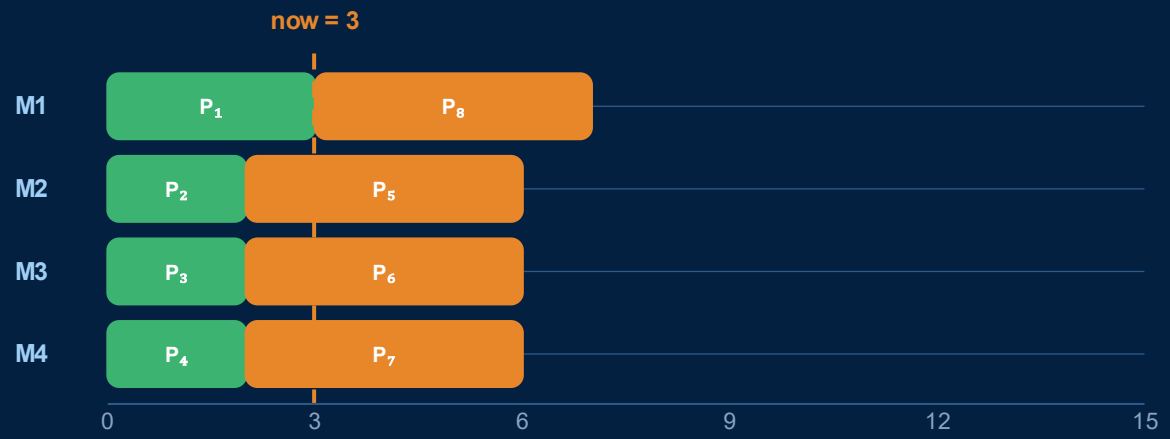
The scheduler's view at t = 12

- t = 4: P_4 done → P_5–P_8 READY; M1 takes P_5, M2 takes P_6
- t = 8: M1 takes P_7, M2 takes P_8
- makespan 12 = lower bound max(34/3, 12) ✓

t = 12 · P_4 finished at 4, so P_5–P_8 pair up on M1 and M2 (4–8, 8–12) while P_9 runs 3–12 on M3. Makespan 12 = the lower bound: optimal.

Graham's nine-process anomaly





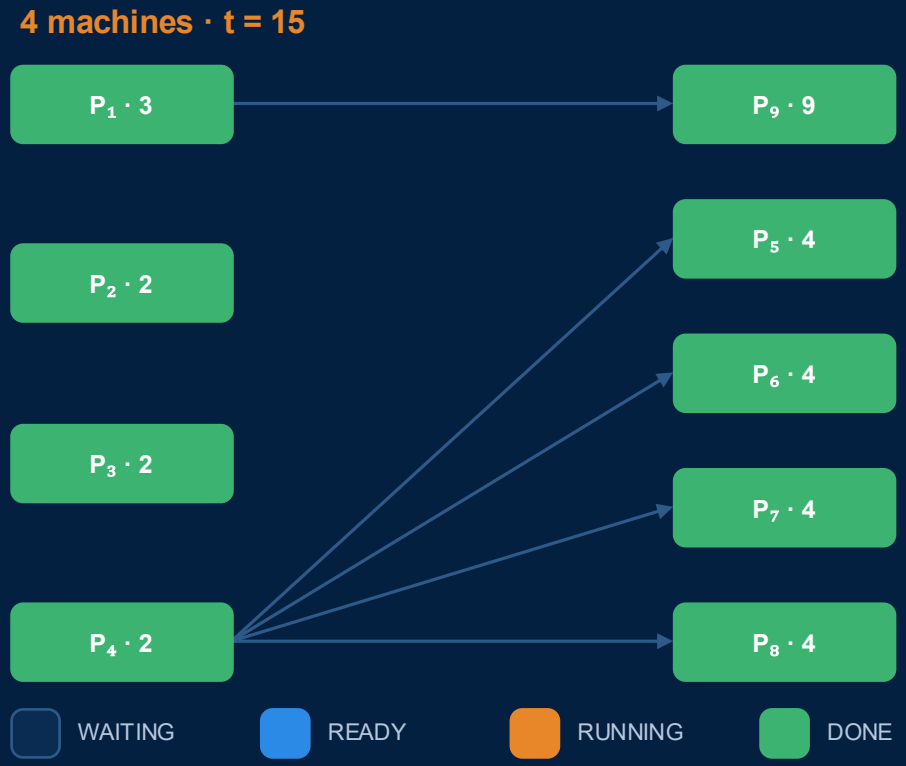
list: P_1 P_2 P_3 P_4 P_5 P_6 P_7 P_8 P_9 — a free machine takes the first READY process on it

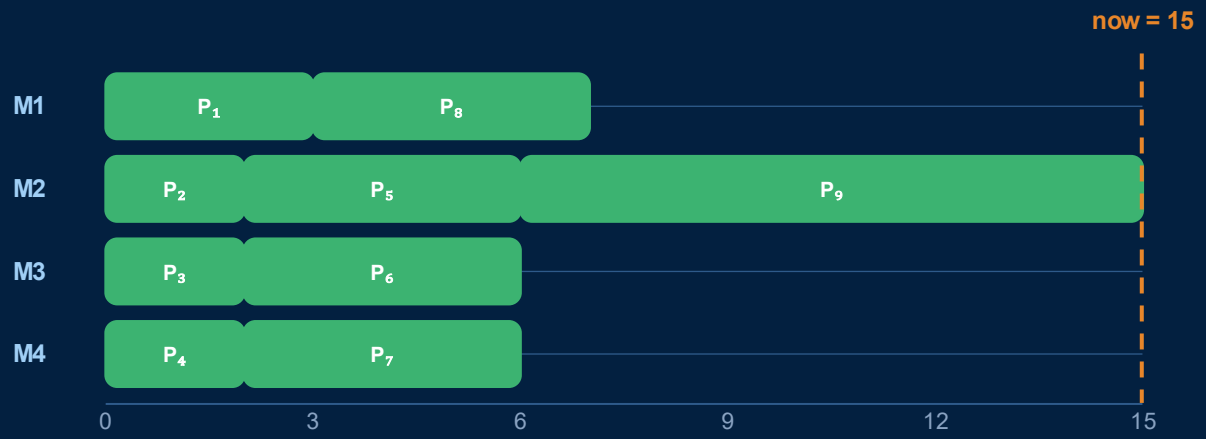
The scheduler's view at t = 3

- t = 2: P_4 done → P_5–P_8 READY, take M2–M4 (P_9 not yet READY)
- READY now: P_8, P_9 · free machine: M1
- decision: P_8 is earlier on the list → M1 takes P_8 · P_9 waits

t = 3 · With a fourth machine P_4 started at 0 and finished at 2: P_5–P_8 became READY before P_9 and took M2–M4. P_1 finishes now; M1 takes P_8, earlier on the list. P_9 is READY — and waits.

Graham's nine-process anomaly





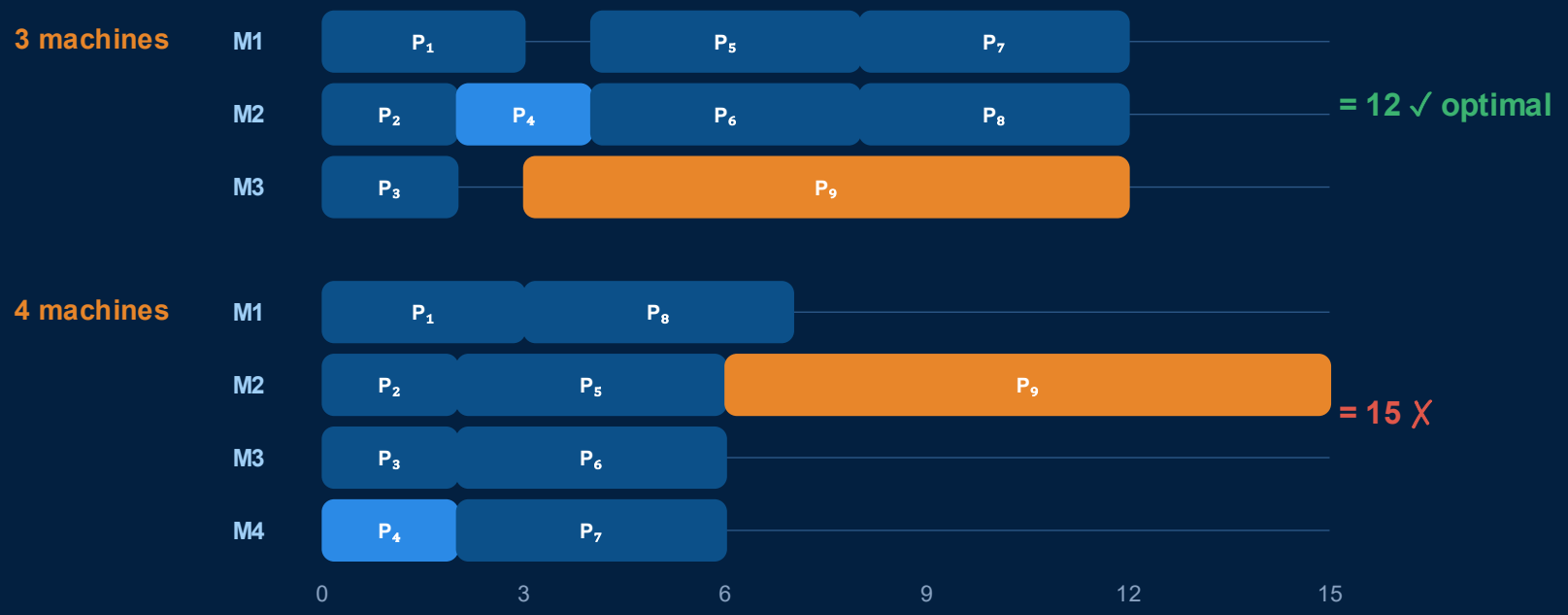
list: $P_1\ P_2\ P_3\ P_4\ P_5\ P_6\ P_7\ P_8\ P_9$ — a free machine takes the first READY process on it

The scheduler's view at t = 15

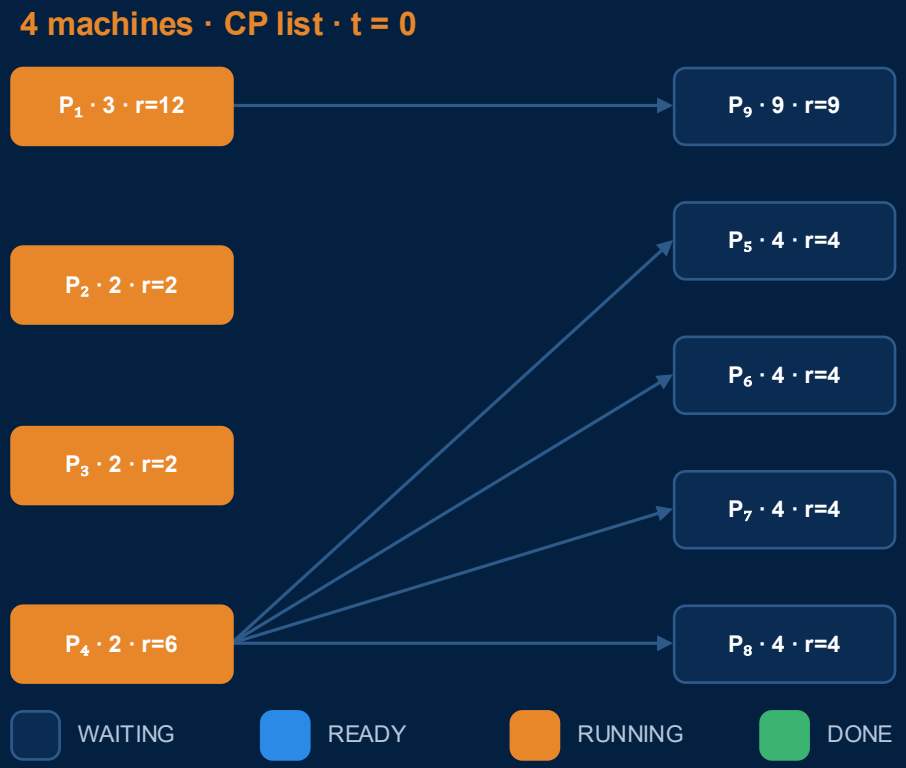
- t = 6: M2–M4 free → P_9 finally starts on M2
- t = 7: M1 free, nothing left
- makespan 15 > 12 — with one machine more

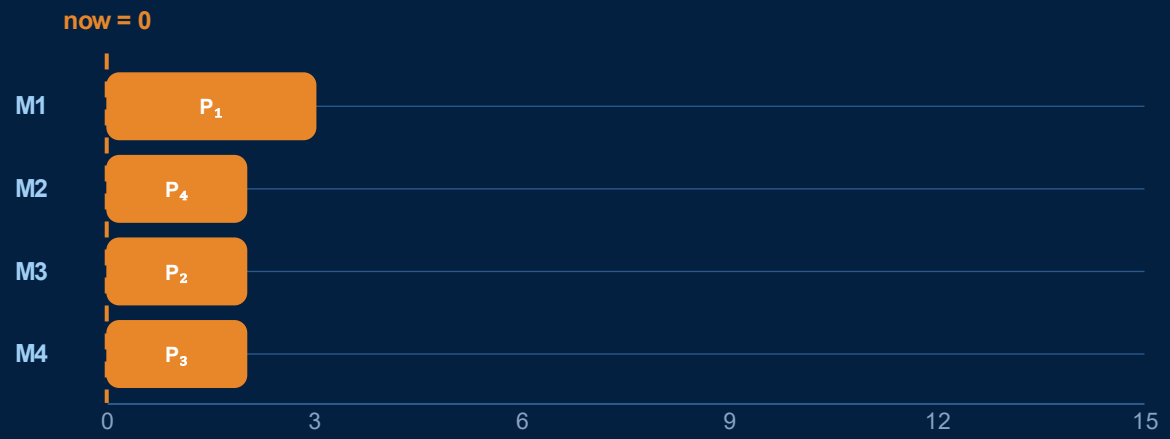
3 machines: 12 ✓ optimal **4 machines: 15 ✗ — one machine more, three time units longer**

Graham's nine-process anomaly



Critical path first on Graham's DAG





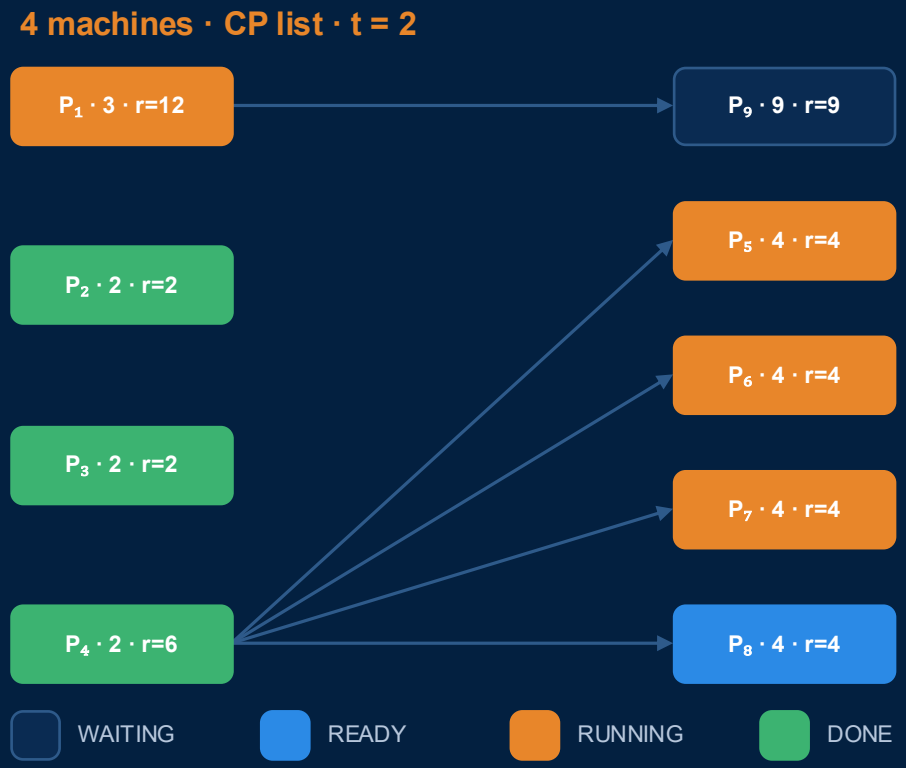
list by rank: P_1 P_9 P_4 P_5 P_6 P_7 P_8 P_2 P_3 — a free machine takes the first READY process on it

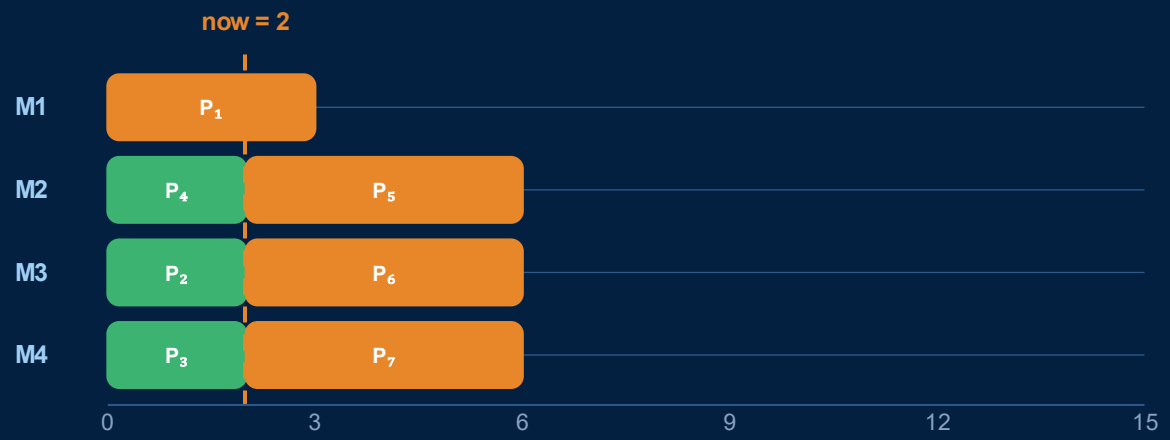
The scheduler's view at t = 0

· rank = own time + longest path through successors: P_1 12, P_9 9, P_4 6, P_5–P_8 4, P_2 P_3 2
· READY now: P_1 (12), P_4 (6), P_2 (2), P_3 (2) · free: M1–M4
· decision: highest rank first → P_1 → M1, P_4 → M2, P_2 → M3, P_3 → M4

t = 0 · Every process carries its rank: the span that remains if it started now. Four READY, four machines — all start, but the rank fixes who gets a machine first when that matters.

Critical path first on Graham's DAG





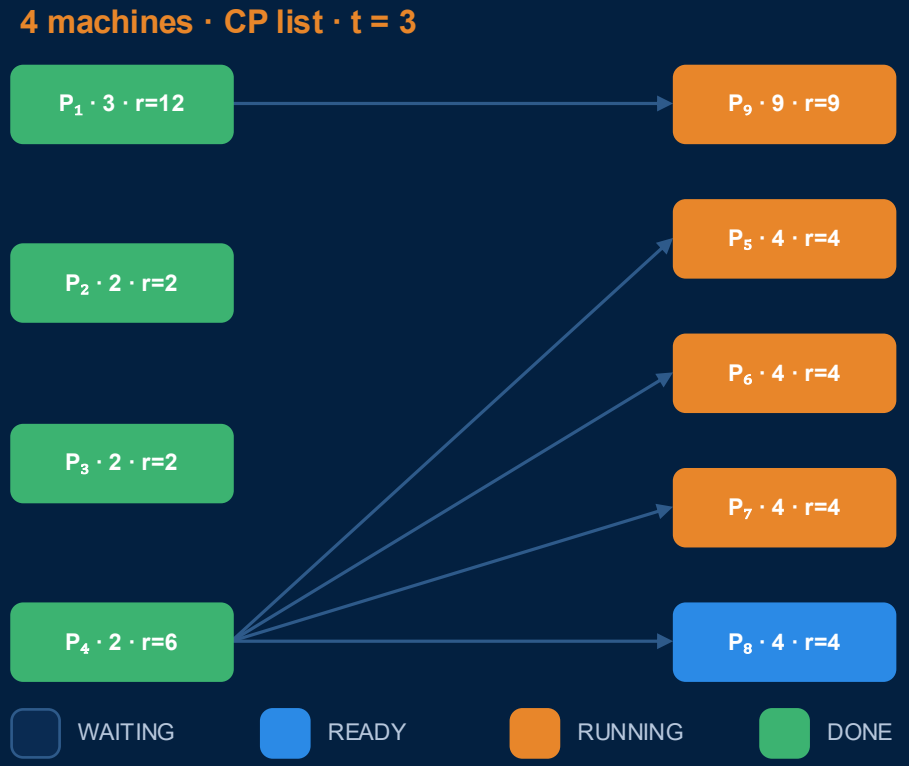
list by rank: P_1 P_9 P_4 P_5 P_6 P_7 P_8 P_2 P_3 — a free machine takes the first READY process on it

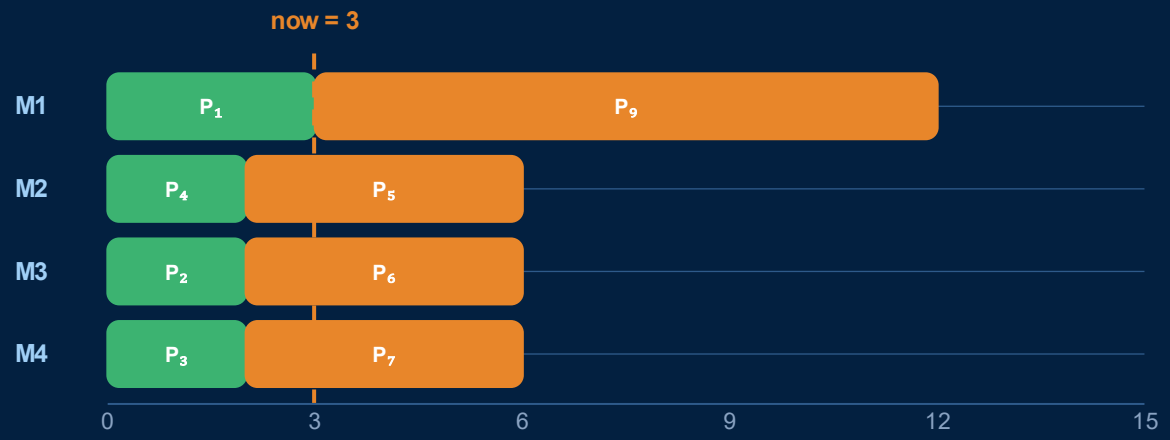
The scheduler's view at t = 2

· t = 2: P_4, P_2, P_3 done → P_5–P_8 READY (rank 4 each)
· free: M2, M3, M4 → P_5, P_6, P_7 start · P_8 has no machine and waits
· P_9 not yet READY: P_1 runs until 3

t = 2 · P_4 finished, so its four children are READY. Three machines are free: P_5, P_6, P_7 start; P_8 waits for a machine. Exactly as with the plain list so far — the ranks have not been needed.

Critical path first on Graham's DAG





list by rank: P_1 P_9 P_4 P_5 P_6 P_7 P_8 P_2 P_3 — a free machine takes the first READY process on it

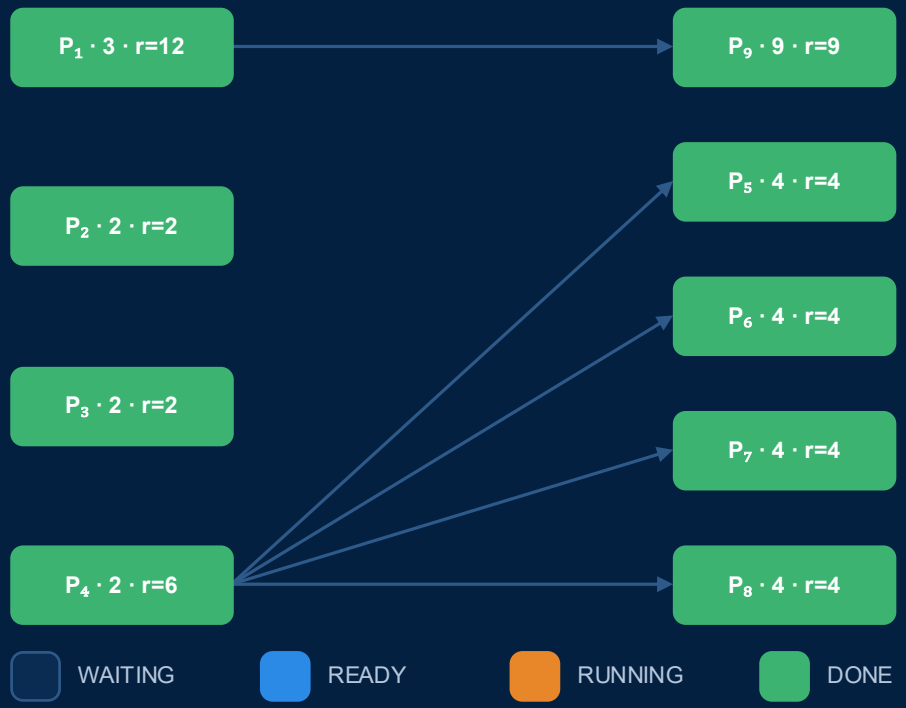
The scheduler's view at t = 3

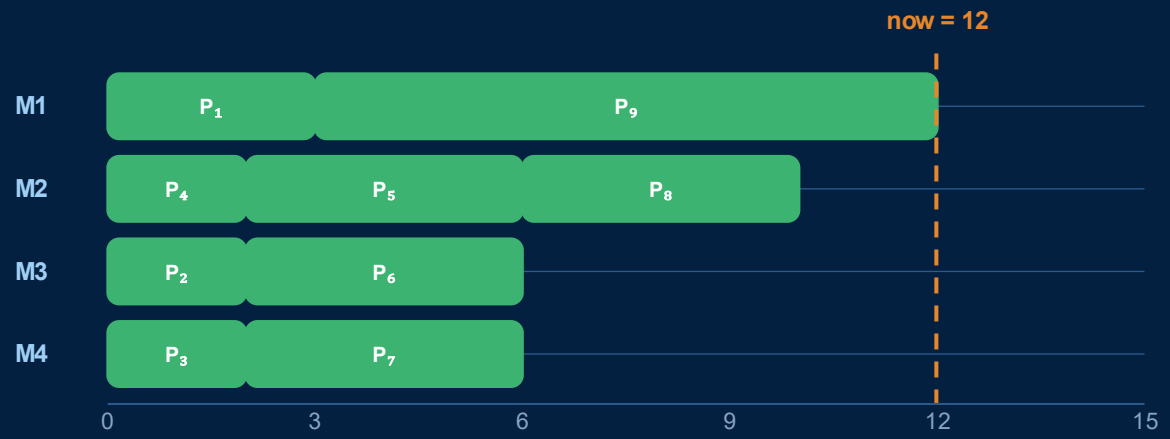
· t = 3: P_1 done → P_9 READY (rank 9)
· READY now: P_9 (9), P_8 (4) · free: M1
· decision: rank 9 beats rank 4 → M1 takes P_9 (3–12) · P_8 keeps waiting

t = 3 · The moment of the anomaly: M1 frees with P_8 and P_9 both READY. The plain list took P_8 and made P_9 wait until 6. The rank says P_9 — the critical path starts now and runs to 12.

Critical path first on Graham's DAG

4 machines · CP list · t = 12





list by rank: P_1 P_9 P_4 P_5 P_6 P_7 P_8 P_2 P_3 — a free machine takes the first READY process on it

The scheduler's view at t = 12

- t = 6: M2, M3, M4 free → P_8 → M2 (6–10)
- t = 10: M2 free, nothing left · t = 12: P_9 done
- makespan 12 = the span: optimal — the plain list gave 15 on these four machines

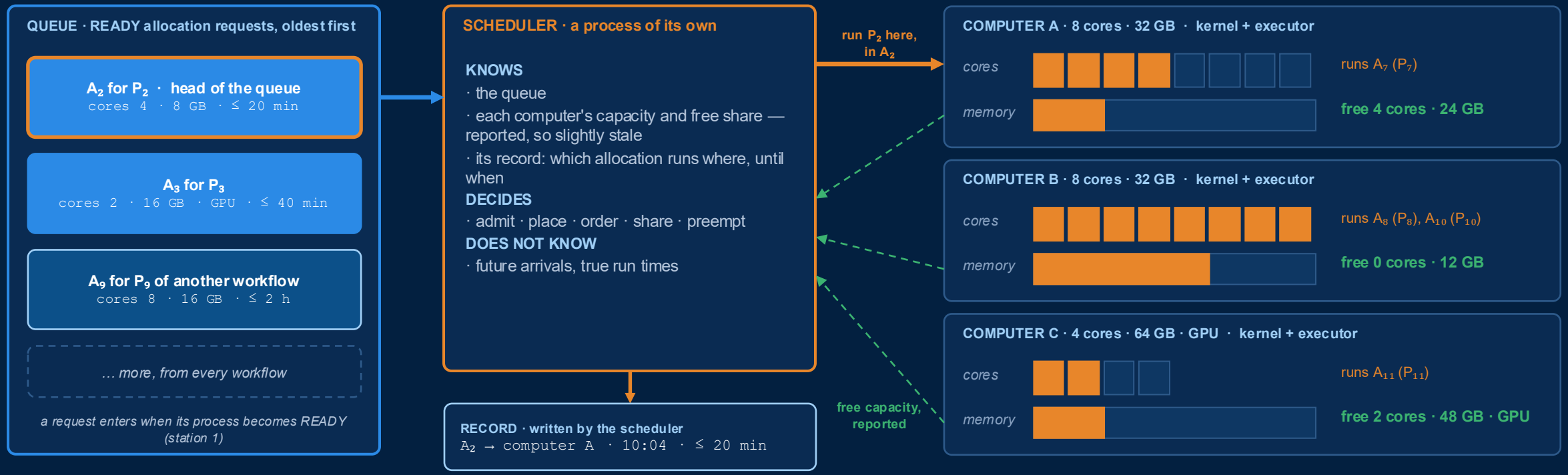
t = 12 · P_8 took M2 at 6 and finished at 10; P_9 finished at 12. Makespan 12 = the span, with four machines. P_8 paid three units of waiting; the workflow gained three.

plain list, 4 machines: 15 ✗ **ranked list, 4 machines: 12 ✓ = the span**

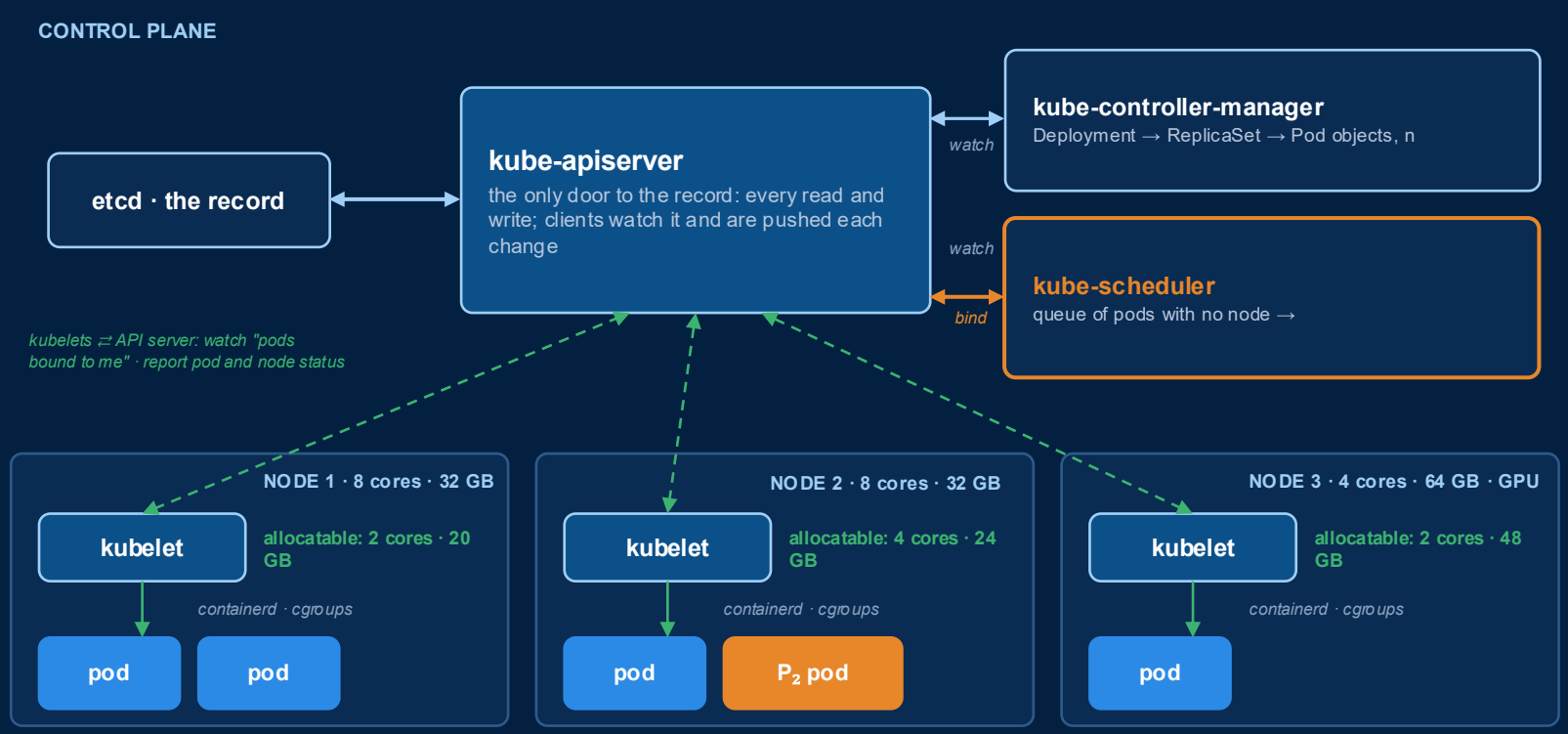
2 · Scheduling

Where and when?

A system model for the scheduler: one queue, computers that report free capacity, one decider



Kubernetes



One pod, from apply to running

1 kubectl apply a Deployment → the API server writes it to etcd.

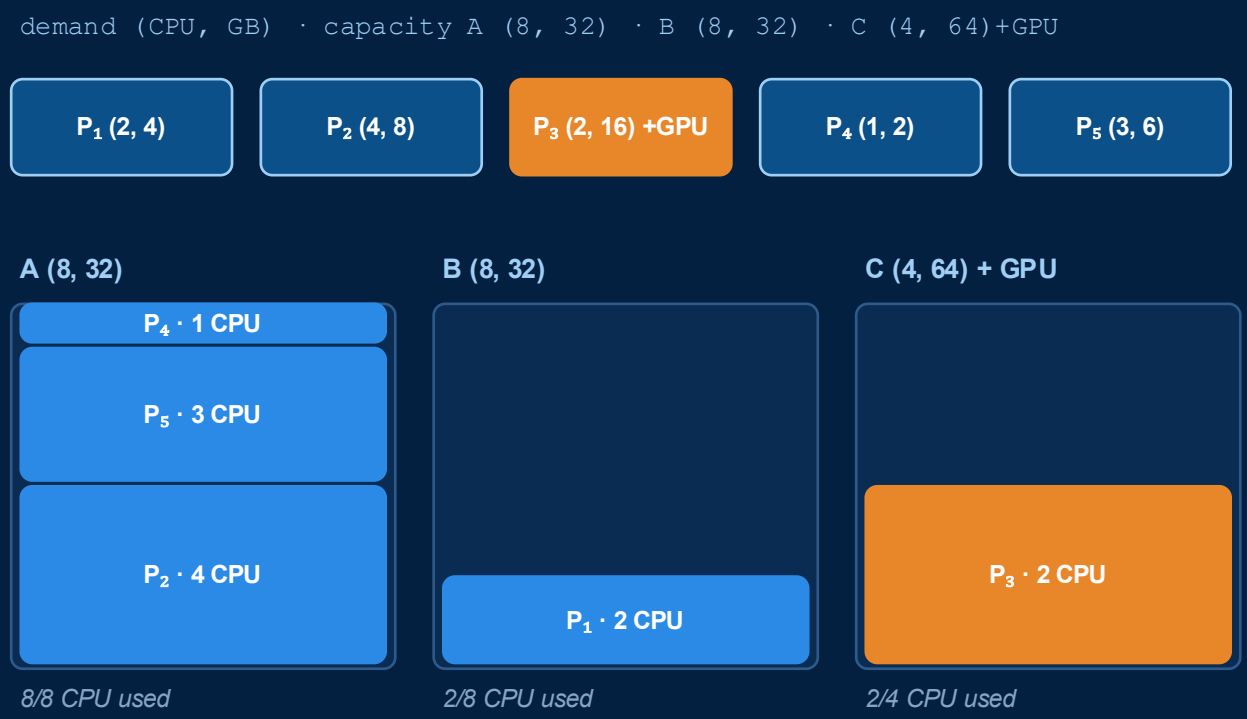
2 Controllers: Deployment → ReplicaSet → Pod objects with nodeName empty: pending.

3 kube-scheduler pops the pod from its queue. Filter: NodeResourcesFit, taints, affinity. Score: LeastAllocated, ImageLocality, topology spread. Bind: nodeName = node-2.

4 node-2's kubelet sees a pod bound to it: pulls images, starts containers, sets cgroups from requests and limits.

5 The kubelet reports pod and node status; a controller recreates a dead pod (station 5).

Placement is bin packing: demand vectors into capacity vectors



The problem

Place every item so no bin overflows, using as few bins as possible. NP-hard already in one dimension — so every scheduler runs a heuristic.

Pack or spread?

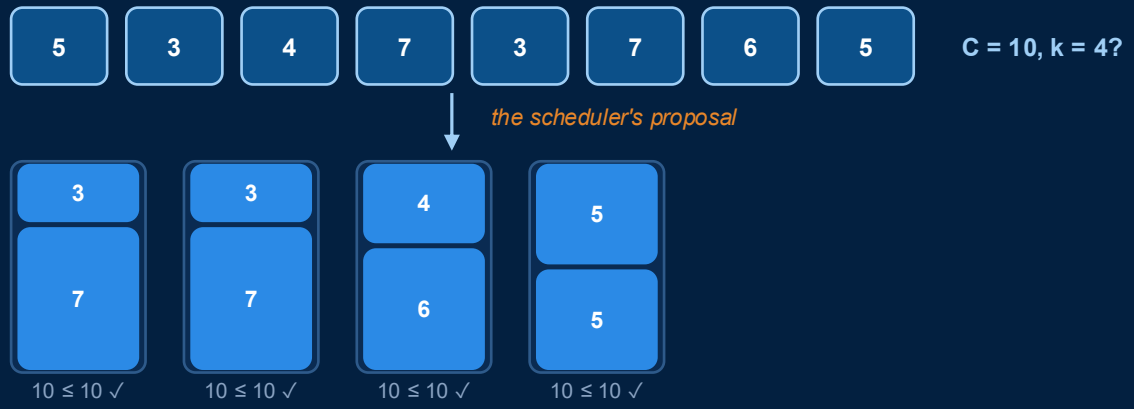
Pack: fewest machines, failures concentrate. Spread: survives machine loss, wastes capacity

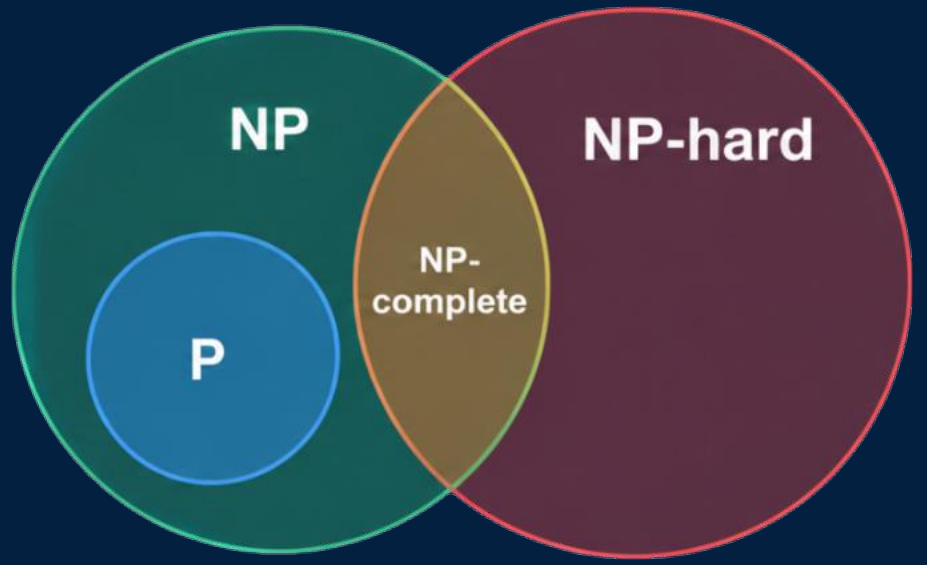
Why bin packing is NP-complete?

The scheduler's question, as a yes/no problem

SCHEDULER: given allocation requests A_1 … A_n (cores asked), k servers of C cores —
is there a placement in which no server is over C?
Computer scientists call this BIN PACKING.

The scheduler proposes a placement; checking it costs one sum per server

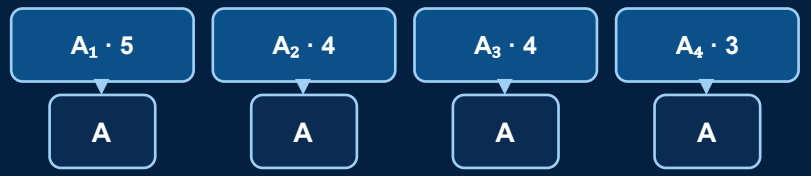




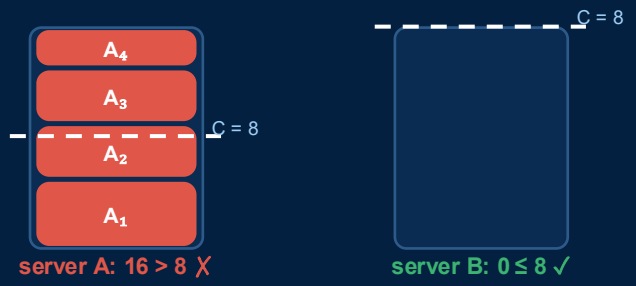
Brute-force placement, one iteration at a time: four allocations, two servers of 8 cores

THE LOOP

```
for k = 1, 2, 3, …:
  for a in {A, B, …}ⁿ:          # kⁿ candidates
    load[j] = Σ size[i] where a[i] = j
    if all load[j] ≤ C:         # n adds, k compares
      return k, a                 # yes
```



a = A A A A (a binary counter: A = 0, B = 1)



candidate 1: where each allocation goes

allocation	cores	server A	server B
A_1	5	**5**	—
A_2	4	**4**	—
A_3	4	**4**	—
A_4	3	**3**	—
load	16	16 > 8 ✗	0 ≤ 8 ✓

orange: the allocation the counter moved. Check: add each column, compare with C = 8.

the log: candidates tried so far

#	$A_1A_2A_3A_4$	load A	load B	verdict
1	AAAA	16	0	A over

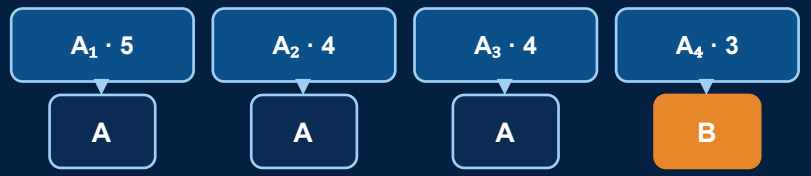
15 candidates left for k = 2 · each check: 4 additions, 2 comparisons

Iteration 1 · Everything on A. Check the loads: A carries 16 > 8. Reject and move to the next assignment — the counter ticks like a binary number.

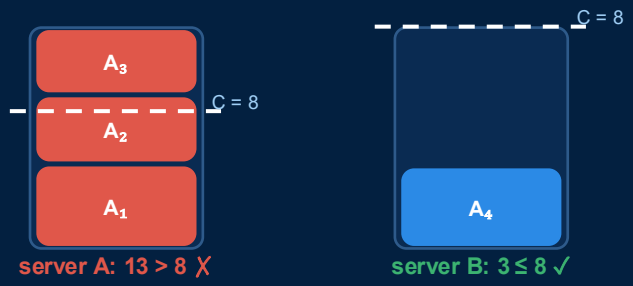
Brute-force placement, one iteration at a time: four allocations, two servers of 8 cores

THE LOOP

```
for k = 1, 2, 3, …:
  for a in {A, B, …}^n:          # k^n candidates
    load[j] = Σ size[i] where a[i] = j
    if all load[j] ≤ C:          # n adds, k compares
      return k, a                  # yes
```



a = A A A B (a binary counter: A = 0, B = 1)



candidate 2: where each allocation goes

allocation	cores	server A	server B
A_1	5	**5**	—
A_2	4	**4**	—
A_3	4	**4**	—
A_4	3	—	**3**
load	16	13 > 8 ✗	3 ≤ 8 ✓

orange: the allocation the counter moved. Check: add each column, compare with C = 8.

the log: candidates tried so far

#	$A_1A_2A_3A_4$	load A	load B	verdict
1	AAAA	16	0	A over
2	AAAB	13	3	A over

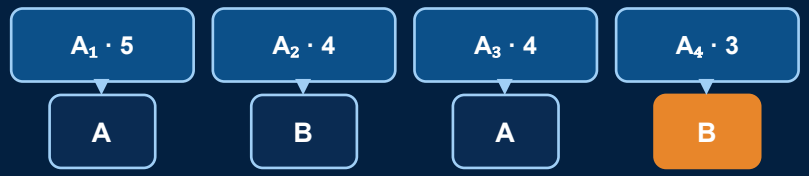
14 candidates left for k = 2 · each check: 4 additions, 2 comparisons

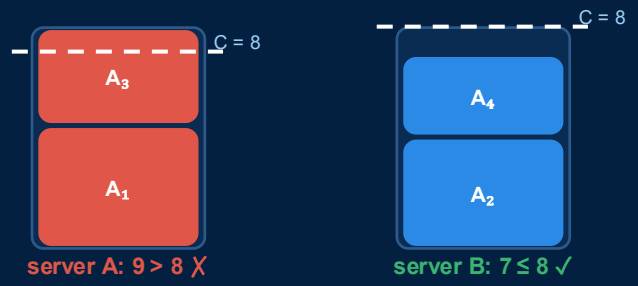
Iteration 2 · Move A_4 to B: A carries 13 > 8. Reject. The loop learns nothing from a rejection except that this one candidate failed.

Brute-force placement, one iteration at a time: four allocations, two servers of 8 cores

THE LOOP

```
for k = 1, 2, 3, …:
  for a in {A, B, …}ⁿ:          # kⁿ candidates
    load[j] = Σ size[i] where a[i] = j
    if all load[j] ≤ C:         # n adds, k compares
      return k, a                 # yes
```





candidate 6: where each allocation goes

allocation	cores	server A	server B
A_1	5	**5**	—
A_2	4	—	**4**
A_3	4	**4**	—
A_4	3	—	**3**
load	16	9 > 8 ✗	7 ≤ 8 ✓

orange: the allocation the counter moved. Check: add each column, compare with C = 8.

the log: candidates tried so far

#	$A_1A_2A_3A_4$	load A	load B	verdict
1	AAAA	16	0	A over
2	AAAB	13	3	A over
3	AABA	12	4	A over
4	AABB	9	7	A over
5	ABAA	12	4	A over
6	ABAB	9	7	A over

10 candidates left for k = 2 · each check: 4 additions, 2 comparisons

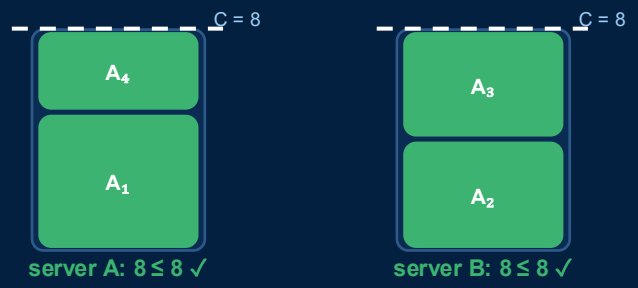
Iterations 3–6 · Four more rejections, all for the same reason: server A over 8. Each cost the same four additions and two comparisons.

Brute-force placement, one iteration at a time: four allocations, two servers of 8 cores

THE LOOP

```
for k = 1, 2, 3, …:
  for a in {A, B, …}ⁿ:          # kⁿ candidates
    load[j] = Σ size[i] where a[i] = j
    if all load[j] ≤ C:         # n adds, k compares
      return k, a                 # yes
```





candidate 7: where each allocation goes

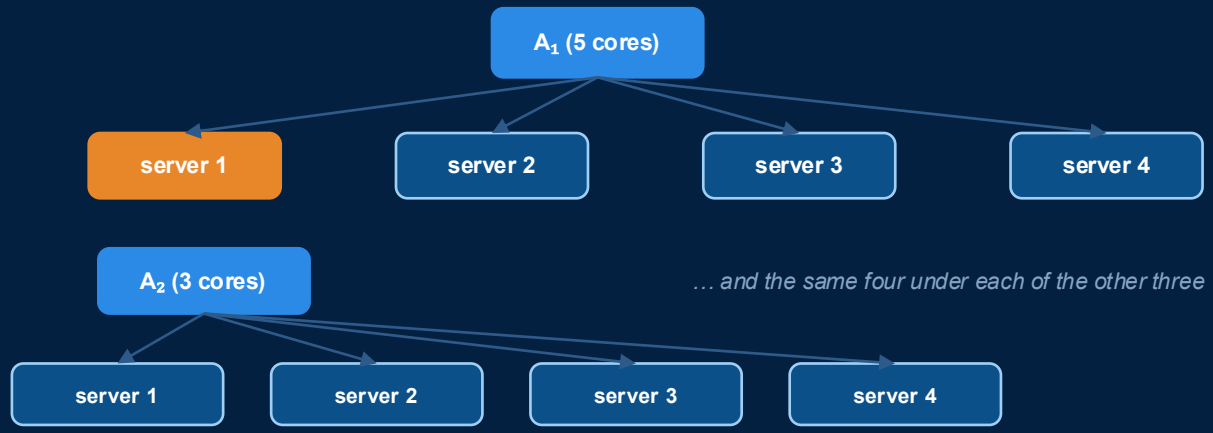
allocation	cores	server A	server B
A_1	5	**5**	—
A_2	4	—	**4**
A_3	4	—	**4**
A_4	3	**3**	—
load	16	8 ≤ 8 ✓	8 ≤ 8 ✓

the log: candidates tried so far

#	$A_1A_2A_3A_4$	load A	load B	verdict
1	AAAA	16	0	A over
2	AAAB	13	3	A over
3	AABA	12	4	A over
4	AABB	9	7	A over
5	ABAA	12	4	A over
6	ABAB	9	7	A over
7	ABBA	8	8	fits ✓

Why bin packing is NP-complete?

Eight allocations, four servers: every allocation has four choices



… and the same four under each of the other three

⋮ A_3 … A_8 each multiply the tree by four again

$4 \cdot 4 \cdot 4 \cdot 4 \cdot 4 \cdot 4 \cdot 4 \cdot 4 = 4^8 = 65\,536$ assignments

each checked with 8 additions and 4 comparisons against the capacity

The same count as the allocations grow (4 servers, 10^9 operations/s)

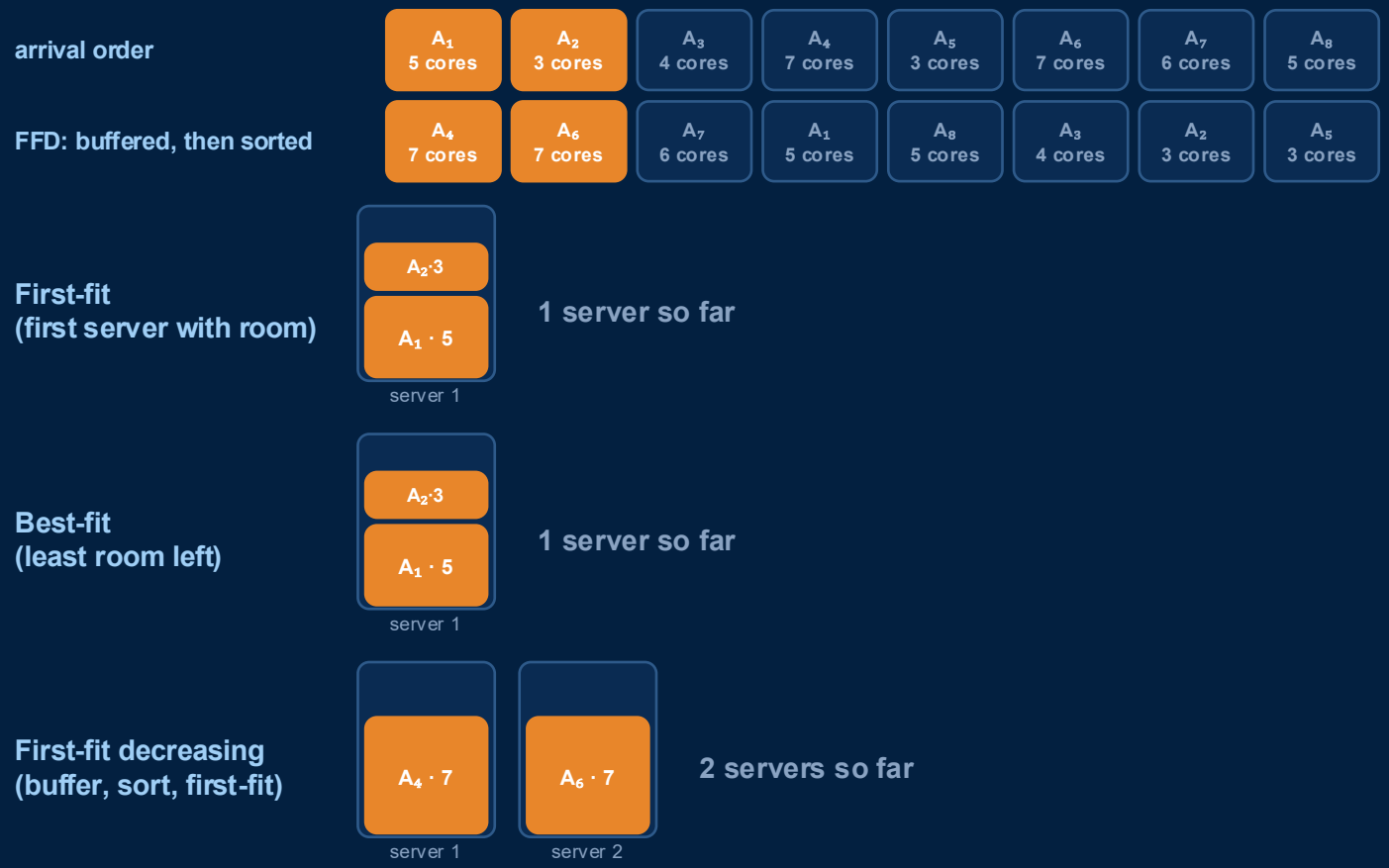
allocations n	assignments 4^n	operations · n	time
8	65 536	$5 \cdot 10^5$	0.5 ms
16	$4.3 \cdot 10^9$	$7 \cdot 10^{10}$	69 s
24	$2.8 \cdot 10^{14}$	$7 \cdot 10^{15}$	78 days
40	$1.2 \cdot 10^{24}$	$5 \cdot 10^{25}$	$1.5 \cdot 10^9$ years

Easy to check, hard to find

That asymmetry is what NP captures. NP-complete adds that bin packing is among the hardest such problems — which a reduction from PARTITION proves (reference deck).

Bin-packing heuristics: eight allocations, servers with 10 cores

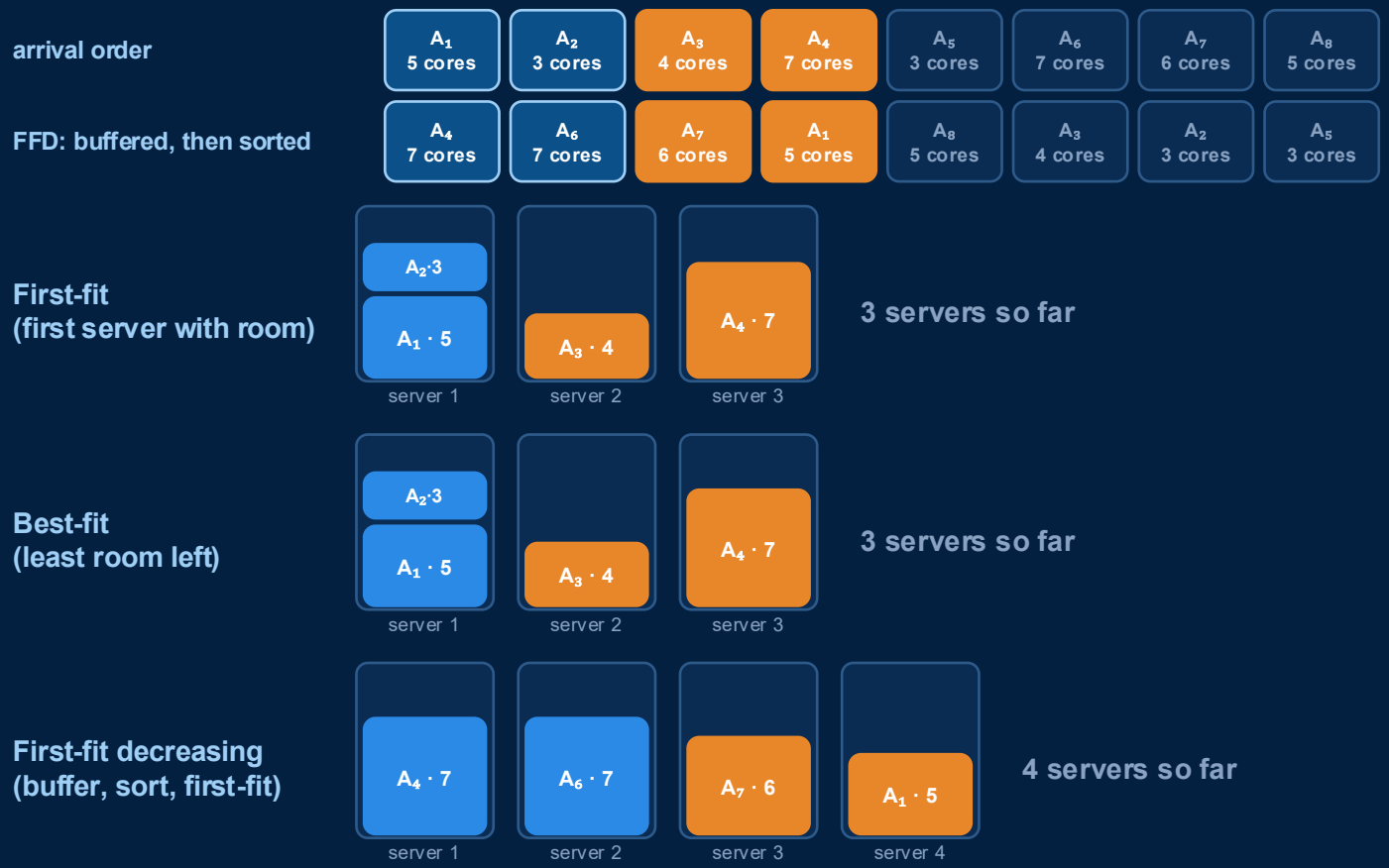
Each bin is one server with 10 cores; each block is one allocation request A_i, as tall as the cores it asks for. Eight requests need 40 cores → at least 4 servers.



next: allocations 3 and 4 arrive

Bin-packing heuristics: eight allocations, servers with 10 cores

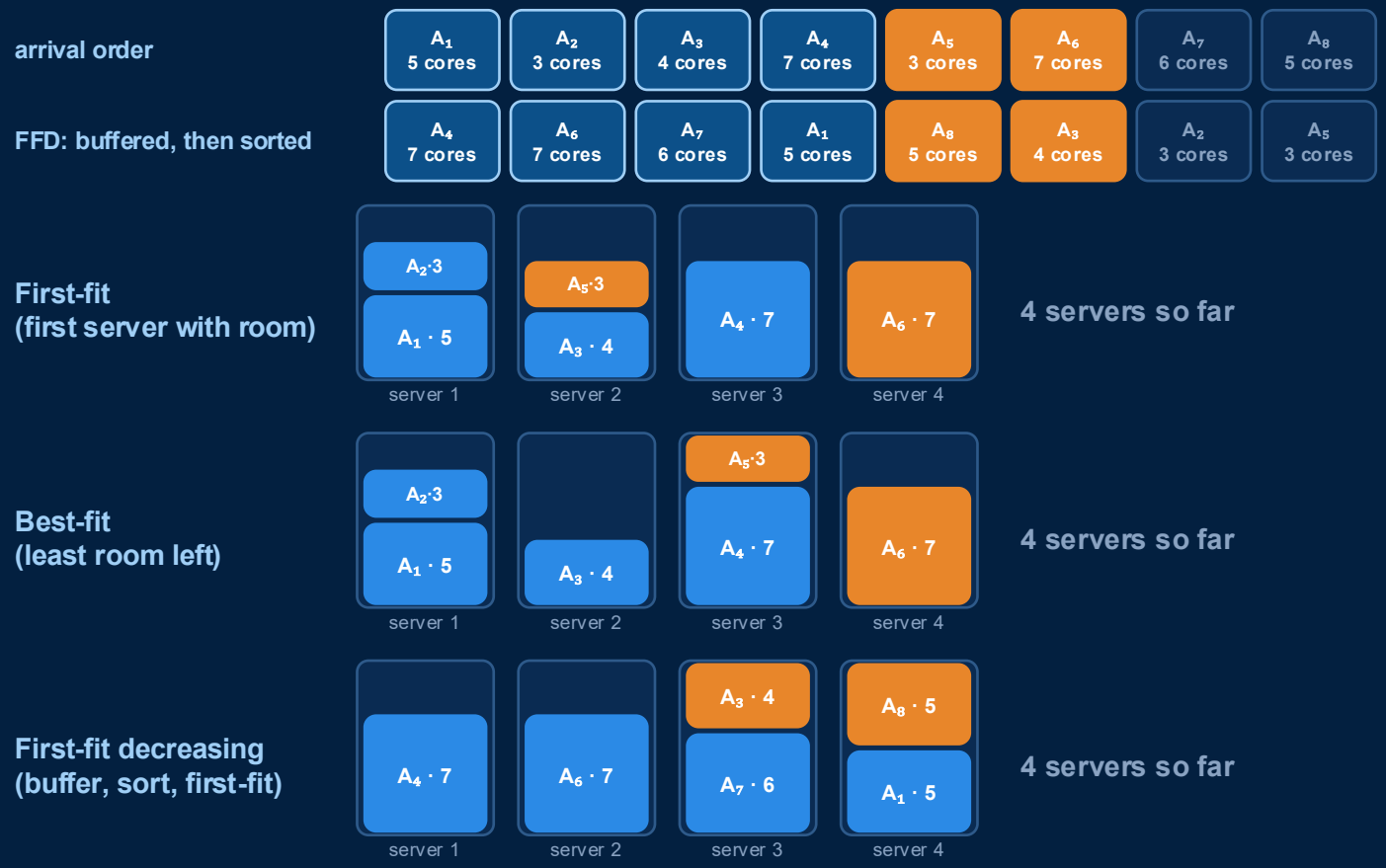
Each bin is one server with 10 cores; each block is one allocation request A_i, as tall as the cores it asks for. Eight requests need 40 cores → at least 4 servers.



next: allocations 5 and 6 arrive

Bin-packing heuristics: eight allocations, servers with 10 cores

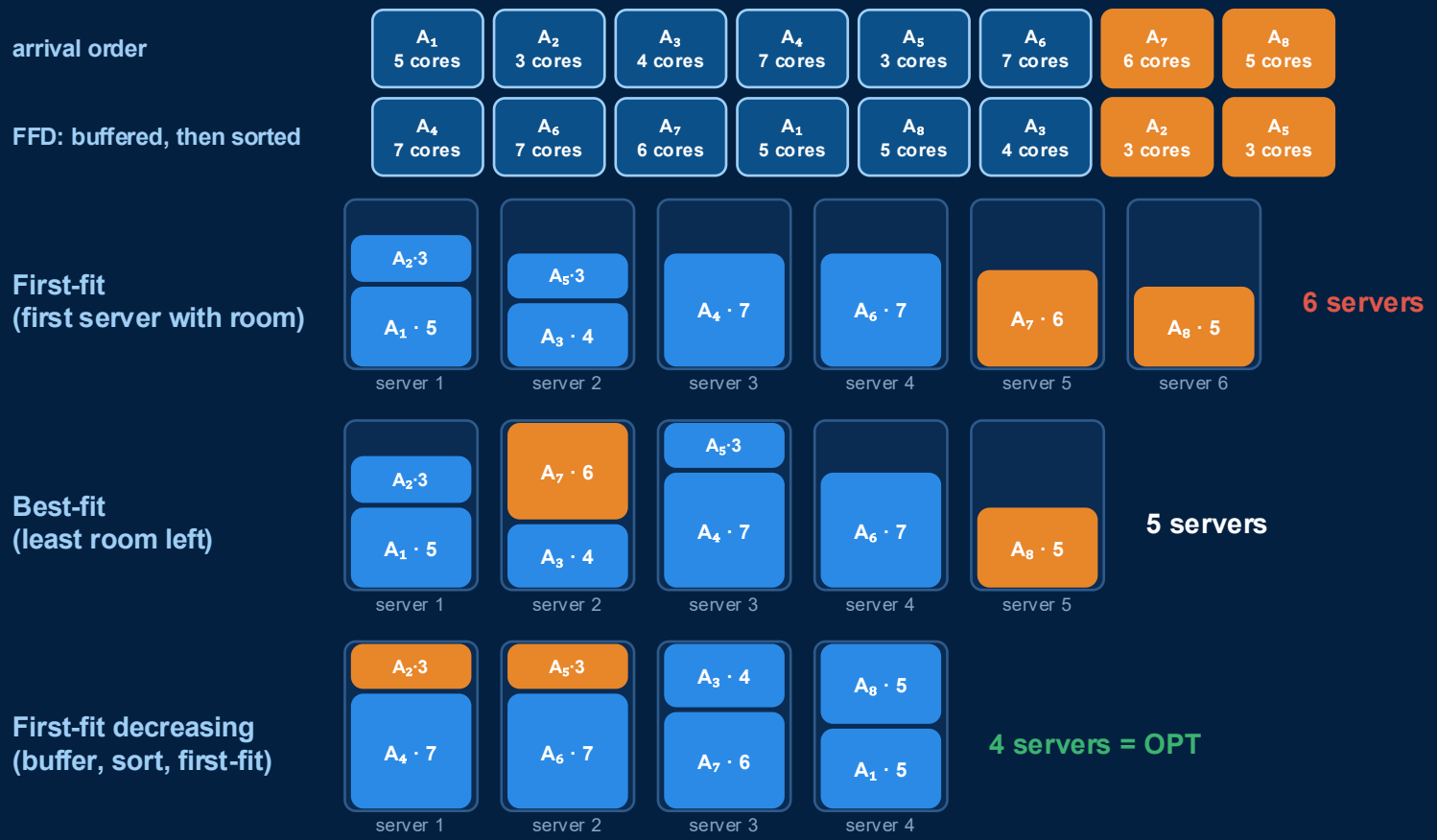
Each bin is one server with 10 cores; each block is one allocation request A_i, as tall as the cores it asks for. Eight requests need 40 cores → at least 4 servers.



next: allocations 7 and 8 arrive

Bin-packing heuristics: eight allocations, servers with 10 cores

Each bin is one server with 10 cores; each block is one allocation request A_i, as tall as the cores it asks for. Eight requests need 40 cores → at least 4 servers.



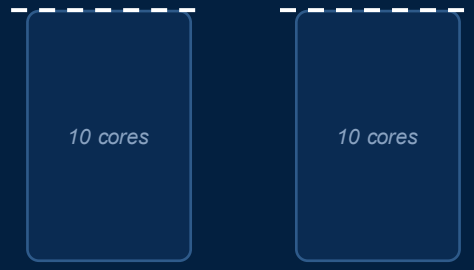
Genetic algorithm (meta-heuristic)

The placement question again, small enough to evolve by hand

ALLOCATIONS

A_1	A_2	A_3	A_4	A_5	A_6
5 cores	4 cores	4 cores	3 cores	3 cores	1 core

SERVERS · A and B, 10 cores each · 5 + 4 + 4 + 3 + 3 + 1 = 20 cores asked



server A: empty server B: empty

which allocation on which server, so that no server is over 10?

THE SCORE OF A PLACEMENT

Its over-allocation: for each server, the cores above 10, summed. 0 means the placement fits; 3 means three cores were promised that do not exist. Lower is better.

The algorithm will need nothing else about the problem than this score.

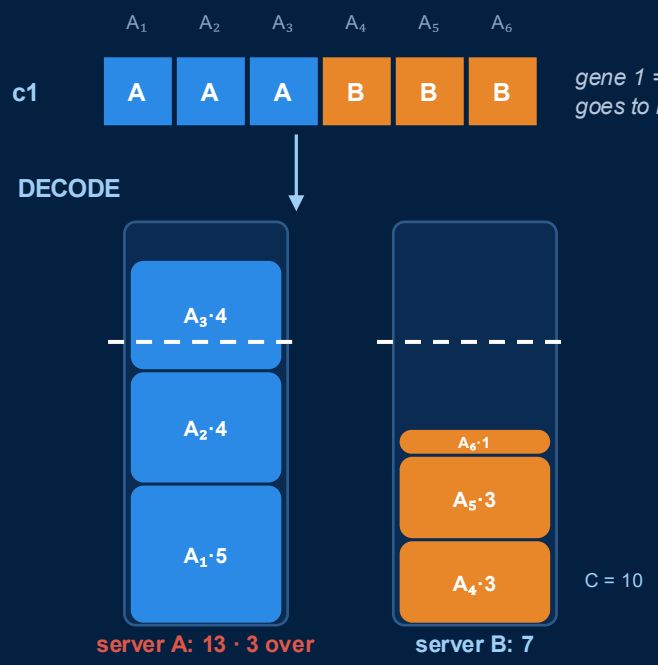
Why a genetic algorithm here

$2^6 = 64$ possible placements. Brute force (earlier) tries them all. A genetic algorithm keeps a handful, scores them, and breeds better ones — and works the same way when brute force is impossible.

Step 1 of 10 · The problem: six allocations, two servers of 10 cores. A placement is scored by its over-allocation — the cores above capacity, summed over servers. 0 = it fits.

Genetic algorithm (meta-heuristic)

One placement, written as a string



gene 1 = A: allocation A_1 goes to server A; gene 4 = B: A_4 goes to B …

server A: 5 + 4 + 4 = 13 → 3 over
server B: 3 + 3 + 1 = 7 → 0 over
score = 3 + 0 = 3

CHROMOSOME · GENE · FITNESS

CHROMOSOME: one candidate placement written as a string — here six letters.
GENE: one position of the string: the server that one allocation goes to.
DECODE: put every allocation on its server and read the loads.
FITNESS: the decoded chromosome's score — its over-allocation, 3 for c1. Lower is fitter; 0 fits.

Step 2 of 10 · A chromosome is a placement written as a string: gene i = the server of allocation A_i. Decode it, read the loads: c1 = AAABBB puts 13 on A — 3 cores over — and scores 3.

Genetic algorithm (meta-heuristic)

GENERATION 0 · four random chromosomes



POPULATION · GENERATION

POPULATION: several chromosomes at once — four here, drawn at random; real runs keep 50–500.

GENERATION: one round of the loop — score, select, cross, mutate — producing the next population.

No chromosome is known to be good yet.

Step 3 of 10 · Generation 0: a population of four random chromosomes — four placements guessed without looking.

Genetic algorithm (meta-heuristic)

GENERATION 0 · decoded and scored

	A_1	A_2	A_3	A_4	A_5	A_6	load A	load B	over
c1	A	A	A	B	B	B	13	7	3
c2	A	A	B	A	B	B	12	8	2
c3	A	B	A	A	A	B	15	5	5
c4	B	B	B	A	B	A	4	16	6

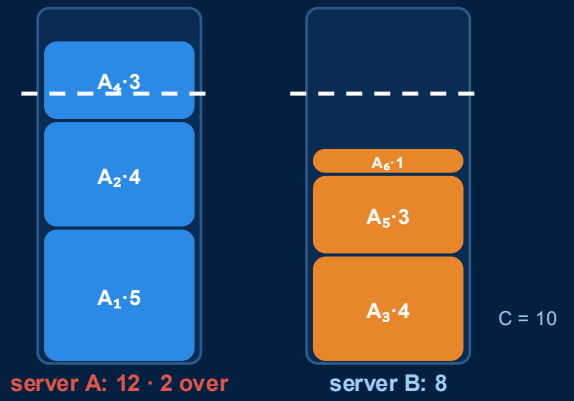
best 2 · mean 4 (0 = fits: no server over its 10 cores)

EVALUATE

Decode every chromosome and read its score: 3, 2, 5, 6 cores over. This is the only step that knows what servers and cores are; the rest of the loop only shuffles letters.

Best so far: c2, 2 cores over. Mean 4. None fits yet.

c2 = AABABB, decoded



Step 4 of 10 · Evaluate: decode each chromosome and score it. c2 is the best, 2 cores over; c4 the worst, 6 over; mean 4. None fits yet.

Genetic algorithm (meta-heuristic)

GENERATION 0 · ranked

	A_1	A_2	A_3	A_4	A_5	A_6	load A	load B	over	
c1	A	A	A	B	B	B	13	7	3	parent
c2	A	A	B	A	B	B	12	8	2	parent
c3	A	B	A	A	A	B	15	5	5	dropped
c4	B	B	B	A	B	A	4	16	6	dropped

best 2 · mean 4 (0 = fits: no server over its 10 cores)

SELECTION

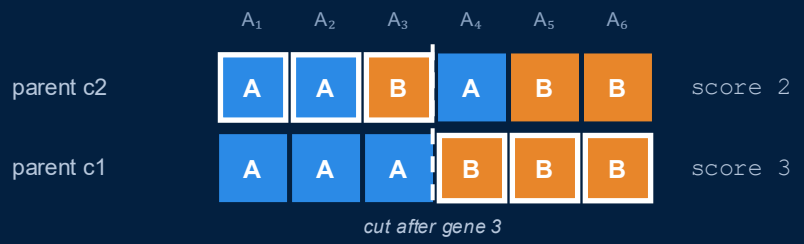
Decide who reproduces and who survives: the fit. Here the simplest rule — keep the best half: c2 (2 over) and c1 (3 over) become the parents; c3 (5) and c4 (6) are dropped.

Other rules: roulette (chance proportional to fitness), tournament (draw two at random, keep the better). All are pressure towards good chromosomes, not certainty.

Step 5 of 10 · Selection: the fit reproduce. Keep the best half — c2 and c1 become the parents; c3 and c4 are dropped.

Genetic algorithm (meta-heuristic)

CROSSOVER · cut both parents after gene 3, swap the tails



CROSSOVER

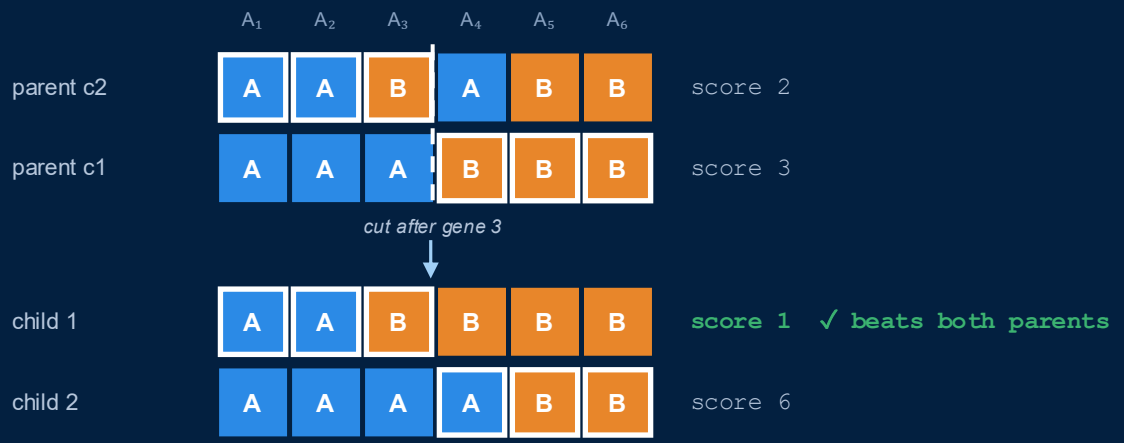
Make children from two parents: cut both chromosomes at the same random point and swap the tails — the child gets the head of one parent and the tail of the other.

The same cut in both keeps one gene per allocation, so every child is a valid placement.

Step 6 of 10 · Crossover: cut both parents at the same point — after gene 3 — so the head of one can be glued to the tail of the other.

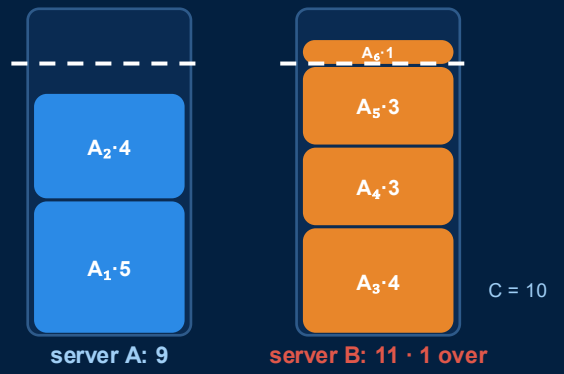
Genetic algorithm (meta-heuristic)

CROSSOVER · cut both parents after gene 3, swap the tails



child 1 = the head of c2 (A A B) + the tail of c1 (B B B); child 2 = the head of c1 (A A A) + the tail of c2 (A B B)

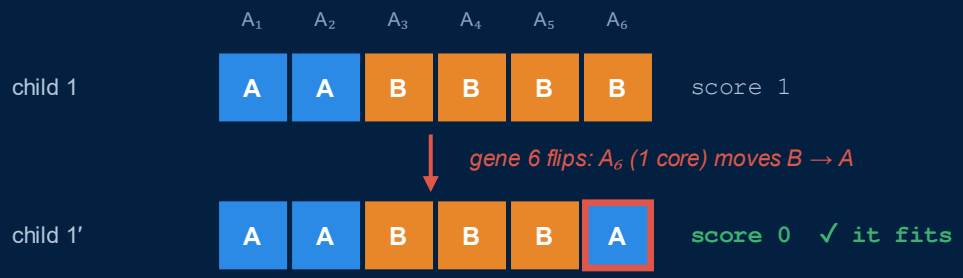
child 1 = AABBBB, decoded



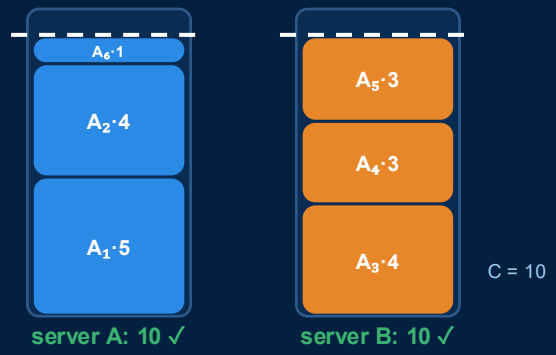
Step 7 of 10 · The children: child 1 = AAB|BBB is 1 core over — better than both parents. Child 2 = AAA|ABB is 6 over — worse than both. Crossover is blind; selection will sort them.

Genetic algorithm (meta-heuristic)

MUTATION · flip one gene at random — one allocation changes server



child 1′ decoded



$A_1 + A_2 + A_6 = 5 + 4 + 1 = 10$
$A_3 + A_4 + A_5 = 4 + 3 + 3 = 10$
nothing over capacity: score 0

MUTATION

Change one gene at random, rarely — one allocation moves to the other server. Here A_6 (1 core) moves from B to A and nothing is over capacity any more: score 0, the placement fits.

Rare on purpose: too much and the search is random; none and a move that no parent contains can never happen — crossover only recombines what exists.

Step 8 of 10 · Mutation: one gene flips at random. A_6 (1 core) moves from B to A: child 1′ = AABBBA scores 0 — nothing over capacity. It fits.

Genetic algorithm (meta-heuristic)

GENERATION 1 · survivors + children

	A_1	A_2	A_3	A_4	A_5	A_6	load A	load B	over	
child 1′	A	A	B	B	B	A	10	10	0	new
c2	A	A	B	A	B	B	12	8	2	
c1	A	A	A	B	B	B	13	7	3	
child 2	A	A	A	A	B	B	16	4	6	new

best 0 · mean 2.75 (0 = fits: no server over its 10 cores)

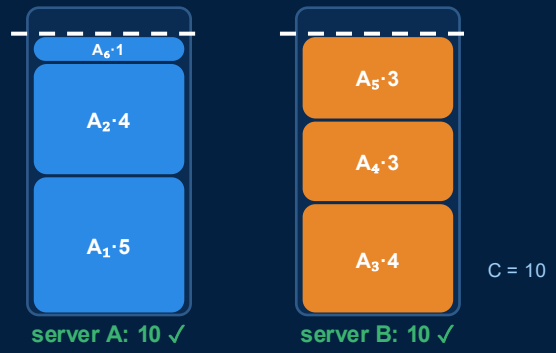
Step 9 of 10 · The next generation: the two parents survive, the two children replace the dropped ones. Best 2 → 0 cores over, mean 4 → 2.75, after one round.

Genetic algorithm (meta-heuristic)

THE LOOP



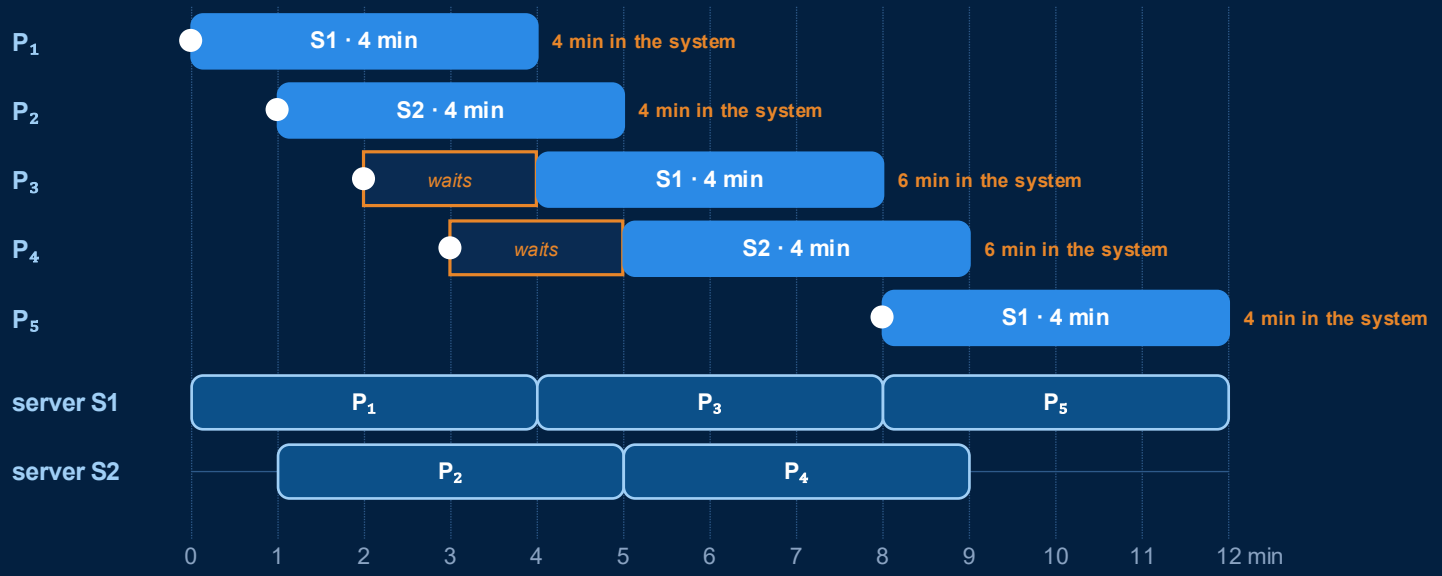
what it found: AABBBA



Step 10 of 10 · The loop: evaluate, select, cross, mutate, next generation — until the score stops improving. All it needed: a string for a placement, a score for a string.

Little's law: requests in the system = arrivals per minute × minutes each one spends in it

Two servers, 4 minutes of service per request; five requests arrive at 0, 1, 2, 3 and 8; a 12-minute window



$$L = \lambda \cdot W$$

requests in the system (waiting + running) = arrivals per minute × minutes each spends in it, arrival to completion.

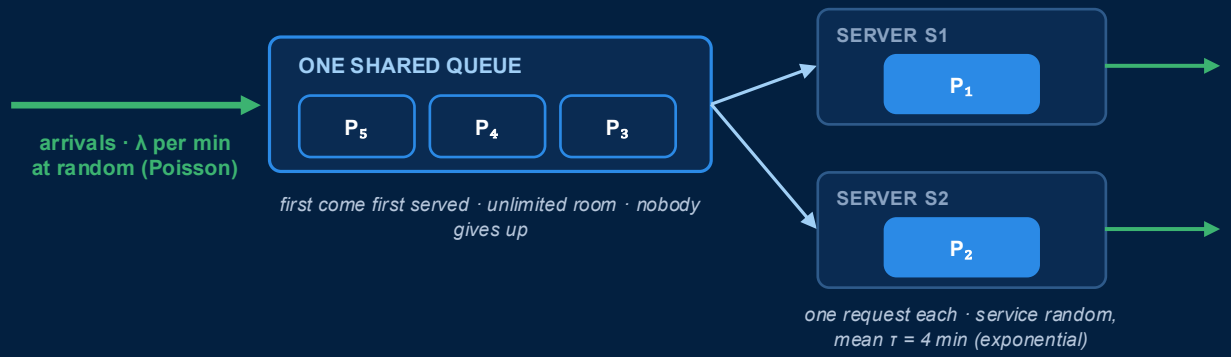
Little (1961): any stable system — arrivals below capacity, so the queue does not grow without bound — with any arrival pattern, any service order, any number of servers.

λ arrivals: 5 in 12 min = 0.42 per minute

W time in the system: 4, 4, 6, 6, 4 → 4.8 min on average (4 of service + 0.8 of waiting)

L in the system on average: 24 request-minutes / 12 min = 2.0 — and λ · W = 0.42 × 4.8 = 2.0 ✓

M/M/2 — one shared queue, two servers, memoryless arrivals and service



M — **arrivals at random**

independent, at a constant average rate λ — a Poisson process. Markovian = memoryless: what happened so does not change what comes next.

M — **service times at random**

exponential with mean τ = 4 min: many short requests, some long ones. Memoryless again: the time already served says nothing about the time left.

2 — **two servers**

one request each, so capacity = 2 / τ = 0.5 requests per minute = 30 per hour. ρ = λ · τ / 2 is the share of that capacity in use.

Stable

Arrivals below capacity: λ < 2/τ, i.e. ρ < 1. Then the queue does not grow without bound, departures equal arrivals in the long run, and the averages L and W exist. At ρ ≥ 1 there is no steady state — no finite mean wait.

The wait, from the model

$W_q = P(\text{wait}) \times E[\text{wait} \mid \text{you wait}]$

$P(\text{wait}) = 2\rho^2 / (1 + \rho)$ both servers busy when you arrive · 0.76 at ρ = 5/6

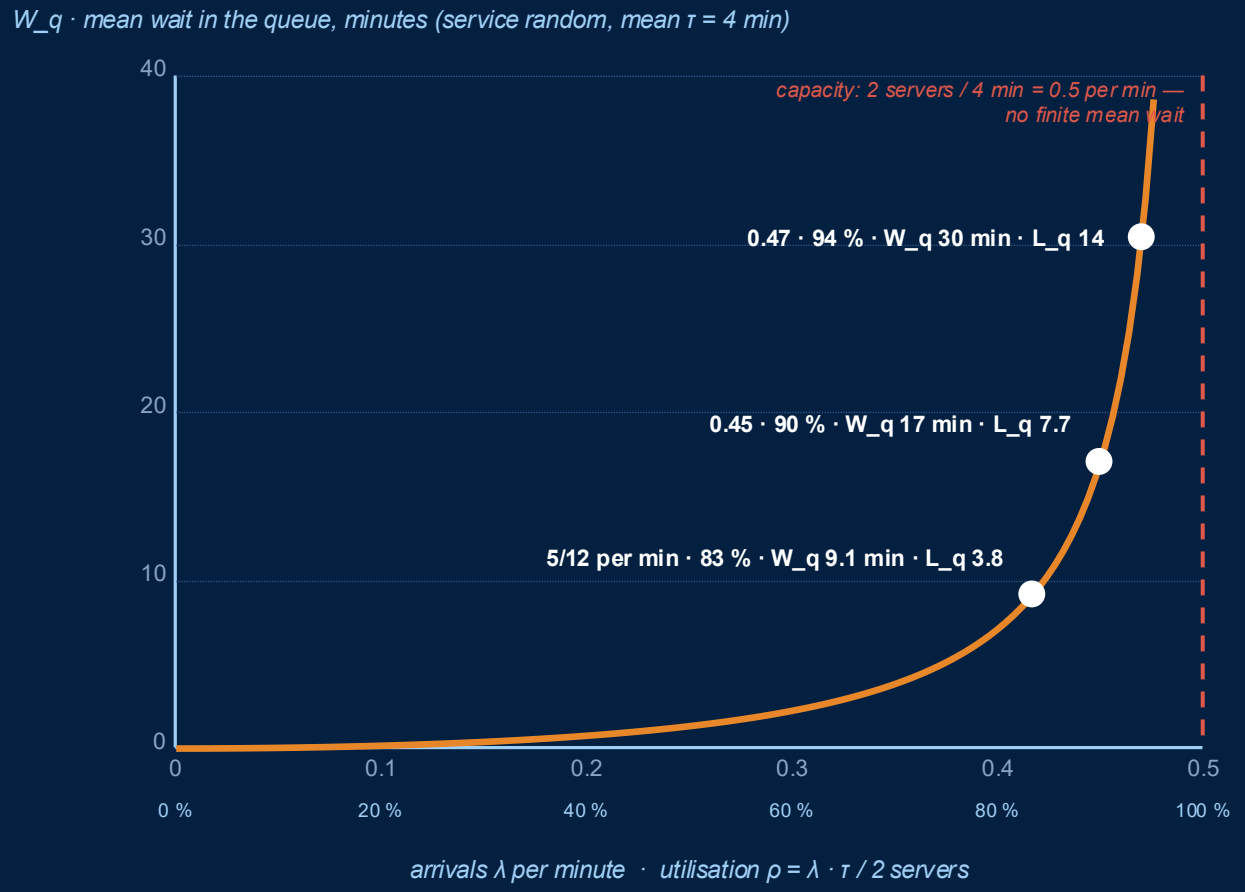
$E[\text{wait} \mid \text{wait}] = \tau / (2(1 - \rho))$ a server frees every τ/2, re-taken with prob. ρ · 12 min

$W_q = \rho^2 \cdot \tau / (1 - \rho^2)$ = 0.76 × 12 = 9.1 min at ρ = 5/6

Back to slide 60

W = W_q + τ = τ / (1 − ρ²) = 13.1 min. Little: L_q = λ · W_q = 3.8 waiting; L = λ · W = 5.4 in the system = 3.8 waiting + 1.67 running.

The utilisation curve: the two-server system as arrivals grow towards its capacity



From Little's law to the curve

Little relates L_q and W_q; it gives neither. The M/M/2 model (previous slide) gives W_q; Little then gives the rest:

$W_q = \rho^2 \cdot \tau / (1 - \rho^2)$ this curve · ρ = 5/6 → (25/36)·4/(11/36) = 9.1 min

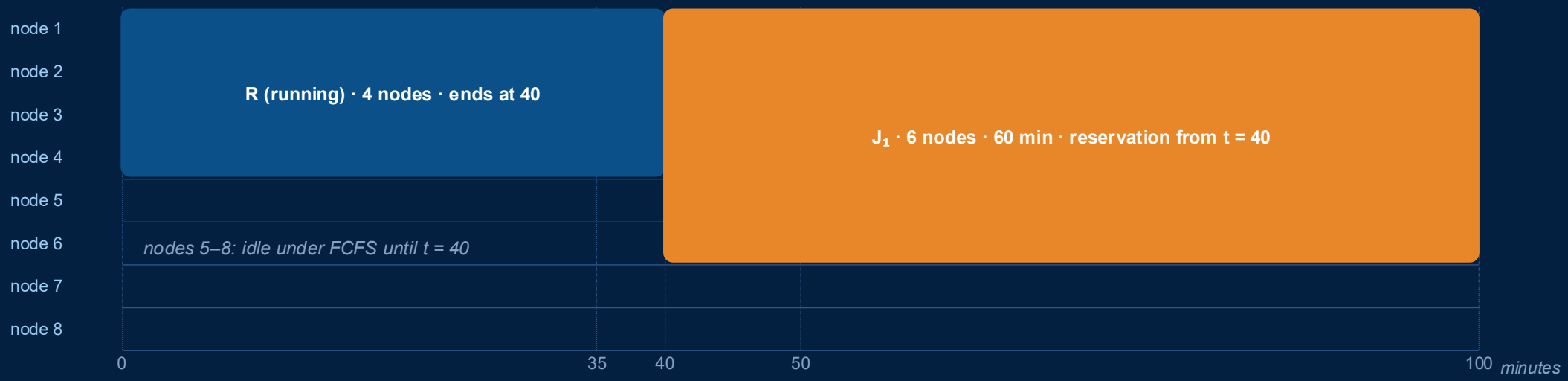
$L_q = \lambda \cdot W_q = 2\rho^3 / (1 - \rho^2)$ Little · 0.417 × 9.1 = 3.8 waiting

$L = L_q + \lambda \cdot \tau$ waiting + running · 3.8 + 1.67 = 5.4

arrivals	ρ	busy λ·τ	W_q	L_q = λ·W_q	L
0.2 / min	40 %	0.8	0.8 min	0.15	0.95
0.417 (5/12)	83 %	1.67	9.1 min	3.8	5.4
0.45 / min	90 %	1.8	17 min	7.7	9.5
0.47 / min	94 %	1.88	30 min	14	16
0.5 / min	100 %	2	no finite mean	—	—

Backfilling: keep the machines busy without delaying the head of the queue

8 nodes · queue: J_1 (6 nodes, 60 min) at the head, then J_2 (4 nodes, 35 min), then J_3 (3 nodes, 50 min) · R is running on 4 nodes until t = 40

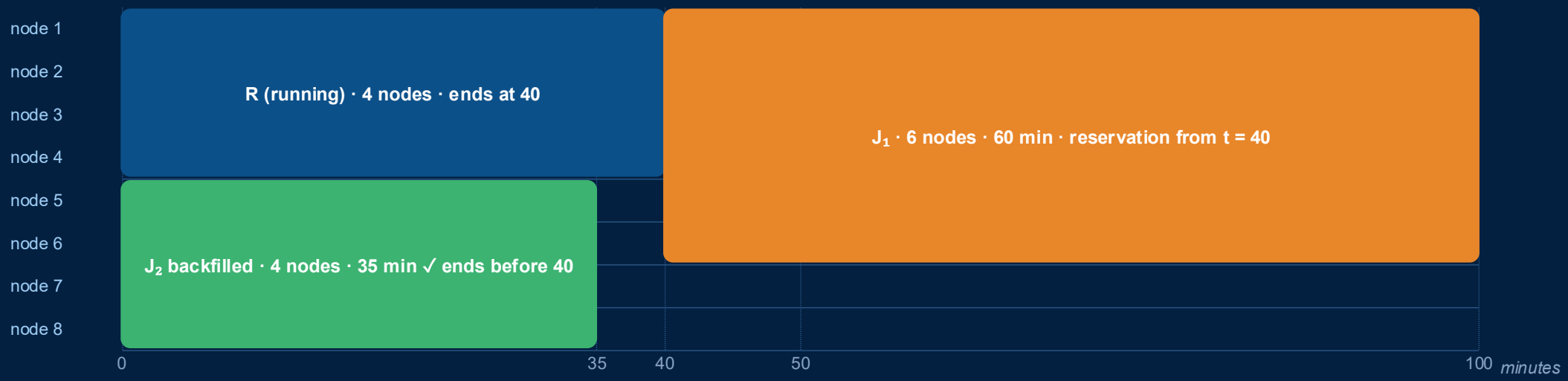


R runs on nodes 1–4 until t = 40. J_1 (head of the queue) needs 6 nodes for 60 min: the earliest it can start is t = 40. Reserve nodes 1–6 from 40 to 100.

What can start now without delaying J_1?

Backfilling: keep the machines busy without delaying the head of the queue

8 nodes · queue: J_1 (6 nodes, 60 min) at the head, then J_2 (4 nodes, 35 min), then J_3 (3 nodes, 50 min) · R is running on 4 nodes until t = 40



J_2 (4 nodes, 35 min) is behind J_1 in the queue. It fits on nodes 5–8 now and ends at 35 < 40: it does not touch J_1's reservation. Backfilled.

What can start now without delaying J_1?

Priority Decay

Priority Decay for Edge-Cloud Autoscaling

Casper Lundberg
Lulea University of Technology
Luleå, Sweden
casper.lundberg@ltu.se

Johan Kristiansson
Lulea University of Technology
Luleå, Sweden
johan.kristiansson@ltu.se

Ulf Bodin
Lulea University of Technology
Luleå, Sweden
ulf.bodin@ltu.se

***Abstract*—Industrial IoT workloads on the IoT-to-Edge-to-Cloud continuum process streams of prioritised batch jobs, with cloud bursting absorbing demand spikes that exceed local edge capacity. Autoscalers decide from infrastructure-observable state, yet the same event that triggers a spike often reveals that some queued work no longer requires immediate processing. Existing application-infrastructure channels either fix relevance at submission time or treat urgency as a passive function of elapsed time, missing relevance changes triggered by external events after submission. This paper presents priority decay, an application-originated, per-job, downward-only priority channel that operates on the orchestrator's queue. The application lowers the priority of queued jobs whose relevance has diminished while the autoscaler observes only the resulting queue state, with no coupling to decay as a mechanism in its decision engine. Evaluated in simulation on production logs from an operational seismic monitoring deployment with synthetic anomaly injection, the paper shows that priority decay enables the autoscaler scale against a queue whose priorities reflect the application's current view of relevance. Cloud commitments shrink under overload, queue-time compliance improves for non-deferred work, and the application gains selective control over which jobs wait under contention.**

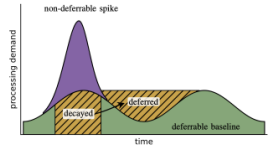


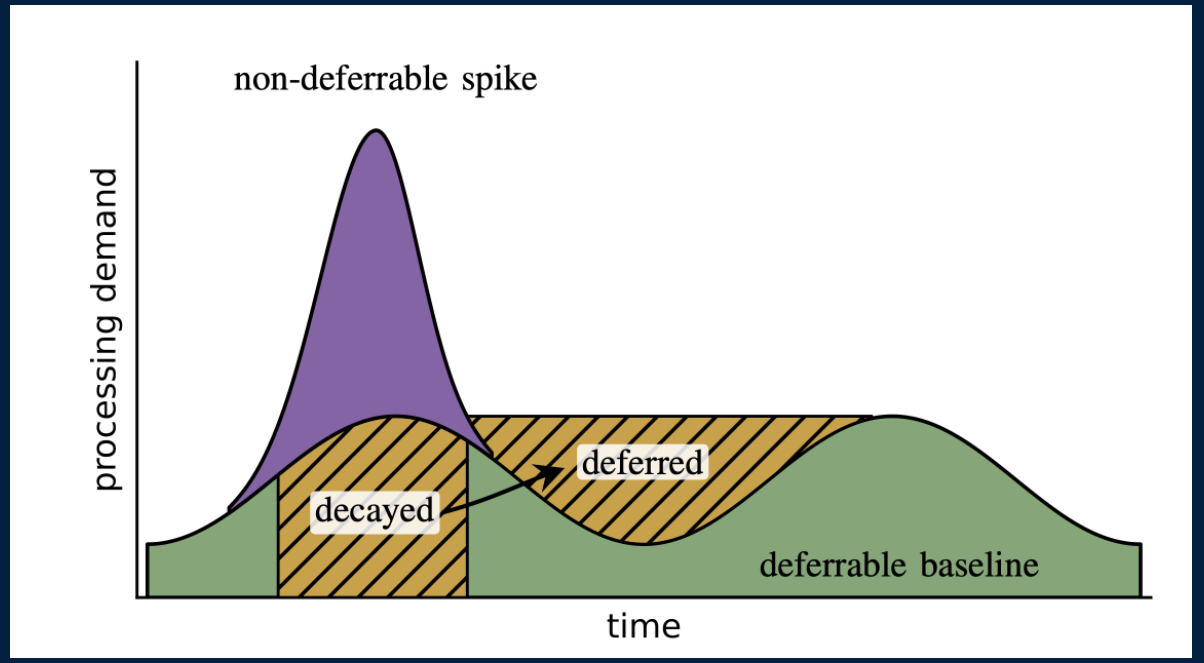
Fig. 1: Conceptual view of priority decay. When a spike arrives, the application signals the orchestrator to lower the priority of queued jobs that have become deferrable.

I. Introduction

Industrial IoT workloads increasingly run on the edge-cloud computing continuum, processing continuous streams of prioritised batch jobs across local edge resources backed by cloud overflow [1]. An orchestrator manages the queue and assigns jobs to workers, processing jobs, while an autoscaler adjusts the worker pool to keep pace with incoming demand. Local edge capacity is preferred for data locality and cost, and cloud resources are brought up as overflow when local capacity is insufficient, a pattern called cloud bursting [2], [3]. Under predictable demand this works well. When demand rises unexpectedly, the autoscaler must commit cloud capacity quickly but carefully, balancing cost and quality of service.

A class of these deployments shares three structural properties that the standard cloud-bursting response does not exploit: (1) work arrives as a stream of prioritised jobs on edge-cloud infrastructure, (2) the application continuously acquires runtime knowledge about which queued jobs still matter, knowledge the infrastructure cannot observe, and (3) the same event that causes a demand spike also reveals which queued work can wait, so capacity is most constrained exactly when the application knows the most about what to defer. Several domains exhibit this pattern. In seismic monitoring of underground mines, a detected seismic event floods the system with urgent follow-up jobs from the affected area while making queued jobs from unaffected areas deferrable, and the application knows this because it detected the event. In Earth observation, a natural disaster concentrates processing demand on the affected regions, while queued imagery from elsewhere can wait without losing analytical value. In agentic AI monitoring, an LLM-based agent observes infrastructure such as a Kubernetes cluster or a power grid, queues routine analysis work, and demotes that work when it detects a critical event worth reasoning about. In each case, the relevance of queued work is a runtime property held by the application, and the infrastructure has no way to observe it.

Current edge-cloud orchestrators do not use this information. They make scaling decisions from infrastructure-observable state, and the application-infrastructure channels that exist are not designed for per-job runtime reassessment triggered by external events. The Brownout [4] algorithm lets applications mark per-request computations as optional at design time and adjusts their inclusion probability at runtime when resources are scarce, but what is optional is fixed in advance rather than reassessed per queued job. Sun et al. [5] model task value as a decay function of total task latency and use this to drive offloading decisions, treating urgency as a passive function of elapsed time with no channel for the application to signal that relevance has changed for event-driven reasons. Design-time classification cannot capture knowledge that emerges only after jobs have been submitted, and passive time-based relaxation cannot capture relevance changes triggered by external events rather than by time. The result is overprovisioned cloud capacity for work the



Carbon-Aware Scheduling

Carbon-Aware Scheduling for Cyber-Physical Systems in the Edge-Cloud Continuum

Johan Kristiansson
Department of Computer Science, Electrical and Space Engineering
Luleå University of Technology
Luleå, Sweden
johan.kristiansson@ltu.se

Jerker Delsing
Department of Computer Science, Electrical and Space Engineering
Luleå University of Technology
Luleå, Sweden
jerker.delsing@ltu.se

Thomas Ohlson Timoudas
Department of Computer Science
RISE Research Institutes of Sweden
Stockholm, Sweden
thomas.ohlson.timoudas@ri.se

***Abstract*—Cyber-Physical Systems (CPS) powered by Artificial Intelligence (AI) have the potential to revolutionize industries by enabling advanced analytics and autonomous decision-making. To support resource-intensive applications, there is often a need to dynamically allocate additional compute resources. The Edge-Cloud Continuum enables allocation and deployment of workloads across platforms, including IoT devices, edge clusters, and cloud environments. However, the growing computational demands of these systems can unfortunately result in increased energy consumption and higher carbon emissions.**

This paper investigates the development of a carbon-aware scheduler for the Edge-Cloud Continuum, designed to optimize workload placement by balancing energy consumption, temporal variations in carbon intensity and resource availability. Key contributions of the paper include a spatiotemporal scheduling algorithm, a discrete-event simulator capable of replaying realistic workloads from the MIT SuperCloud dataset, and a comprehensive empirical evaluation.

Findings from the paper show substantial reductions in carbon emissions by prioritizing renewable energy sources and time-shifting workloads to periods of lower carbon intensity. However, when clusters operate under high utilization, time-shifting can inadvertently result in significantly higher emissions. In such scenarios, simpler greedy algorithms can be more effective.

I. Introduction

Cyber-Physical Systems (CPS) play an important role in modern society by seamlessly integrating computations with physical processes [1]. This integration enables real-time monitoring, automation, and decision-making across a wide range of industries. As these applications grow in complexity and scale, the demand for efficient and scalable computational platforms becomes increasingly important. The rapid advancement in AI further underscores the need for new computing platforms that can meet growing computational demands.

The Edge-Cloud Continuum [2] is a new paradigm in distributed computing that aims to unify edge and cloud computing. By dynamically distributing workloads, it becomes possible to optimize allocation of computational resources across platforms. For example, AI-driven CPS applications can process time-sensitive tasks on local edge servers and run resource-intensive tasks on remote cloud systems.

However, as the demand for computing continues to grow, so does energy consumption. Data centers are expected to account for a significant share of global energy use [3], thus contributing to increased greenhouse gas emissions and accelerating climate change. Developing technologies to reduce carbon emissions is therefore critical to reduce environmental impacts.

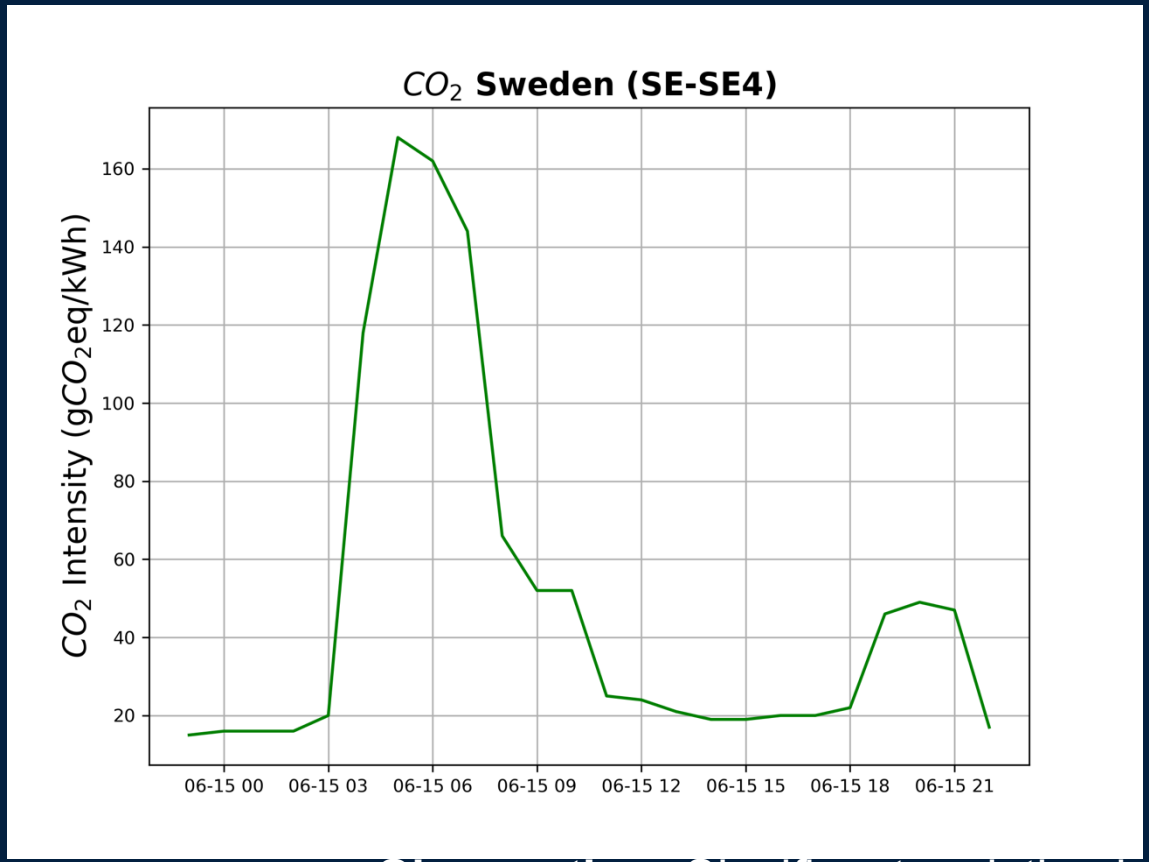
Lowering carbon emissions often correlates with energy efficiency and costs. For example, scheduling jobs in regions or at times when renewable energy sources, such as solar or wind power sources, are available can reduce both costs and environmental impact, as renewable energy is typically cheaper than fossil-fuel-based alternatives. Micro-grids CPS can optimize renewable energy usage by delaying task execution until sufficient renewable energy is available. Edge platforms can process data locally to maximize the use of renewable energy and offload execution to cloud resources in regions with lower carbon intensity, thereby minimizing reliance on carbon-intensive infrastructures.

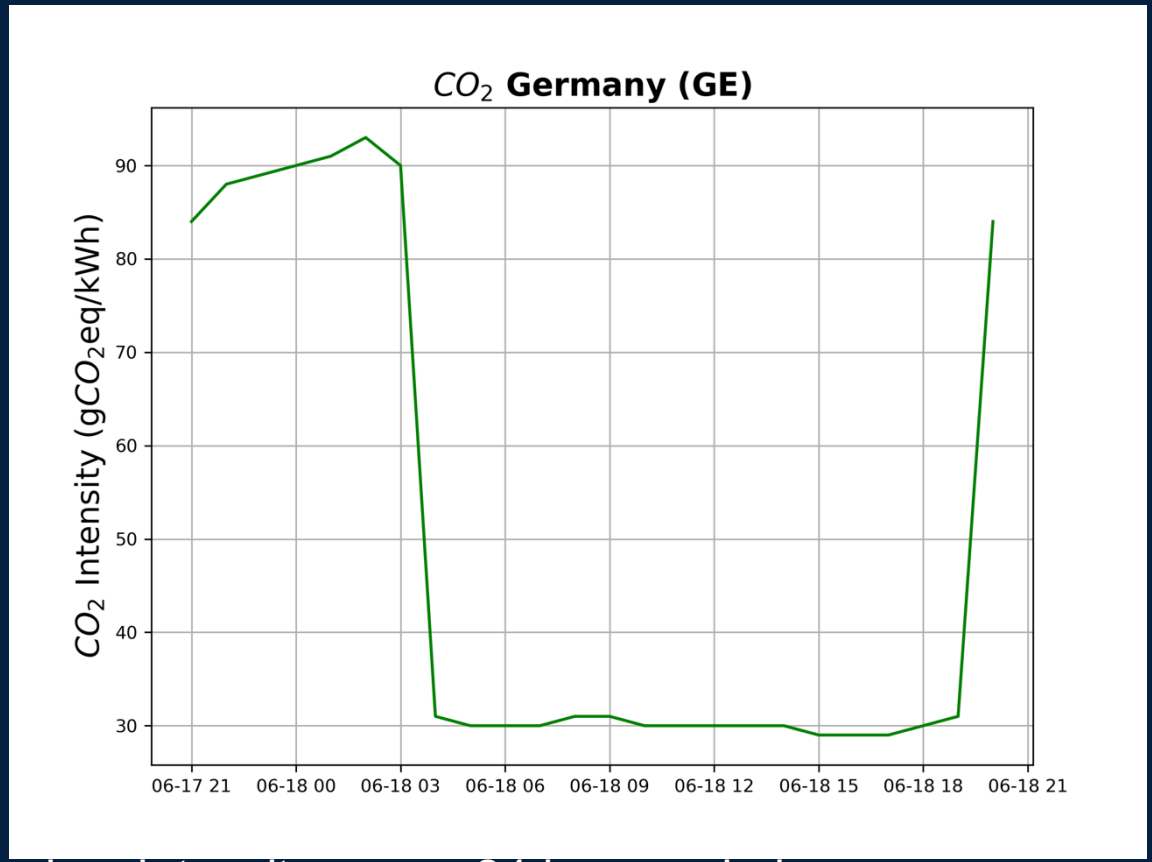
This paper addresses the challenges of designing a scheduler to minimize the carbon emissions of workloads across an Edge-Cloud Continuum. Specifically, the paper proposes a time-shifting algorithm to dynamically allocate workloads across platforms and delay executions to periods of lower carbon intensity. The long-term vision is to integrate the proposed scheduler with Arrowhead [4] and ColonyOS [5], a meta-operating system designed to build computing continuums.

The paper is organized as follows: Section II presents related work. Section III briefly describes the Arrowhead framework, followed by a definition of the research problem in Section IV. Section V presents the proposed scheduling algorithm, which is evaluated in Section VI using real-world data and simulations. Finally, Section VII concludes the paper and discusses future research.

Carbon Intensities 24h

Carbon Intensities predication for next 24h is available from e.g. Electricity Maps[1]

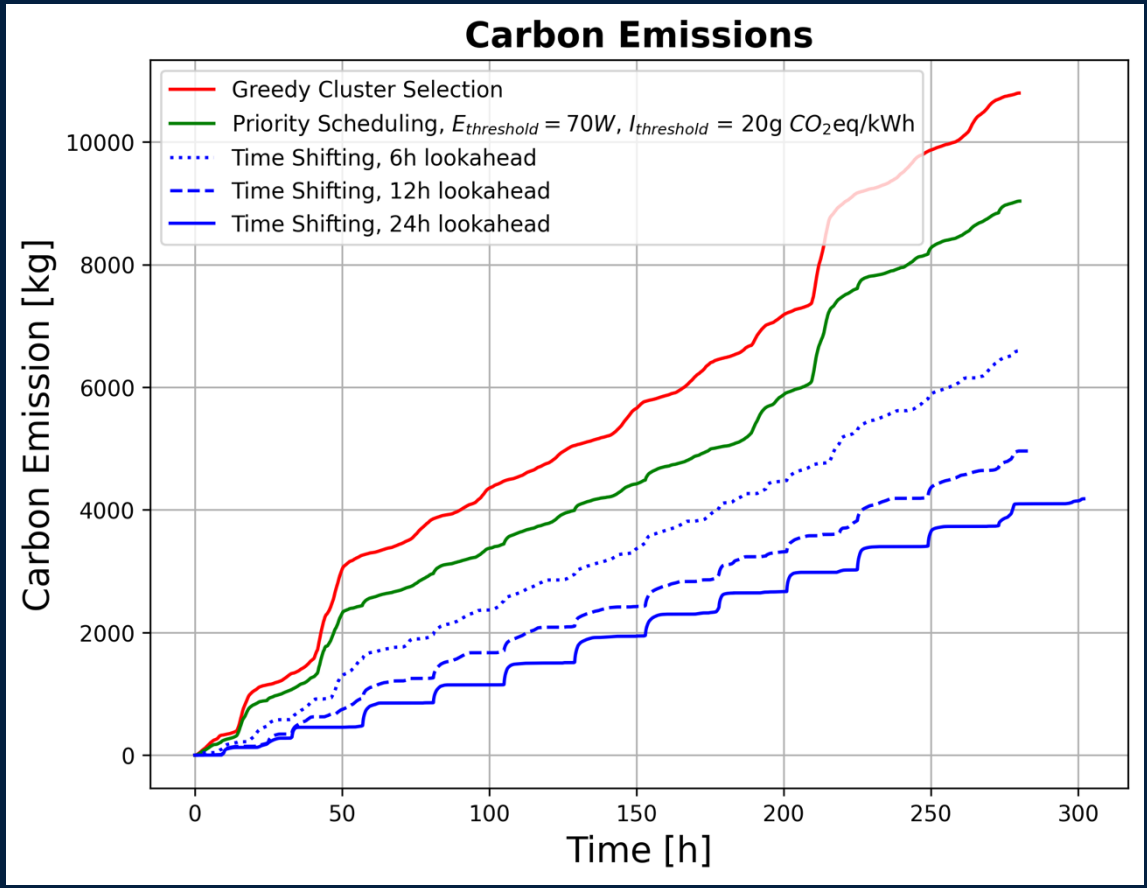


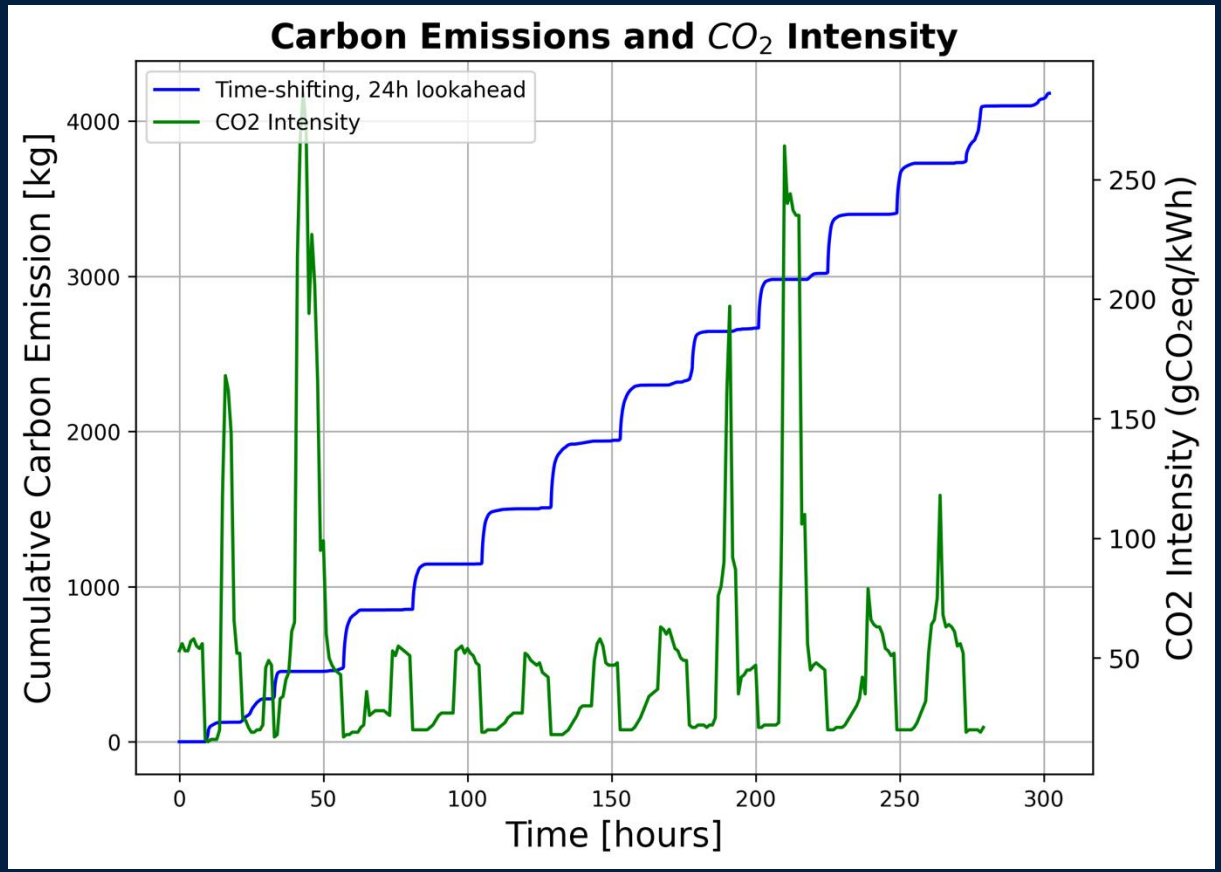


Observation: Significant variation in carbon intensity over a 24-hour period

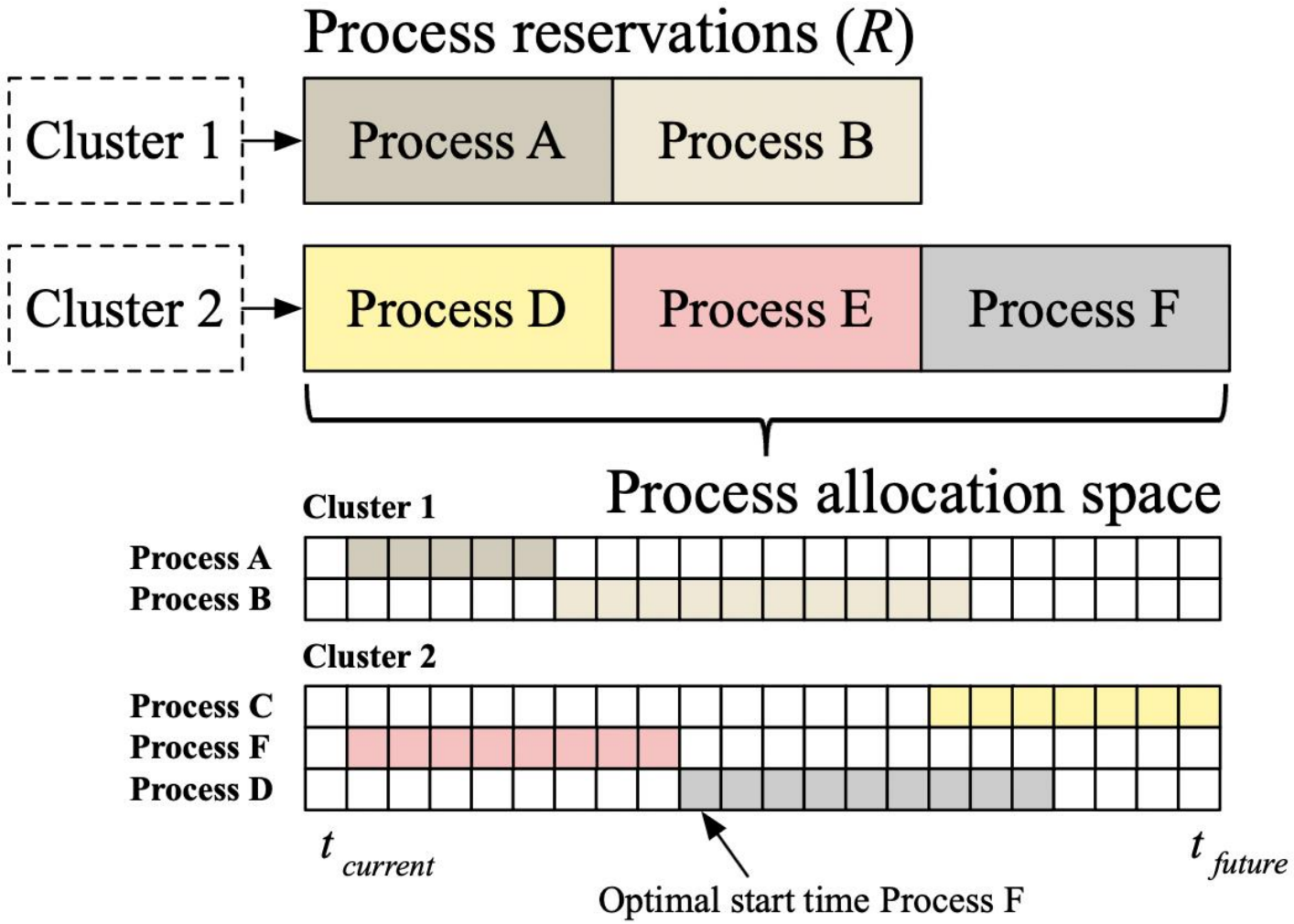
[1] https://www.electricitymaps.com

Result Experiment 1





Carbon-Aware Scheduling



Algorithm 2 Time Shifting

```
1:  function SCHEDULE(p, A)
2:      e_best ← ∞, c_best ← None, t_best ← None
3:      for c ∈ C do
4:          for t ∈ [t_current, t_current + d(p)] do
5:              if HASFREERESOURCES(c, t, p, A) then
6:                  e ← EMISSION(p, c, t)
7:                  if e < e_best then
8:                      e_best ← e, c_best ← c, t_best ← t
9:                  end if
10:             end if
11:         end for
12:     end for
13:     if c_best ≠ None then
14:         p.planned_time ← t_best
15:         p.planned_cluster ← c_best
16:         # Update Process Allocation matrix
17:         for t ∈ [t_best, t_best + p.duration) do
18:             A[c_best][t] ← A[c_best][t] + r(p)
19:         end for
20:     end if
21: end function

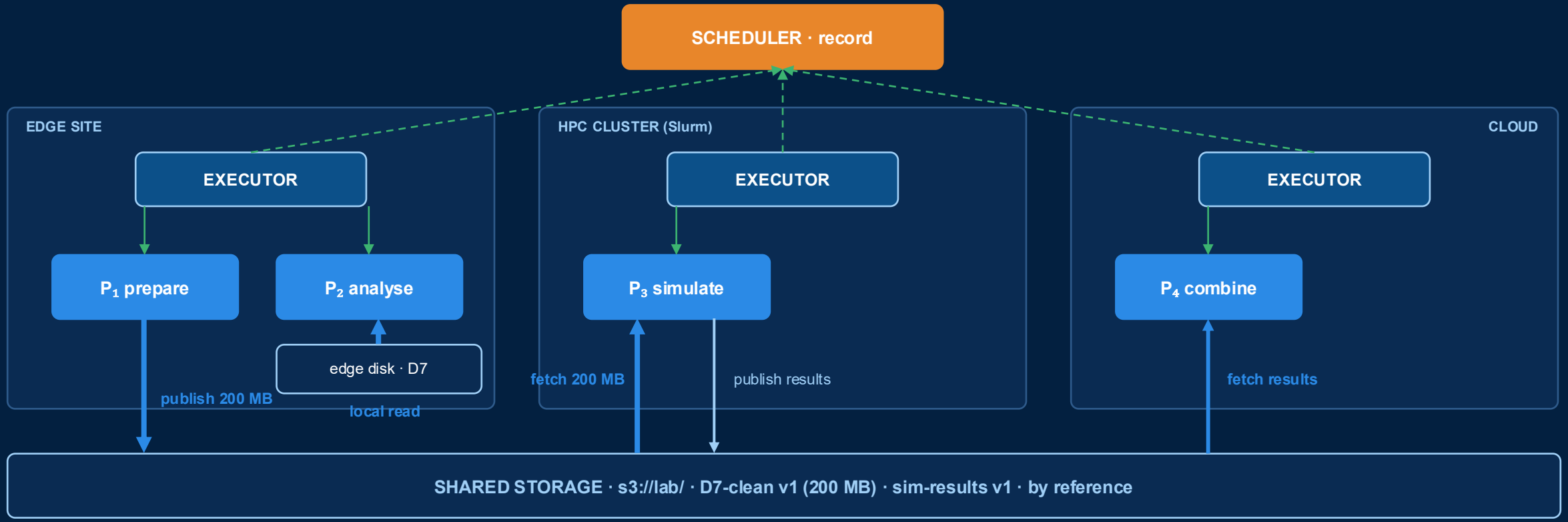
22: function HASFREERESOURCES(c, t_start, p, A)
23:     for t ∈ [t_start, t_start + p.duration) do
24:         if A[c][t] + r(p) > R(c) then
25:             return false
26:         end if
27:     end for
28:     return true
29: end function

30: function EMISSION(p, c, t_start)
31:     emission ← 0
32:     for t ∈ [t_start, t_start + p.duration) do
33:         emission ← emission + e_p(t) · i_c(t)
34:     end for
35:     return emission
36: end function
```

3 · Data & communication

How do processes exchange information?

The DAG placed on machines: control messages are small, but data must be fetched



By value

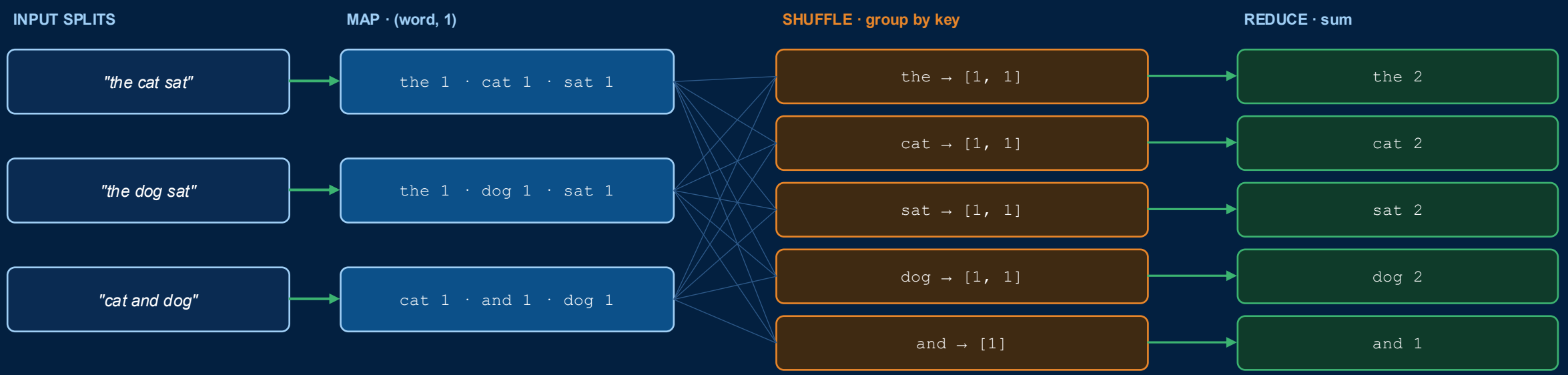
The message carries the bytes: RPC arguments, a result, an XCom. Kilobytes, not D7.

By reference

The message carries a name; the bytes are downloaded where the consumer runs.

Thin arrows: who asks whom (communication). Thick arrows: what must move (data). Placement decides how thick.

MapReduce: the programming model — *map, shuffle, reduce*

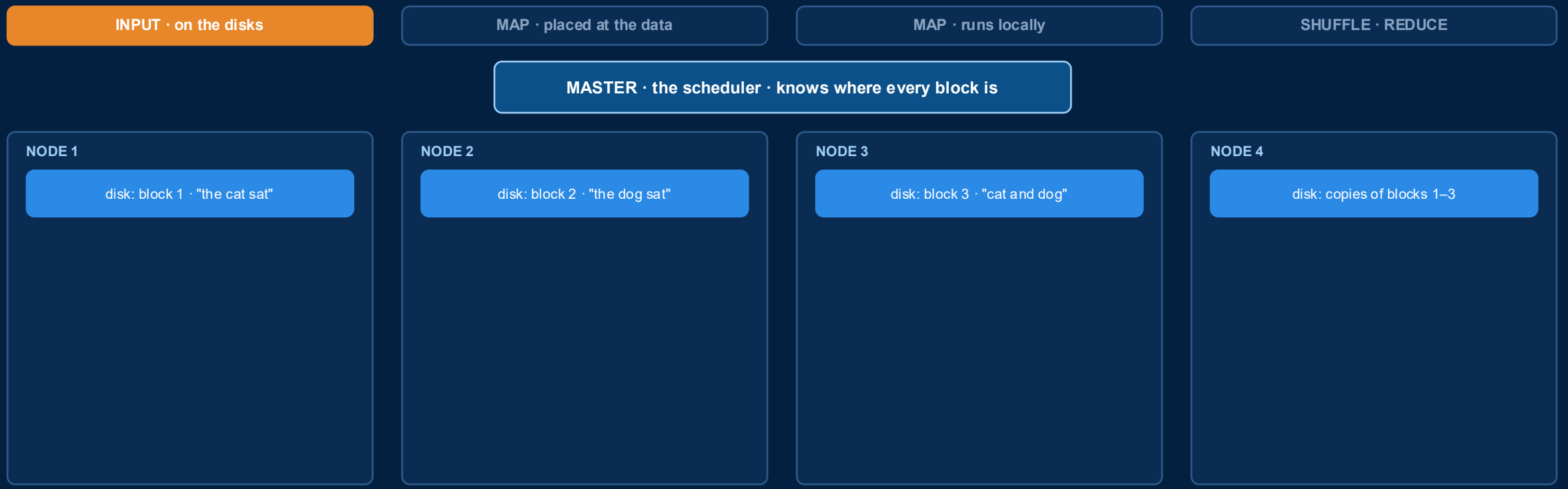


$\texttt{map}(k_1, v_1) \rightarrow \texttt{list}(k_2, v_2)$ $\quad$ $\texttt{reduce}(k_2, \texttt{list}(v_2)) \rightarrow \texttt{list}(v_3)$

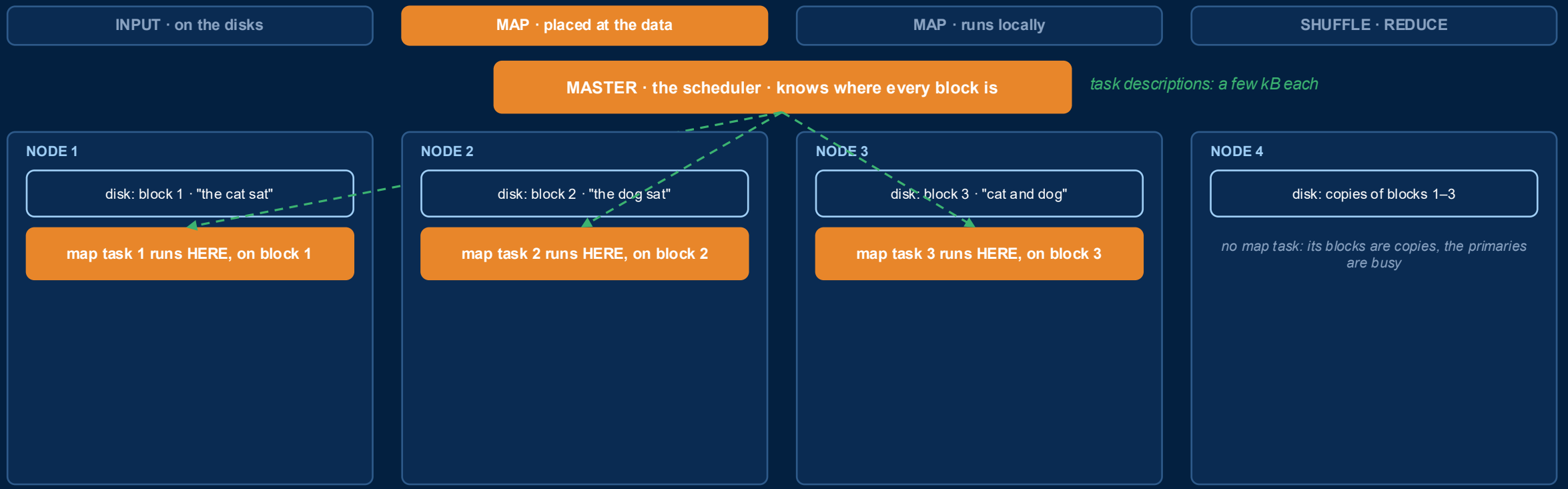
The user writes two pure functions. The framework splits the input, runs map in parallel, groups every intermediate value by key across all machines, runs reduce in parallel — and handles placement, retries and stragglers.

Two functions, no communication code: the DAG is map → shuffle → reduce, and the shuffle is the only place data crosses the network by design.

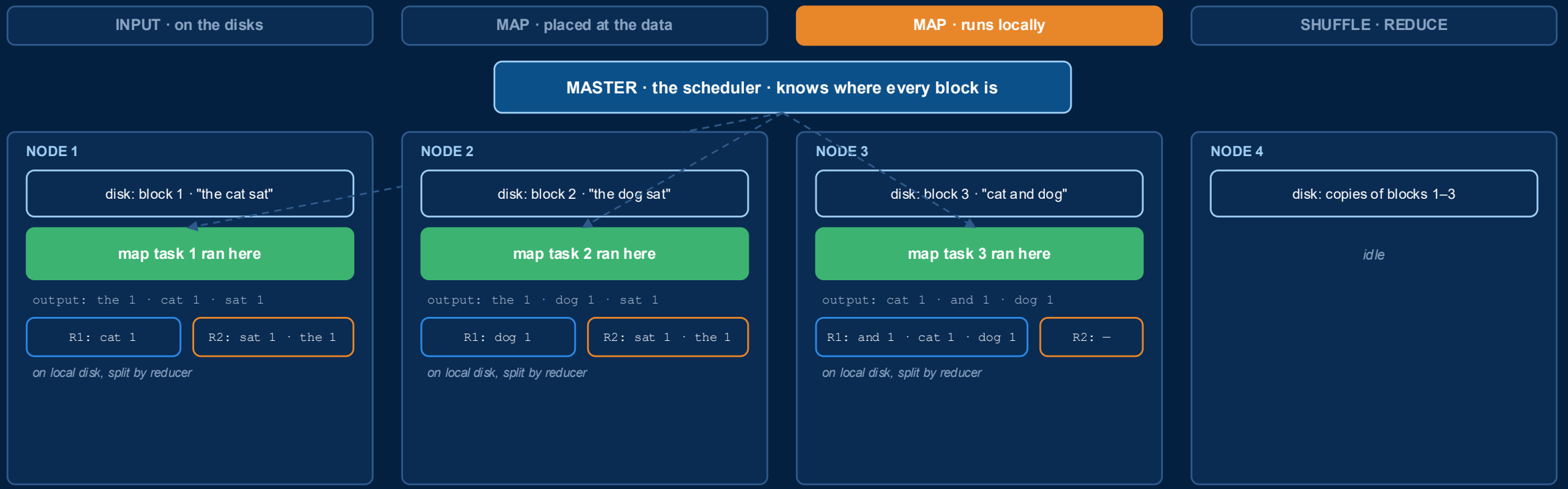
MapReduce on a cluster



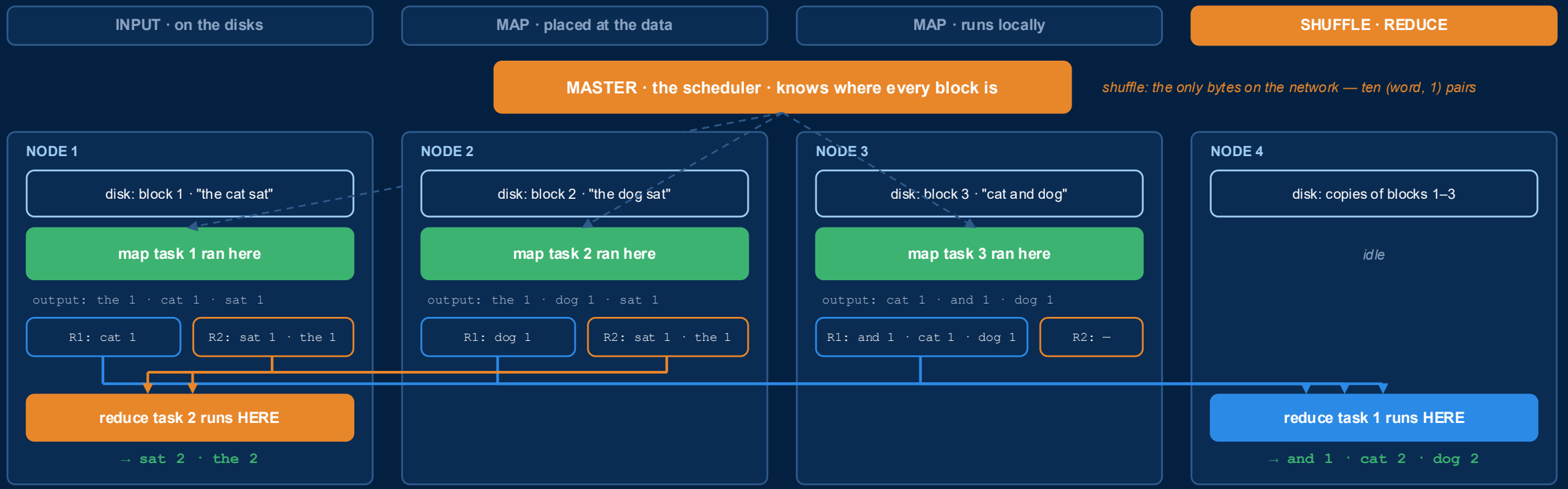
MapReduce on a cluster



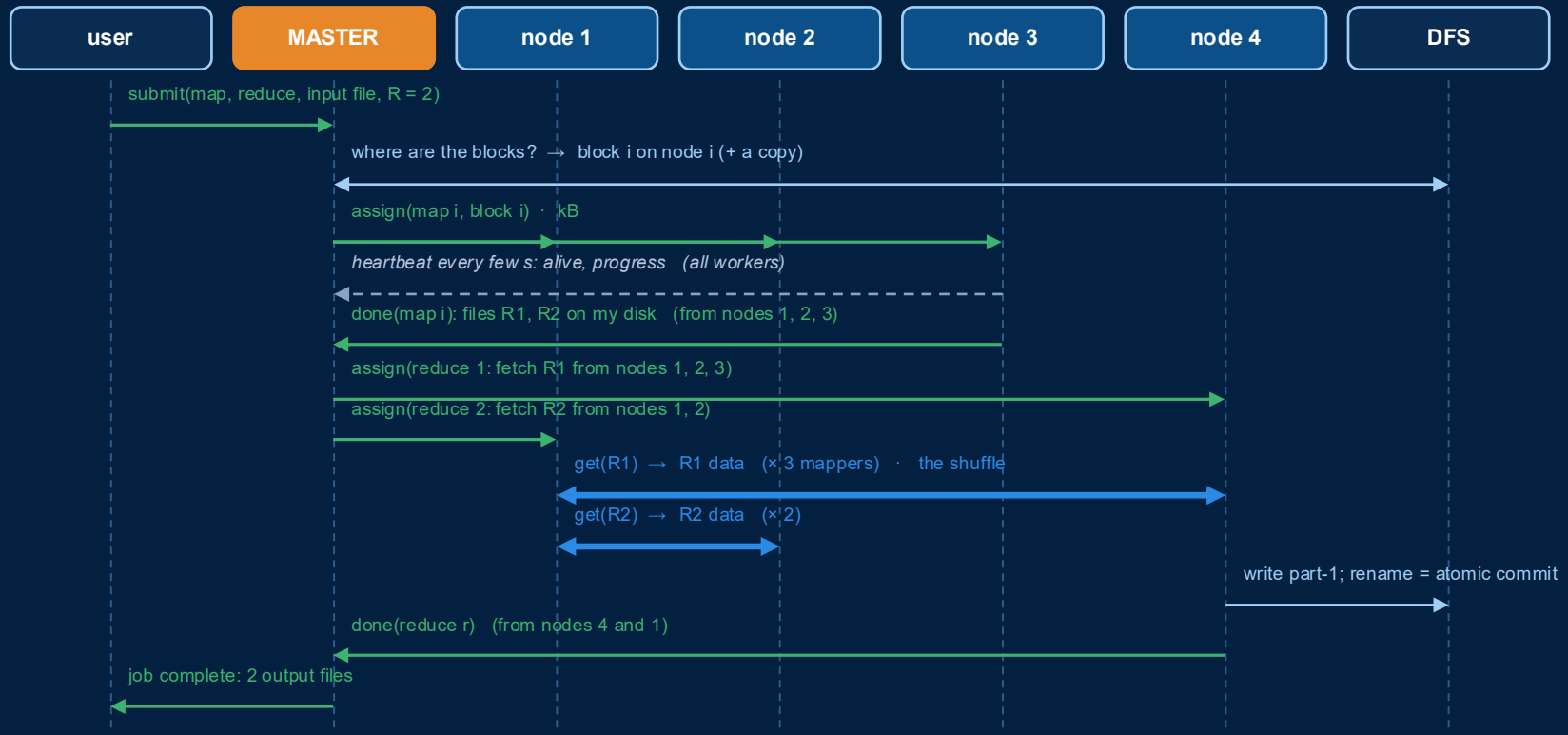
MapReduce on a cluster



MapReduce on a cluster



MapReduce as a distributed system



for M map and R reduce tasks: M + R assignments, M + R completions, M × R fetches in the shuffle, heartbeats throughout — and one master that sends every decision

The record lives in the master

State of every task (idle · running · done), which worker runs it, where the intermediate files are. Workers only report; every decision is a message from the master — station 2's scheduler, station 3's star.

Failure is silence

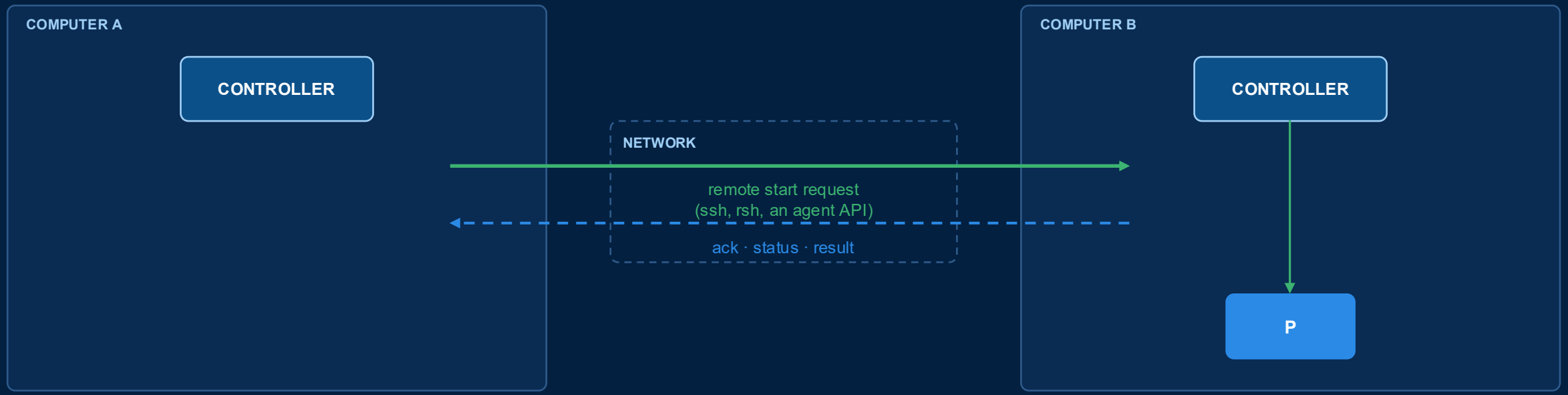
No heartbeat → the worker is presumed dead: its running tasks and its finished map tasks go back to idle (their files died with the disk); finished reduce tasks do not (output is in the DFS). Slow worker → a backup task. Station 5.

Two coordination points

The barrier: reduce is assigned only after every done(map). The atomic rename: two copies of a reduce task cannot both commit — re-execution is safe because tasks are deterministic. Station 4.

Seen as messages: one master decides and everyone reports; data moves only in the M × R fetches of the shuffle; silence means failure, and re-execution is the cure.

Two controllers can communicate directly — inside one network

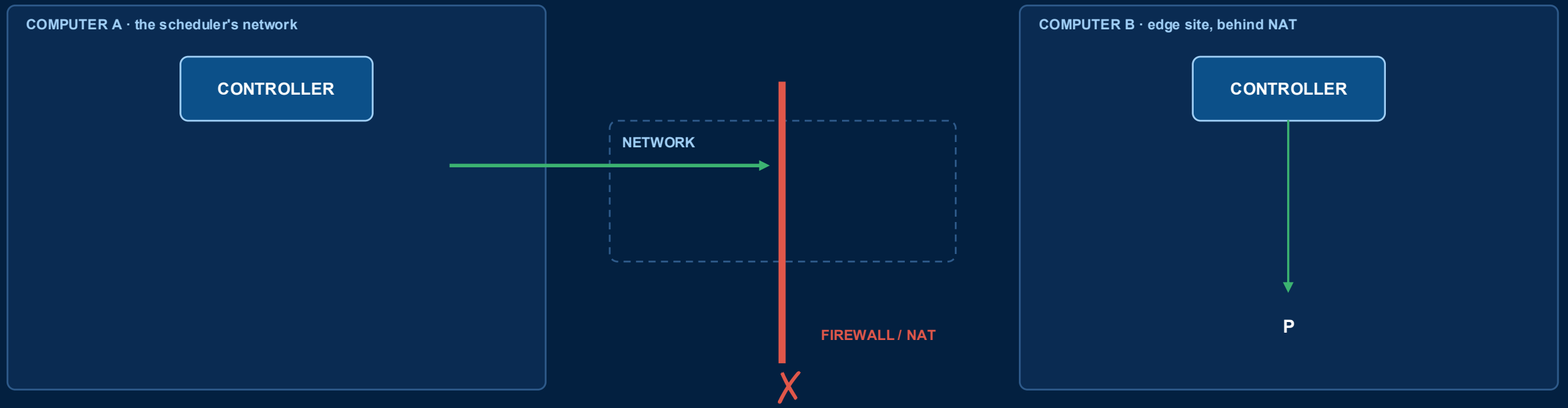


The direct design depends on

a stable address for B · an open inbound route · compatible protocols · both machines up at once — ssh, Ansible, MPI launchers.

Direct remote start works when A can reach B. The next slide removes that assumption.

But the network path is usually blocked



Why

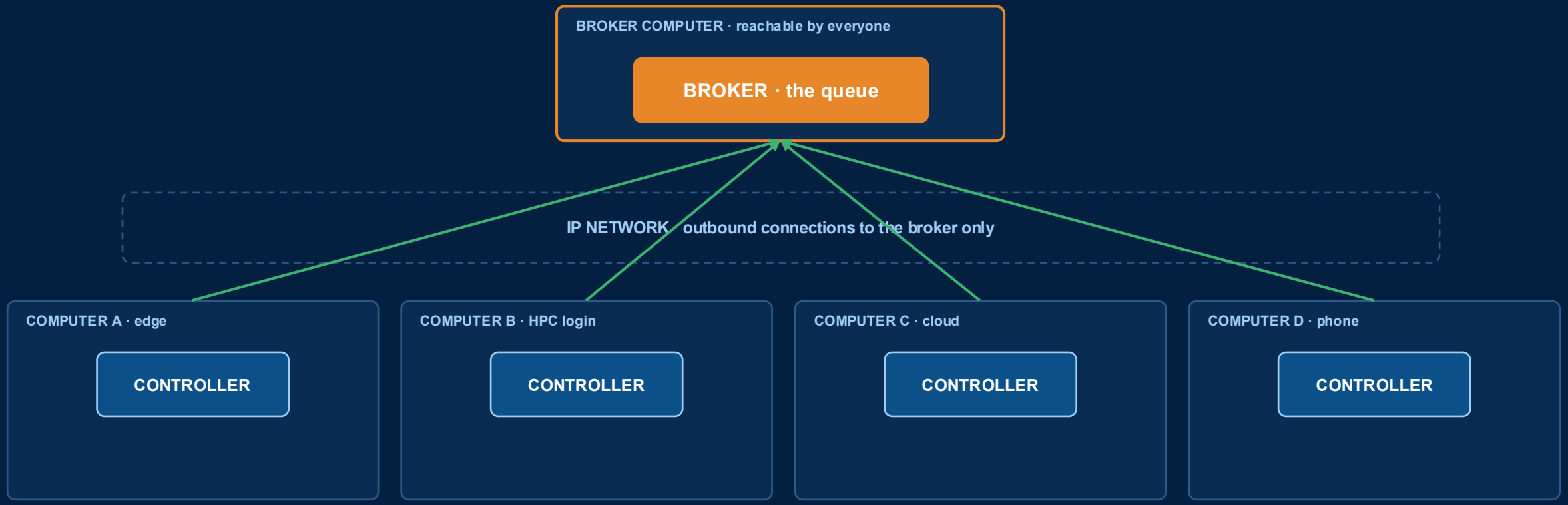
NAT: no routable address. Firewalls drop inbound. Different organisations own the sites.

Workarounds

VPNs and tunnels someone must own. Outbound connections, however, are allowed nearly everywhere.

A controller outside cannot call in to the edge or the HPC partition; both can call out. That asymmetry shapes every design that follows.

A broker turns the controllers into a star topology — the grid's rendezvous, generalised



What the broker is

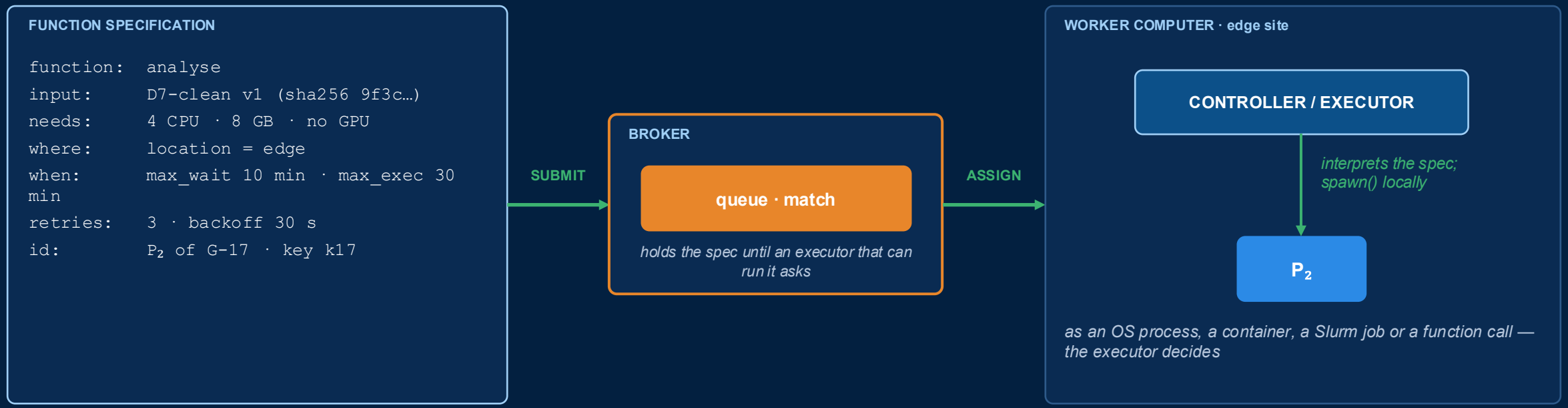
Station 2's scheduler — its queue and record — placed where every controller can reach it: a rendezvous with a stable address, and a single point of failure to make durable.

What it is not

It does not run the workload. It connects requests with controllers that can realise them.

One reachable rendezvous; every controller connects outward. Condor's matchmaker and BOINC's server, stripped to the pattern.

Indent-based orchestration



Why intent, not execution

A running process cannot be posted; a description can. It lets the broker decide where and when — and be executed again after a failure.

Each field feeds a station

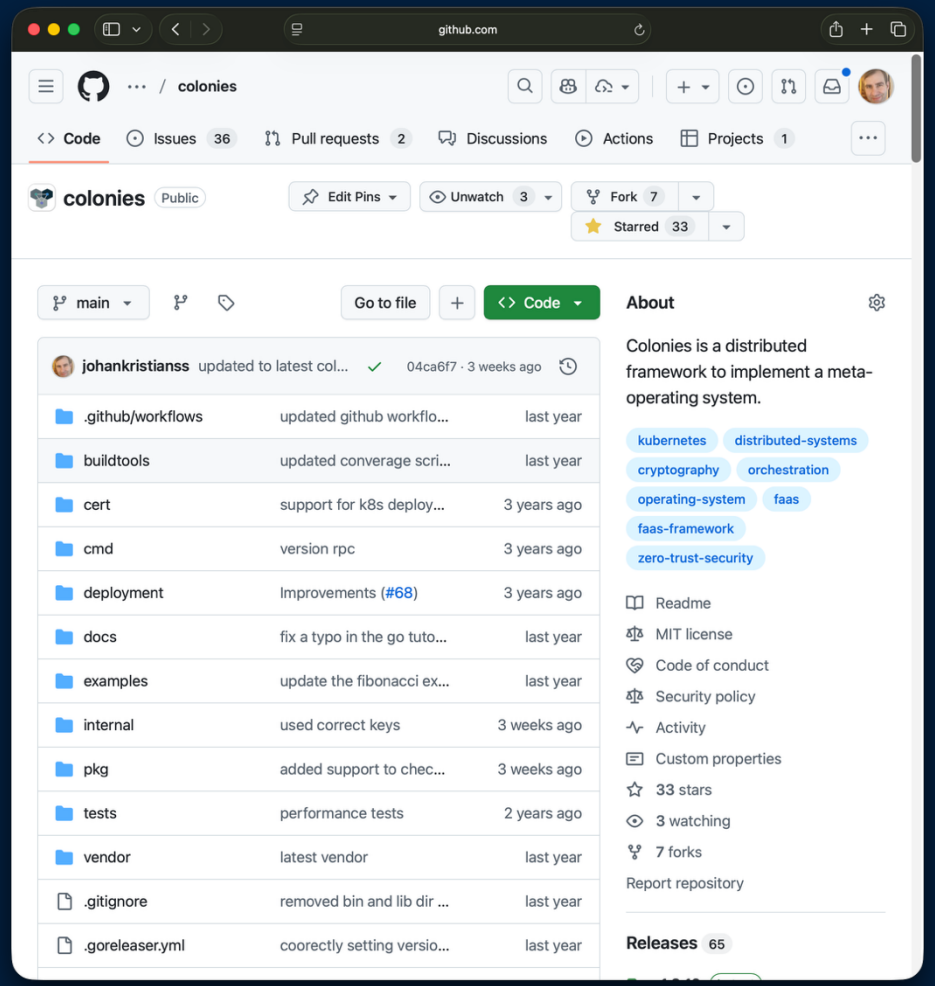
needs → filter · where/when → offloading, admission · retries → recovery · id + key → identity chain.

The description travels; the process is born where it runs. Every later decision reads a field of this specification.

Case study · ColonyOS

Twelve failure modes, and the design they lead to

ColonyOS



https://github.com/colonyos/colonies



https://colonyos.io

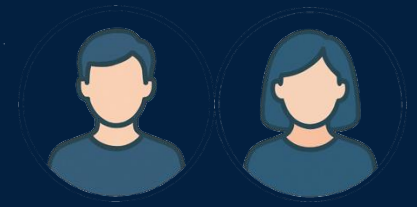
Meta-Operating Systems

Images downloaded from
Freepik and Vecteezy

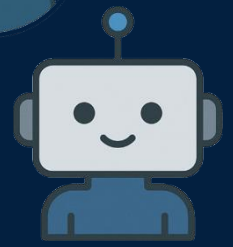


AI Application





Train
AI models



Meta Operating System

Process or Process or Process

Engine (Executor)

Workload Manager

GPUs

Supercomputers (HPC)

Engine (Executor)

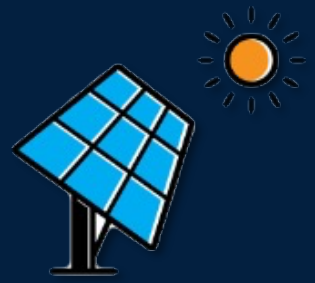
Public cloud

Sensitive data

On-prem cloud

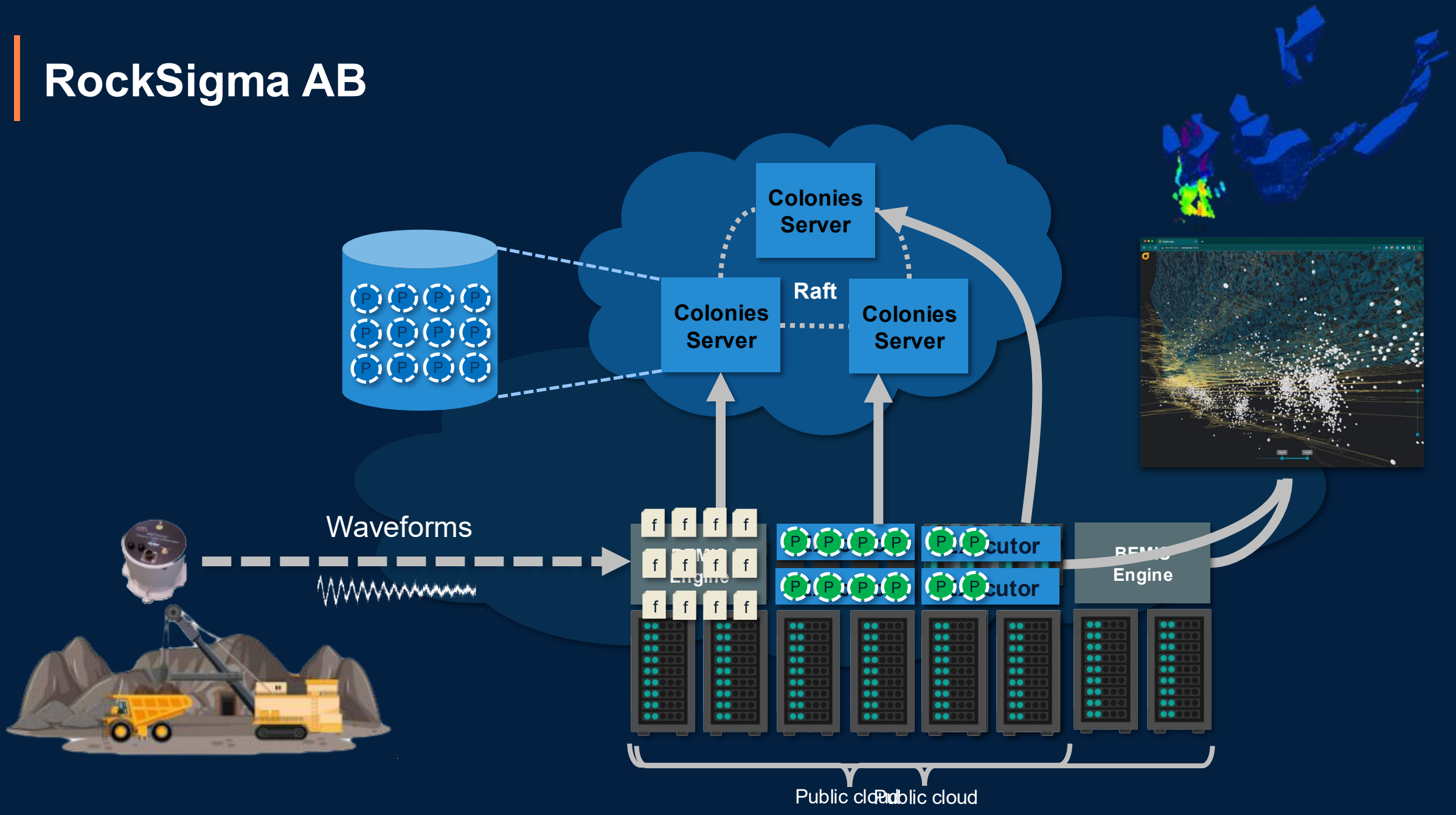
Timeshift



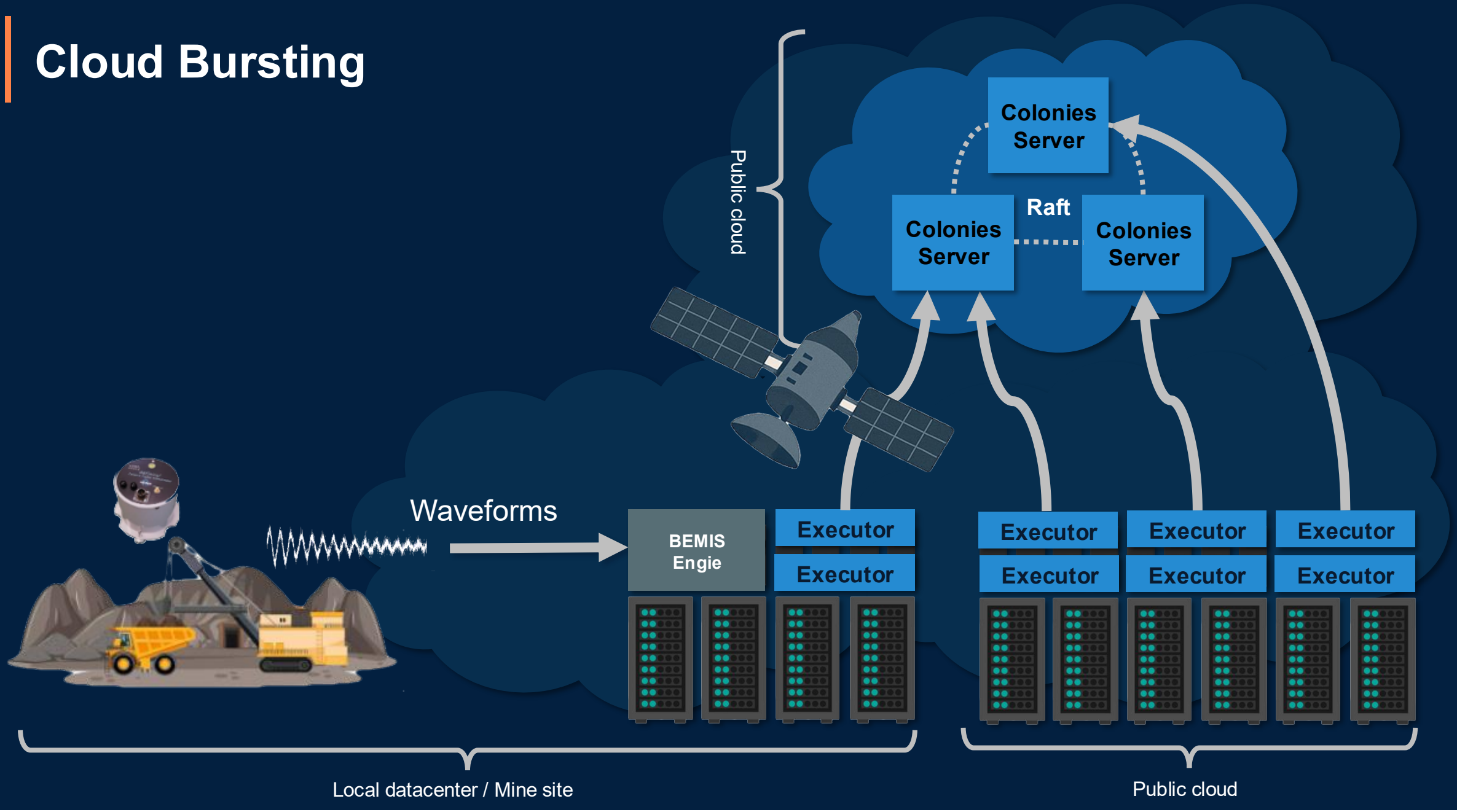


Device Edge / Network Edge

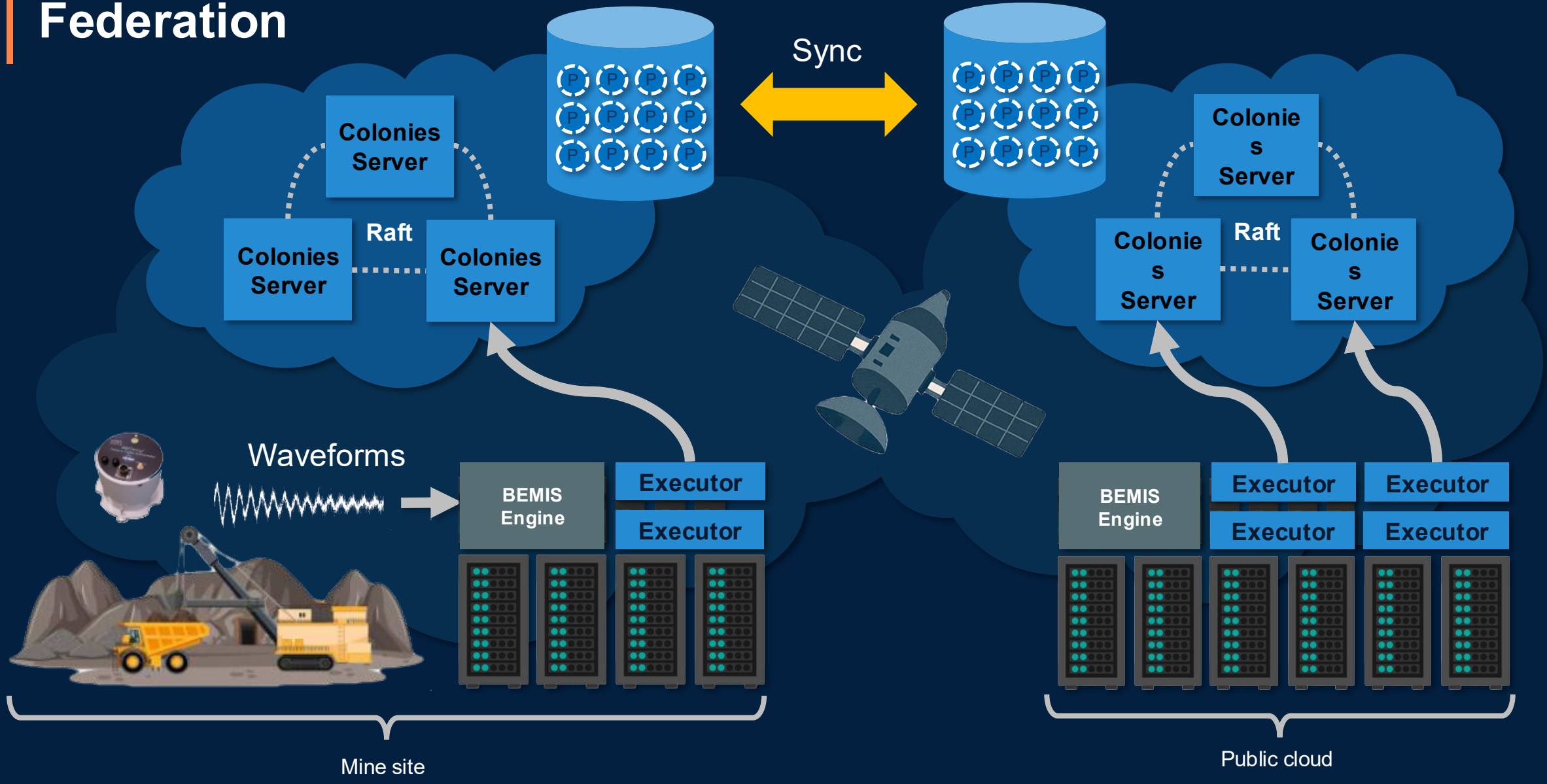
RockSigma AB



Cloud Bursting

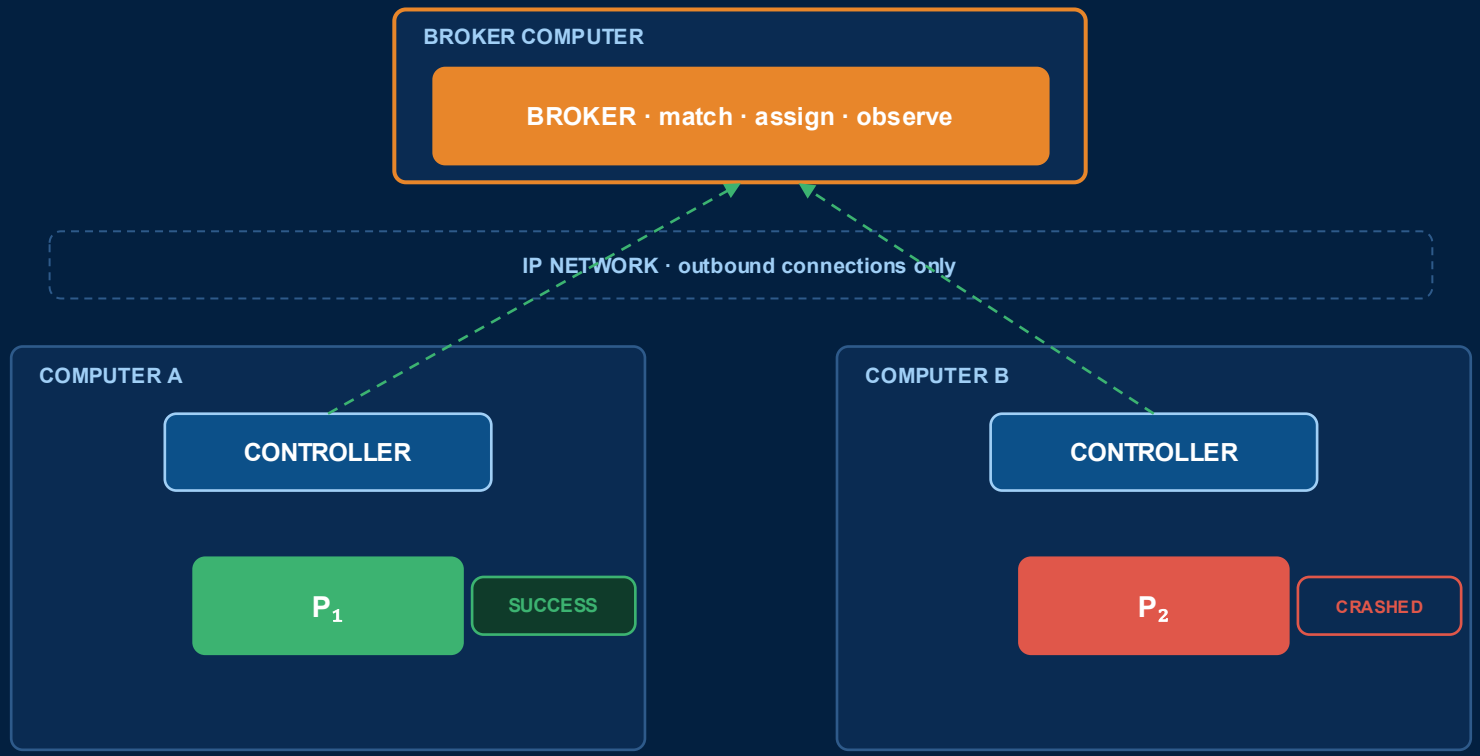


Federation


Sync
Colonies Server
Raft
Colonies Server
Colonies Server
Colonies Server
Raft
Colonies Server
Colonies Server
Waveforms
BEMIS Engine
Executor
Executor
BEMIS Engine
Executor
Executor
Executor
Executor
Mine site
Public cloud

P_2 crashes

FAILURE MODE 1 OF 12 · PROCESS FAILURE



What the broker sees

B's controller reports it: exit code ≠ 0, no output. This one the broker is told about.

What the design must provide

Record the failure as the outcome of this attempt — distinct from a lost machine or a lost message. Keep the error; retry while a budget remains.

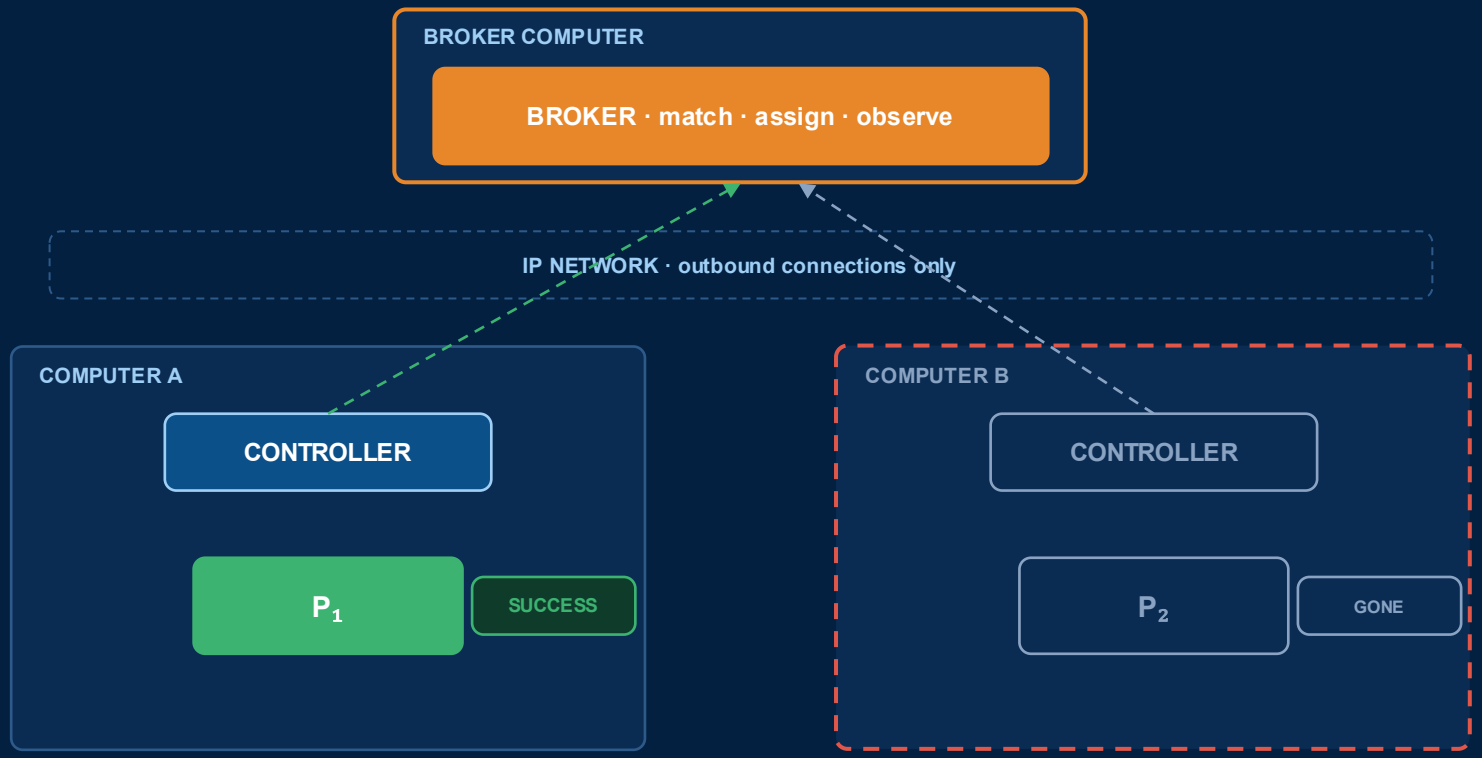
In ColonyOS

Close(FAILED, errors) · maxretries · the attempt history on the Process record.

1 · P_2 crashes: Record the failure as the outcome of this attempt — distinct from a lost machine or a lost message.

Computer B crashes

FAILURE MODE 2 OF 12 · MACHINE FAILURE



What the broker sees

Nothing. No exit code, no message: silence. From the broker's side a dead machine looks exactly like a slow one.

What the design must provide

Detect by time, not by message: a deadline on every assignment (a lease). After it, assume the worst and re-place — remembering that the assumption may be wrong.

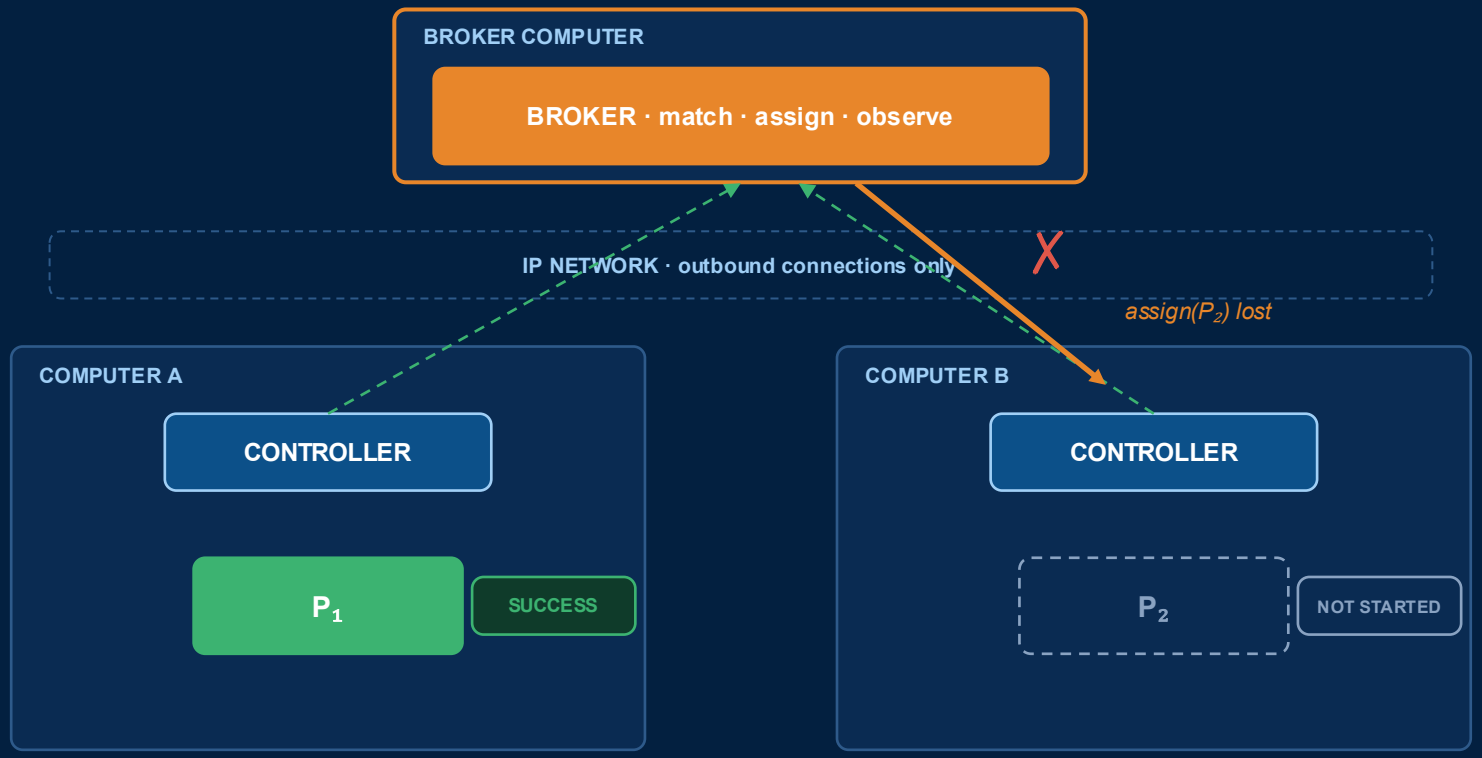
In ColonyOS

maxexectime becomes a deadline at the claim; the timeout loop unassigns and retries.

2 · Computer B crashes: Detect by time, not by message: a deadline on every assignment (a lease).

The request message is lost

FAILURE MODE 3 OF 12 · REQUEST LOSS



What the broker sees

The broker sent the assignment and hears nothing back. Is P_2 running, or did the message never arrive?

What the design must provide

Make assignment repeatable without starting P_2 twice — or let the executor ask: a lost pull is simply asked again, and nothing was started.

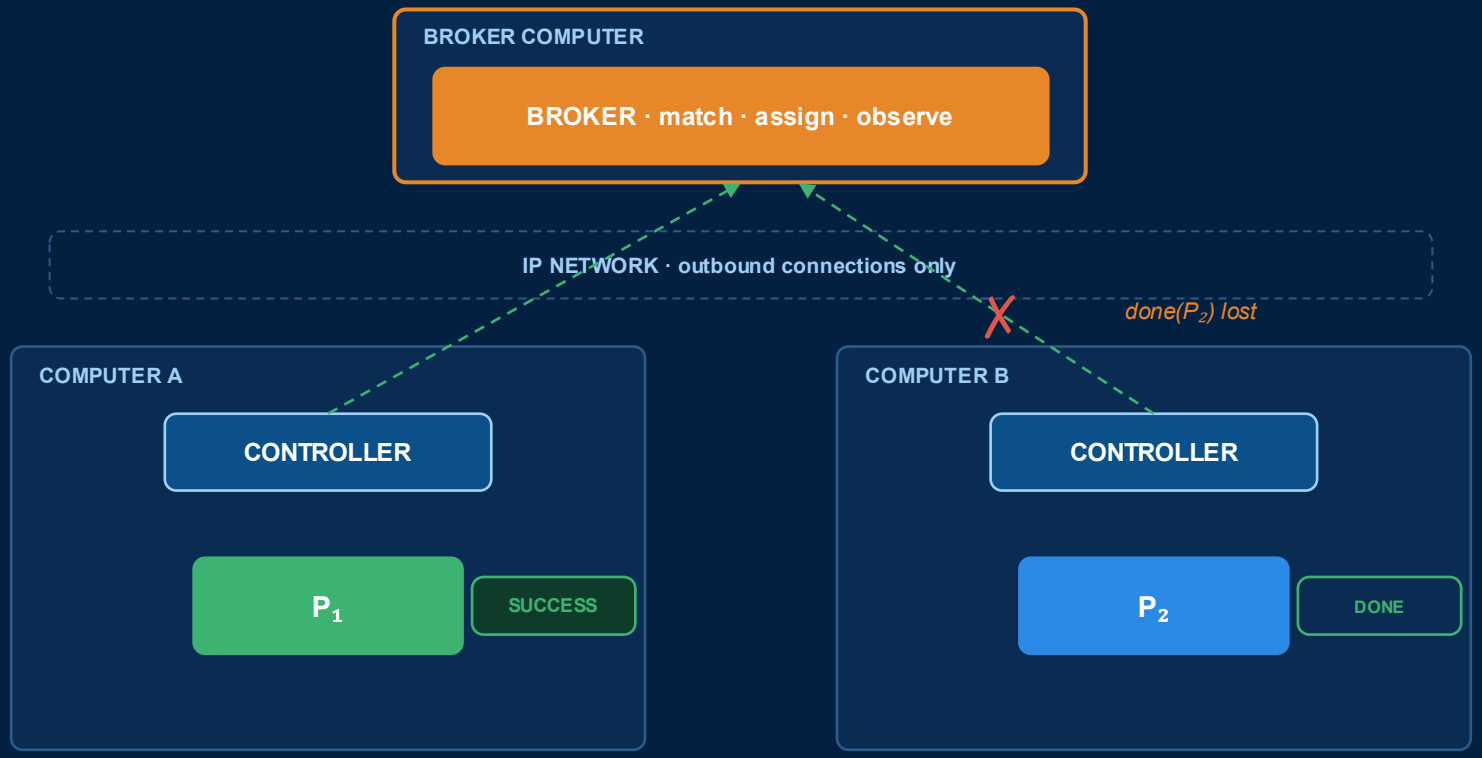
In ColonyOS

Assign is a pull: the executor asks. A lost request costs one more ask, nothing else.

3 · The request message is lost: Make assignment repeatable without starting P_2 twice — or let the executor ask: a lost pull is simply asked again, and nothing was started.

The reply message is lost

FAILURE MODE 4 OF 12 · REPLY LOSS



What the broker sees

P_2 finished and B said so — but the message was lost. The broker still sees RUNNING, and the deadline is ticking.

What the design must provide

Completion must be re-sendable and safe to receive twice: the record accepts one close per attempt and ignores repeats.

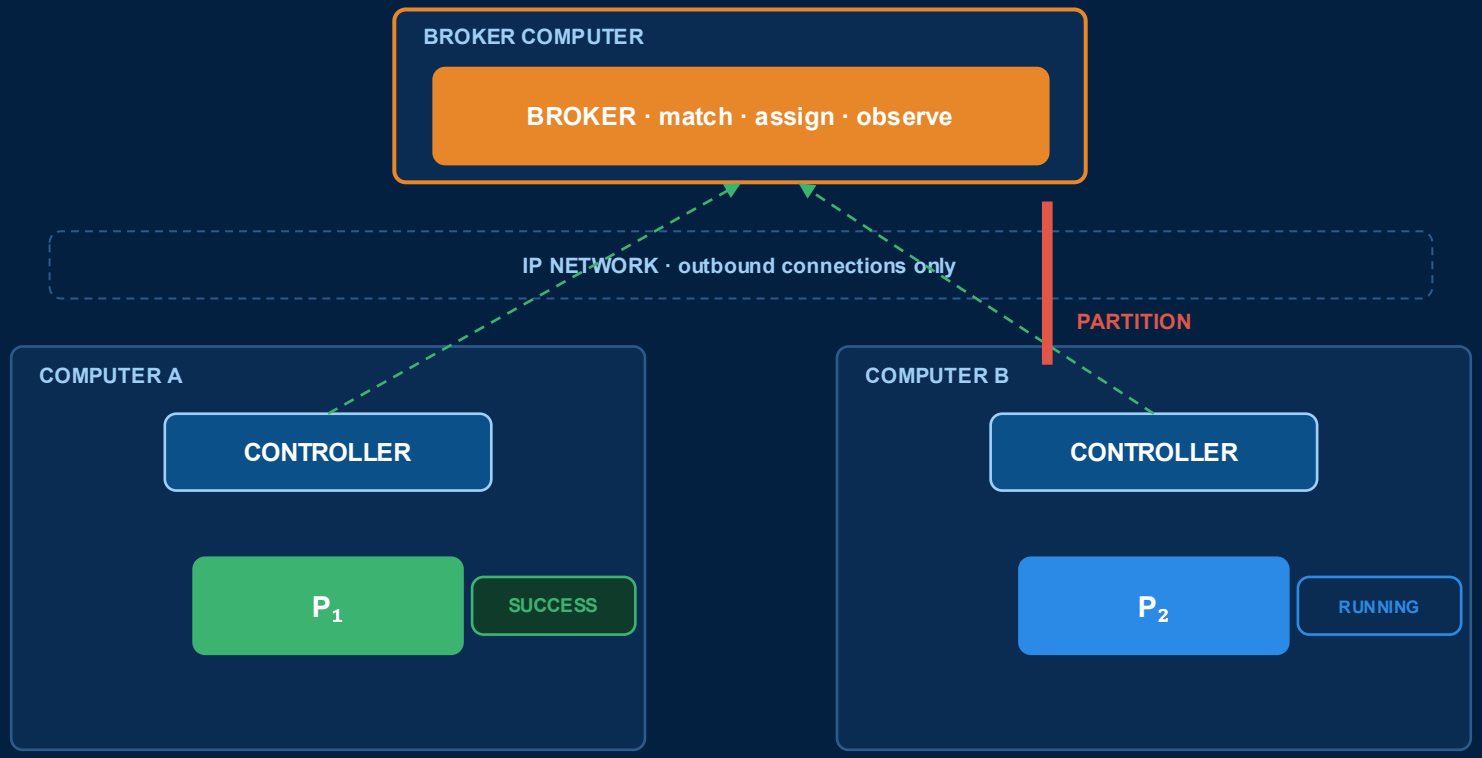
In ColonyOS

Close is idempotent on the record; a close for a process no longer RUNNING is refused.

4 · The reply message is lost: Completion must be re-sendable and safe to receive twice: the record accepts one close per attempt and ignores repeats.

The network is partitioned

FAILURE MODE 5 OF 12 · REACHABILITY



What the broker sees

B keeps running P_2 and cannot say so. The broker's deadline expires and it assigns P_2 elsewhere: two P_2 may now run.

What the design must provide

Ownership that expires on both sides: the executor stops trusting its claim when the lease ends; the record rejects a late close from an old attempt.

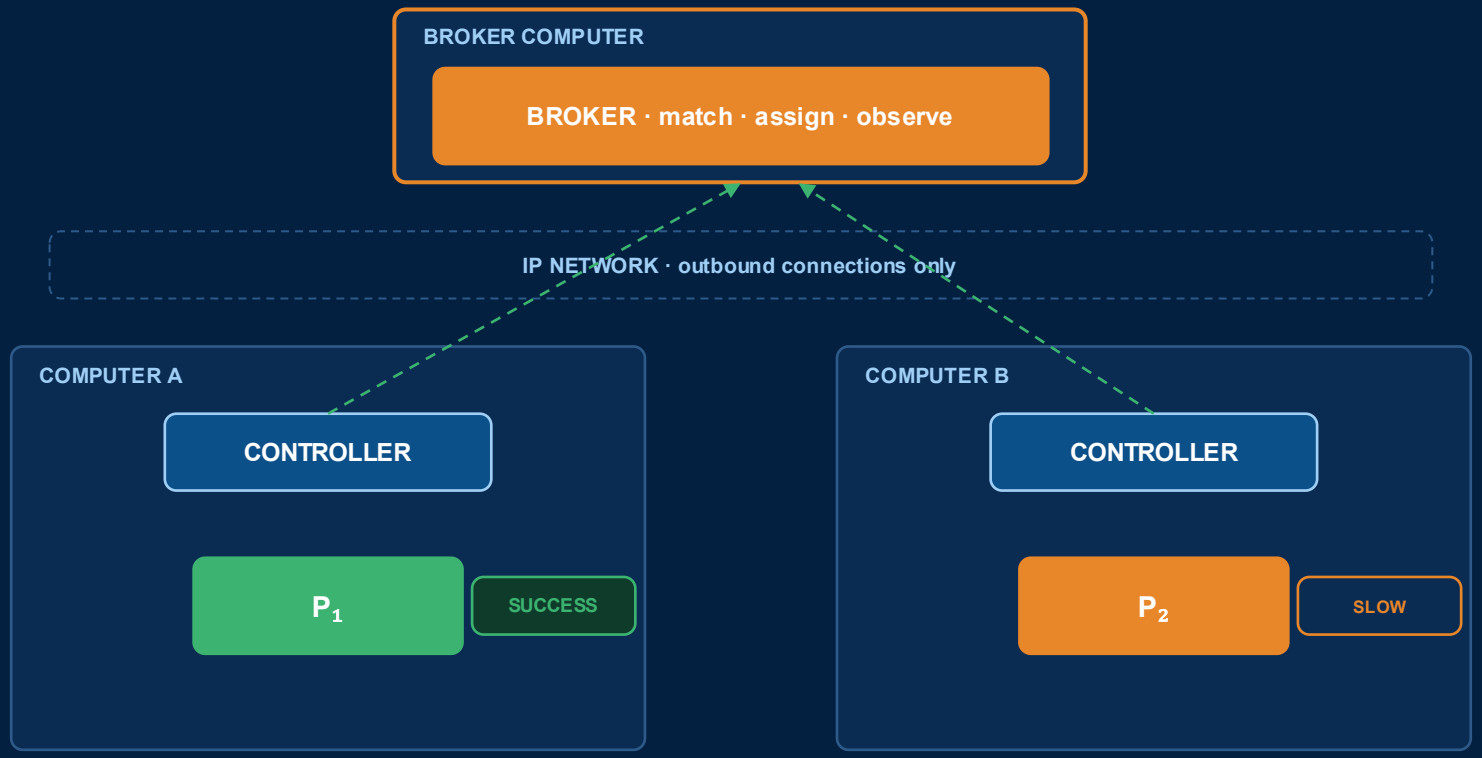
In ColonyOS

The deadline is the lease; a close after unassignment is refused — external effects must check the lease first.

5 · The network is partitioned: Ownership that expires on both sides: the executor stops trusting its claim when the lease ends; the record rejects a late close from an old attempt.

Computer B is overloaded

FAILURE MODE 6 OF 12 · LATE RESPONSE



What the broker sees

Everything is late: heartbeats, replies, P_2 itself. Slow and dead look the same; any timeout is a guess.

What the design must provide

Never send work to a machine that did not ask for it: back-pressure — the executor requests work only when it has capacity (pull). Timeouts are tuned, not trusted.

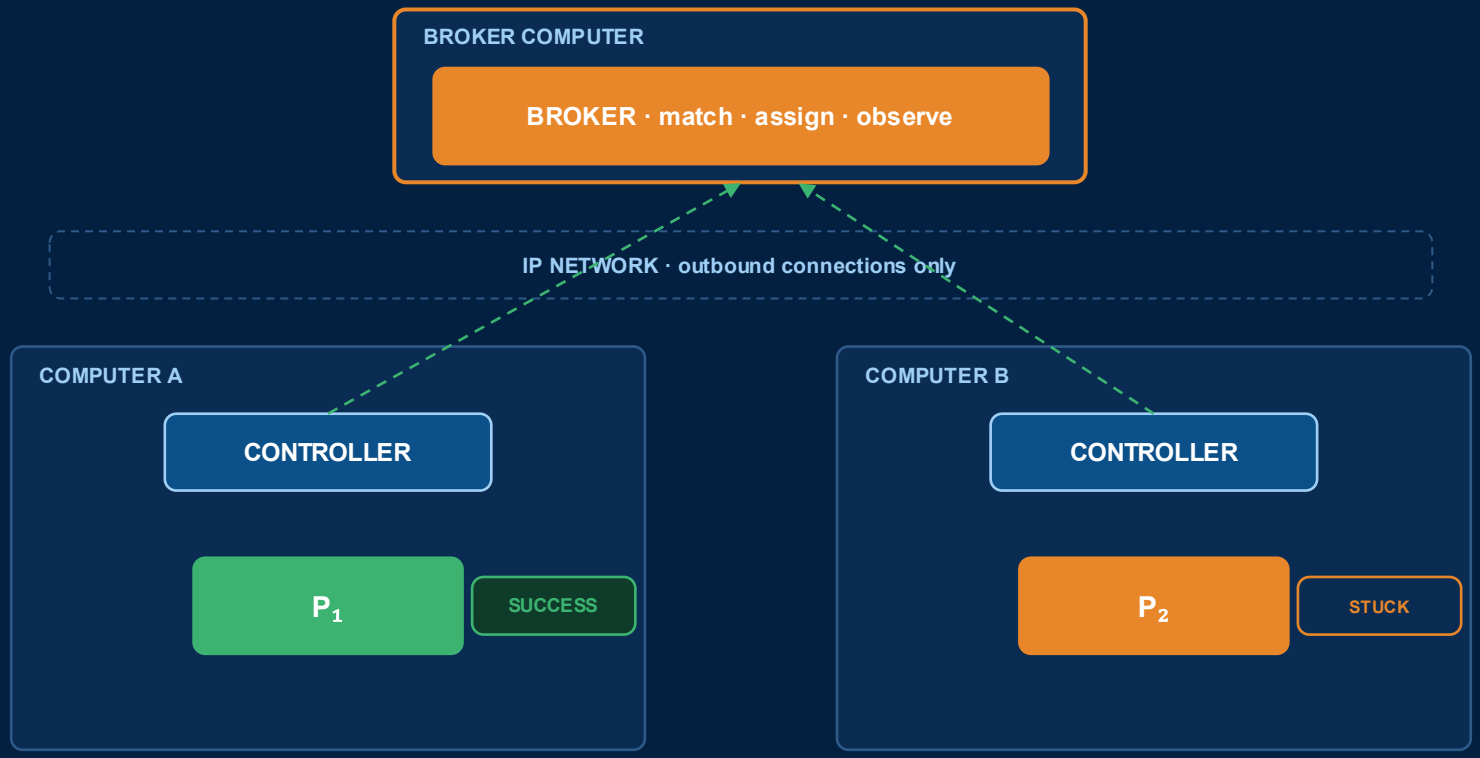
In ColonyOS

Executors call Assign only with free capacity; nothing is pushed to a busy executor.

6 · Computer B is overloaded: Never send work to a machine that did not ask for it: back-pressure — the executor requests work only when it has capacity (pull).

P_2 never finishes

FAILURE MODE 7 OF 12 · NO PROGRESS



What the broker sees

No crash, no silence: P_2 is alive and makes no progress — a deadlock, a wait for an input that never comes.

What the design must provide

Bound every attempt in time: a maximum execution time, after which the attempt is failed and, if the budget allows, retried elsewhere.

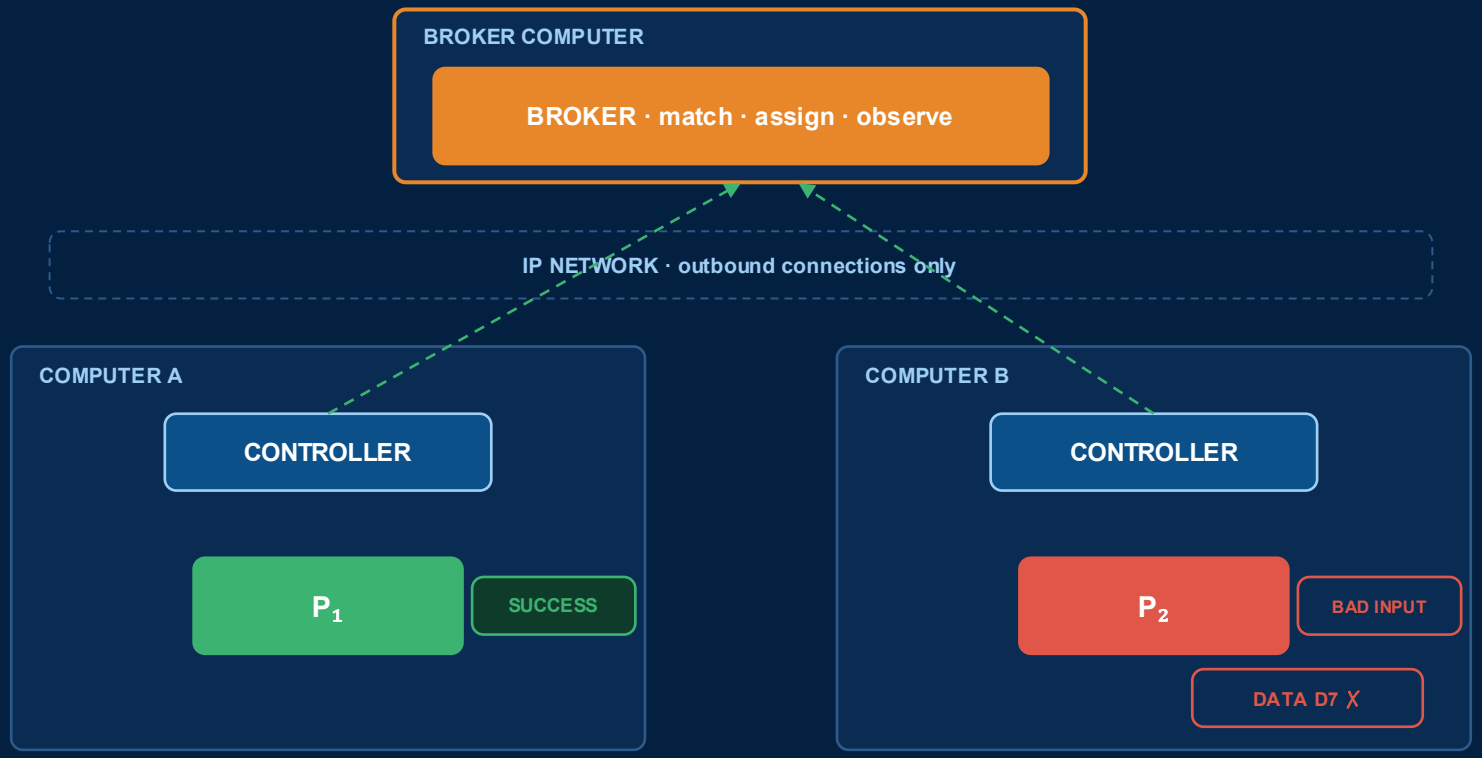
In ColonyOS

maxexectime for running processes; maxwaittime for processes never matched.

7 · P_2 never finishes: Bound every attempt in time: a maximum execution time, after which the attempt is failed and, if the budget allows, retried elsewhere.

The data does not arrive correctly

FAILURE MODE 8 OF 12 · DATA TRANSFER



What the broker sees

P_2 starts, fetches its input by reference, and gets a truncated or stale file. It fails late — or "succeeds" with garbage.

What the design must provide

A data contract: inputs by reference with an identity and a version (a hash), checked before use; fail fast and record which reference was wrong.

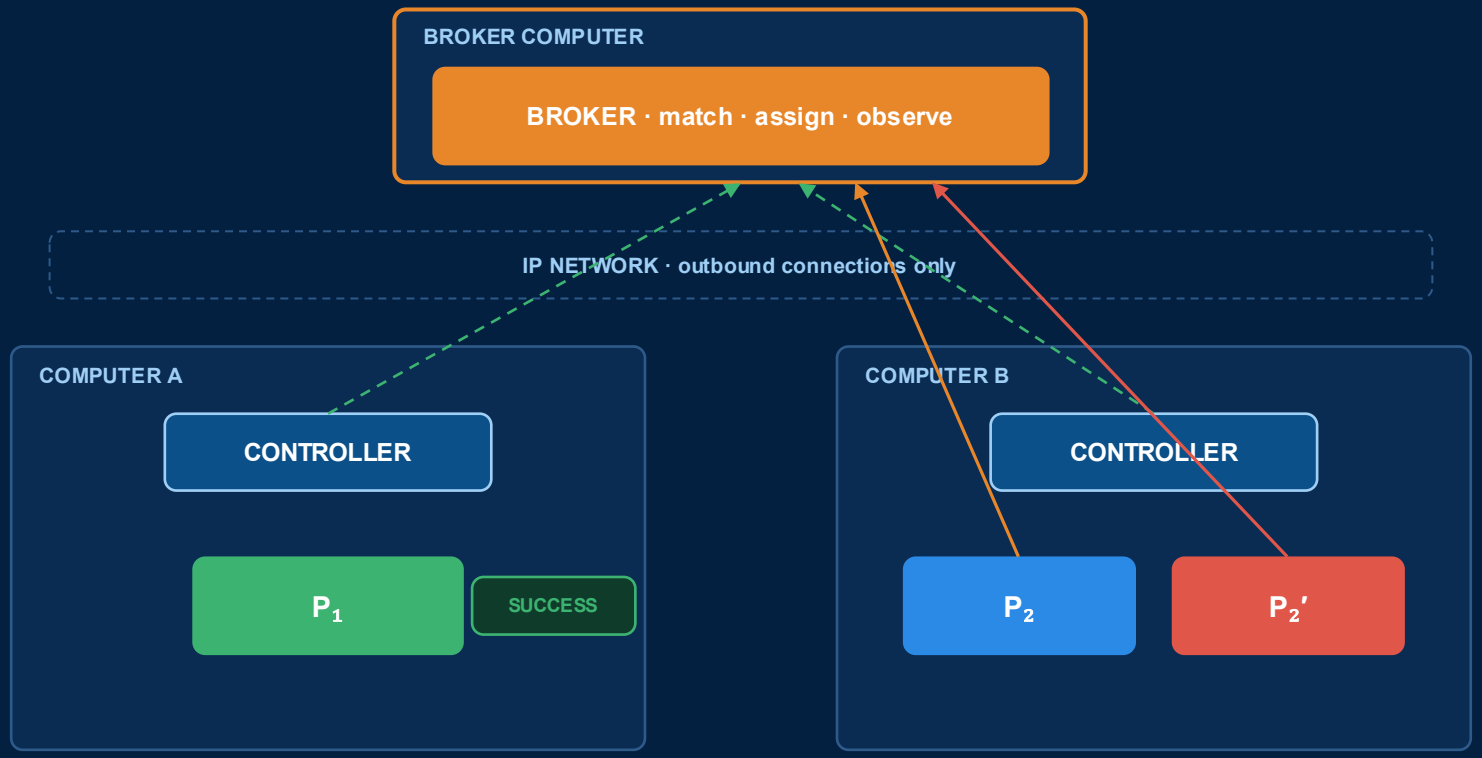
In ColonyOS

Inputs and outputs in the record, by reference; validation is the executor's — the contract is the application's.

8 · The data does not arrive correctly: A data contract: inputs by reference with an identity and a version (a hash), checked before use; fail fast and record which reference was wrong.

A retry starts P_2 twice

FAILURE MODE 9 OF 12 · DUPLICATE EXECUTION



What the broker sees

After a timeout the broker re-placed P_2 — but the first copy was alive. Two copies write the same output; external effects happen twice.

What the design must provide

Exactly one owner per attempt (an atomic claim); functions safe to run twice (idempotent); and reconcile before any retry that has external effects.

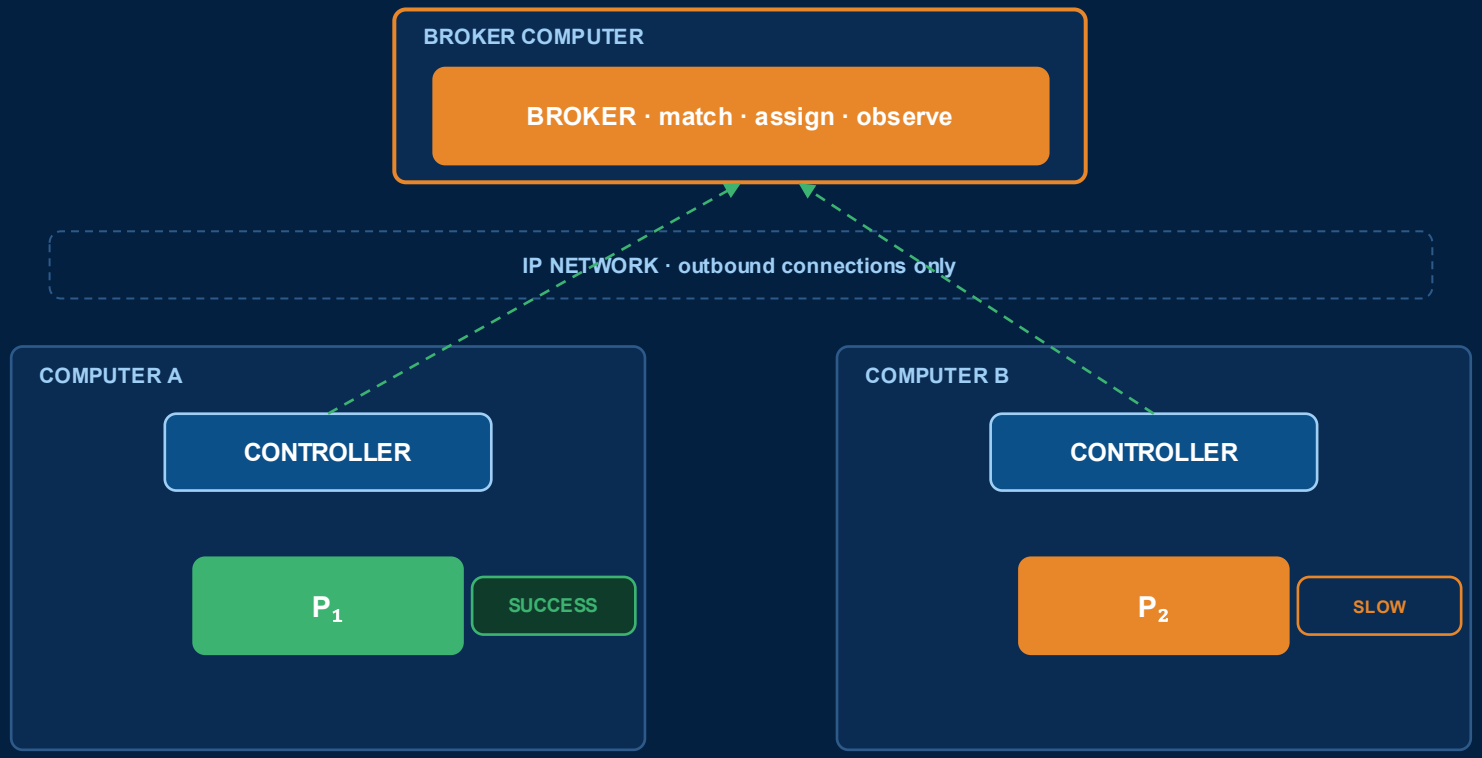
In ColonyOS

Atomic select-and-assign in PostgreSQL; the HPC executor checks the Slurm job before a second sbatch.

9 · A retry starts P_2 twice: Exactly one owner per attempt (an atomic claim); functions safe to run twice (idempotent); and reconcile before any retry that has external effects.

One parallel process is a straggler

FAILURE MODE 10 OF 12 · IMBALANCE



What the broker sees

Nine of ten parallel processes finished in a minute; P_2 is at 10 % after an hour. The workflow waits for the slowest.

What the design must provide

Speculative execution: start a backup copy elsewhere, take the first result, cancel the other — which needs idempotent processes and a record that accepts one result.

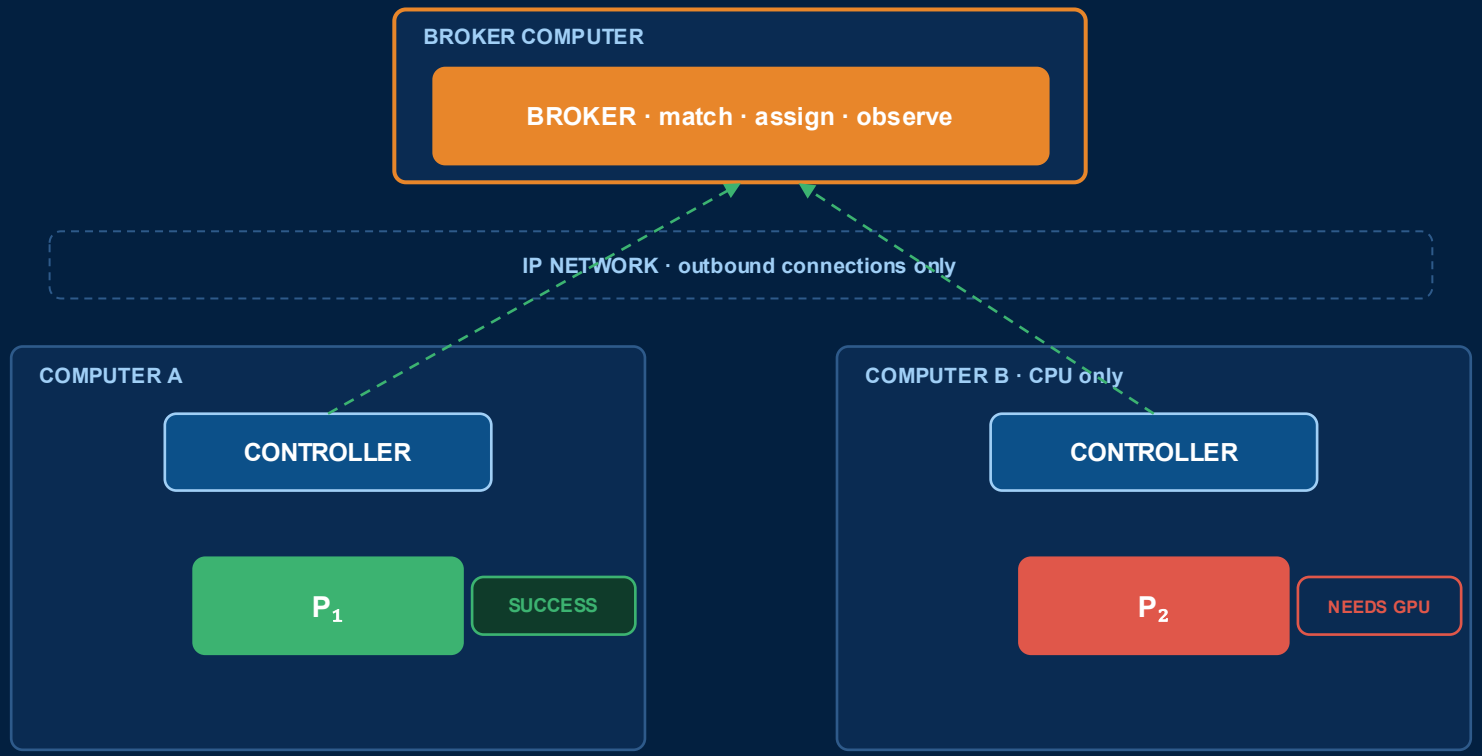
In ColonyOS

Not built in: a client may submit a backup process; the record accepts the first close.

10 · One parallel process is a straggler: Speculative execution: start a backup copy elsewhere, take the first result, cancel the other — which needs idempotent processes and a record that accepts one result.

P_2 is placed on the wrong computer

FAILURE MODE 11 OF 12 · INFEASIBLE PLACEMENT



What the broker sees

B has no GPU. P_2 fails at once — or worse, runs for hours on the CPU.

What the design must provide

Match before assigning: the process states its conditions, the executor its capabilities, and only feasible pairs are assigned — the filter of station 2.

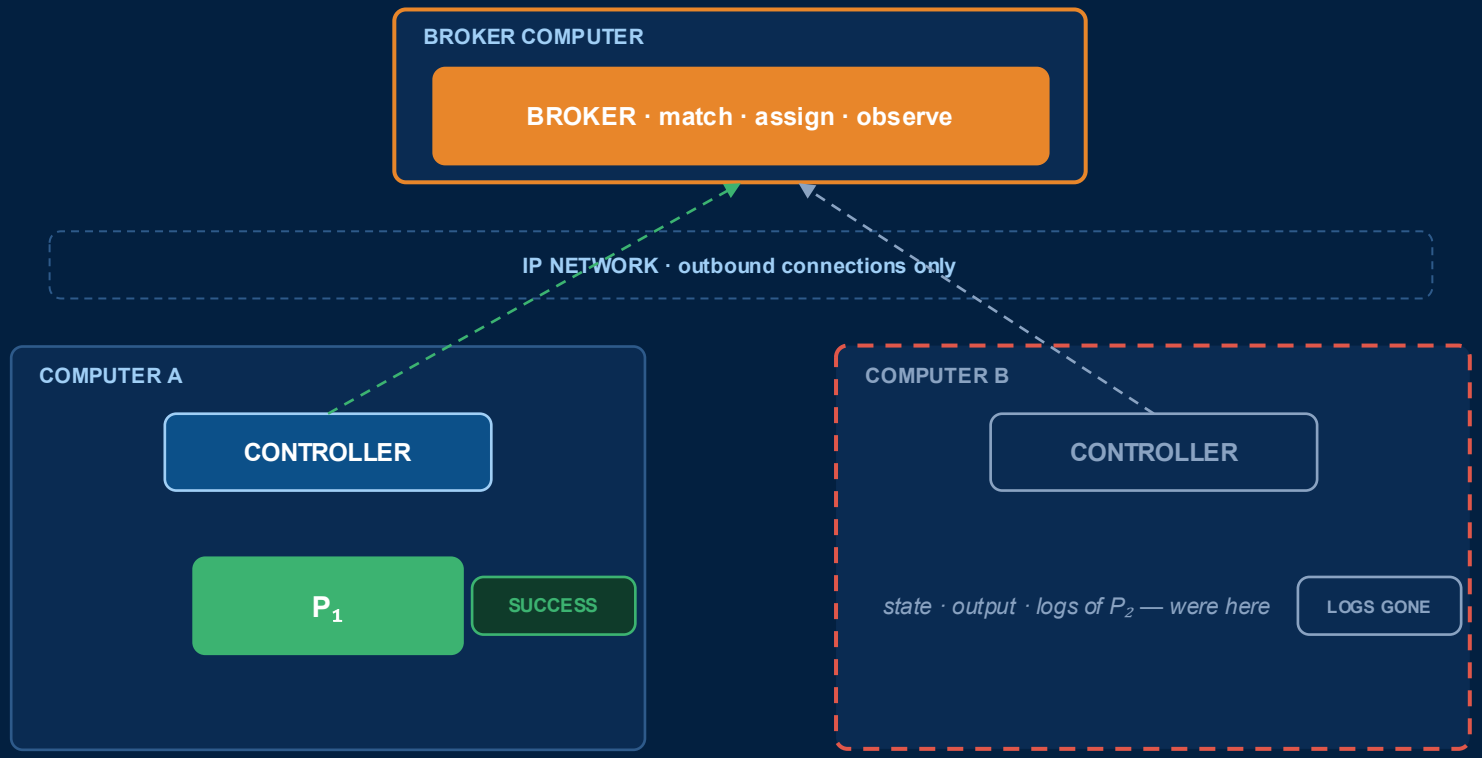
In ColonyOS

FunctionSpec conditions (executor type, names) are matched against the executor at Assign.

11 · P_2 is placed on the wrong computer: Match before assigning: the process states its conditions, the executor its capabilities, and only feasible pairs are assigned — the filter of station 2.

The history disappears with the machine

FAILURE MODE 12 OF 12 · NO DURABLE OBSERVATION



What the broker sees

B held the only copy of what happened to P_2: its state, its output, its logs. B is gone, and the workflow cannot decide what to do next.

What the design must provide

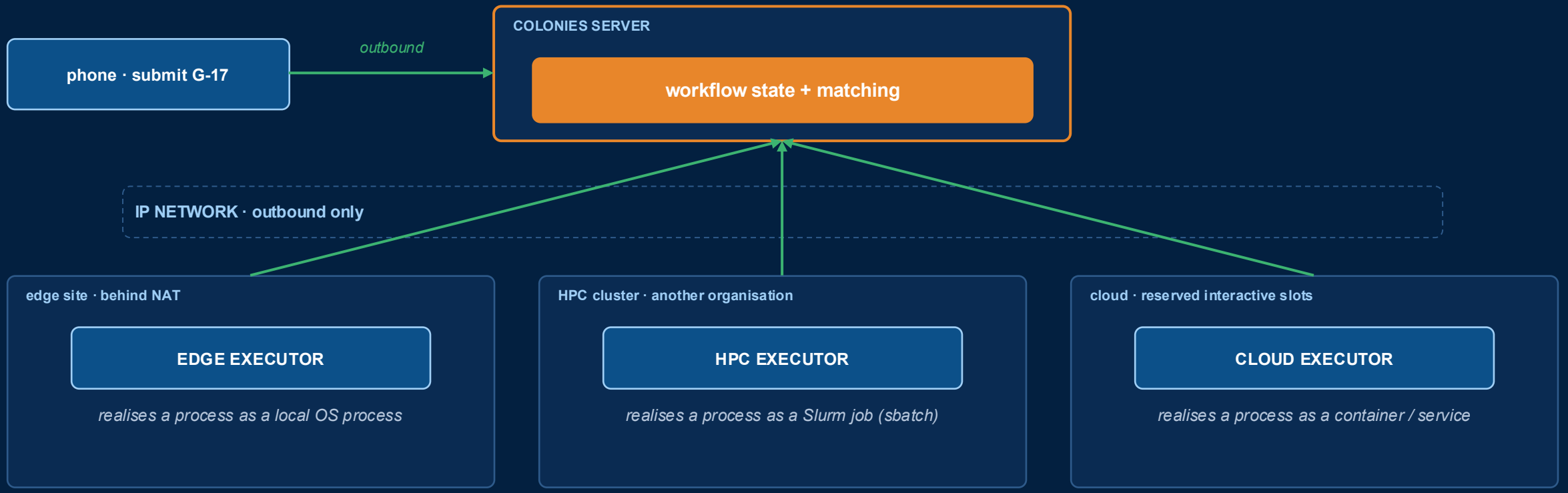
Control state must outlive any one machine: dependency order, observed state, outputs, attempts — kept where every party can reach it, durably.

In ColonyOS

The Process record and ProcessGraph in the server's database; executors keep the native ids.

12 · The history disappears with the machine: Control state must outlive any one machine: dependency order, observed state, outputs, attempts — kept where every party can reach it, durably.

ColonyOS connects execution across different platforms



Meta-orchestration

One workflow, several native execution systems: the executor translates an assigned process into whatever its platform runs; Slurm, Kubernetes or the OS keep their own schedulers.

What the diagram does not prescribe

A single unreplicated server. The role is one reachable control plane with a durable store; replicating it (station 4) is a deployment choice.

Executors connect outward and ask; the server remembers and matches. Question 2, option B — chosen for reachability across owners.

A FunctionSpec describes the work; a Process record gives it durable identity

FUNCTIONSPEC · what is requested

`funcname + args / kwargs`

`conditions: colony · executor type · names · dependencies`

`maxwaittime · maxexectime · maxretries · priority`

The specification of station 3: what, needing what, where, by when, how many times.

PROCESS · one row per node of the graph

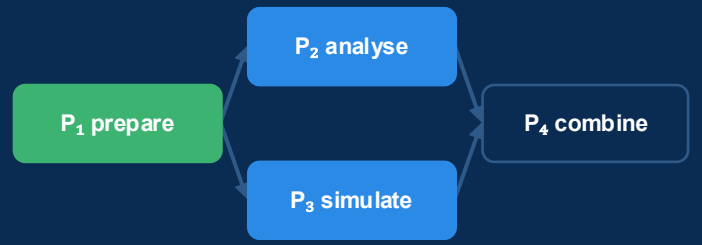
`state: WAITING · RUNNING · SUCCESS · FAILED`

`waits-for-parents · assigned executor`

`input / output · errors · retries · timestamps`

The DAG of station 1 and the record of station 4, in one table.

PROCESSGRAPH G-17



WAITING + flag set = blocked on parents; WAITING + flag clear = READY; only the latter is matched

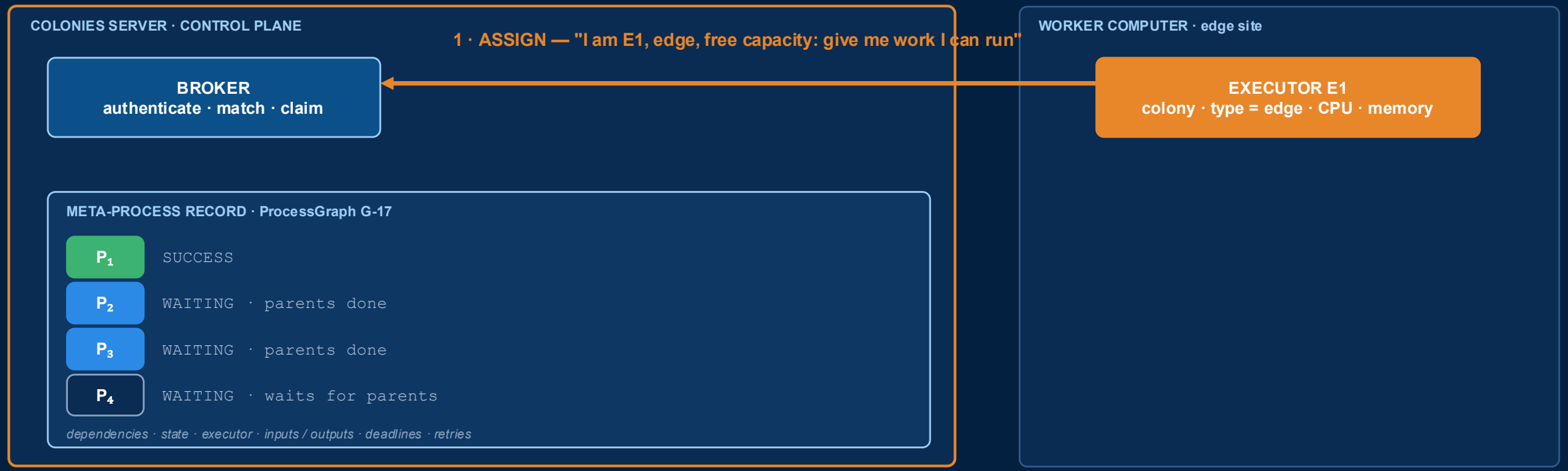
OS process versus ColonyOS process

OS process: one kernel, one live execution. ColonyOS process: many kernels, a durable record — the control plane owns identity, dependencies, placement, recovery and the outcome.

A field sketch, not a schema

The fields shown are the ones the lecture needs; the implementation has more, and not every user-supplied field is enforced by the server.

Assignment begins at the executor

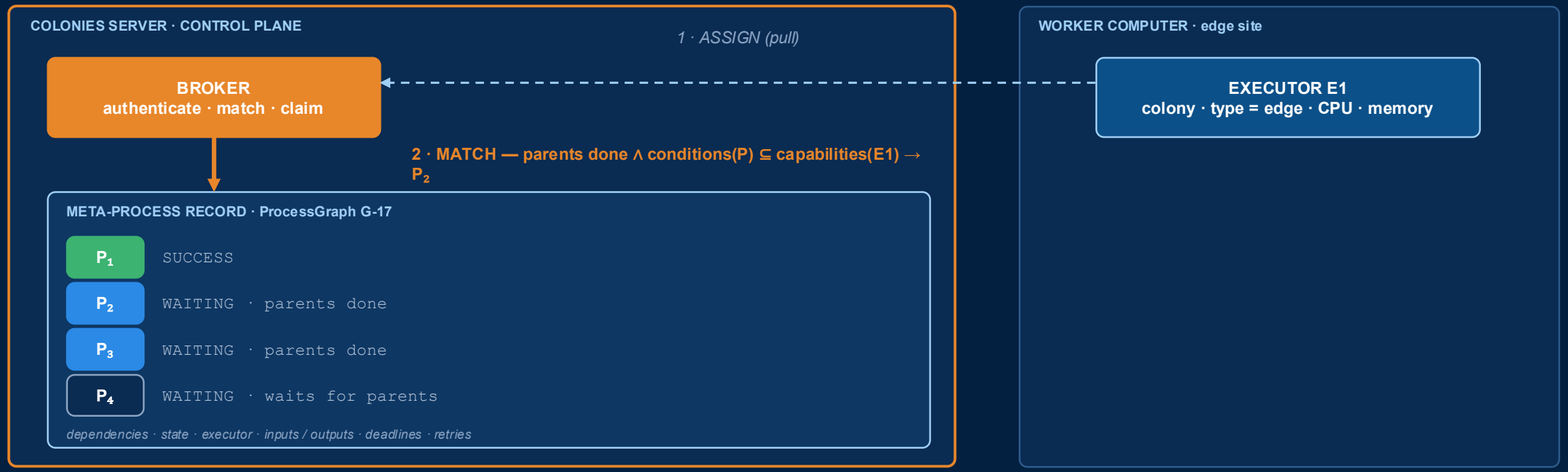


1 · PULL · the executor asks

The executor initiates Assign with its identity (colony, type, names) and capacity. Outbound only — works behind NAT and across organisations; it asks only when it can run something, so the pull is the back-pressure.

Where the lecture argued it: question 2 · station 2 (bounded queues)

Assignment begins at the executor

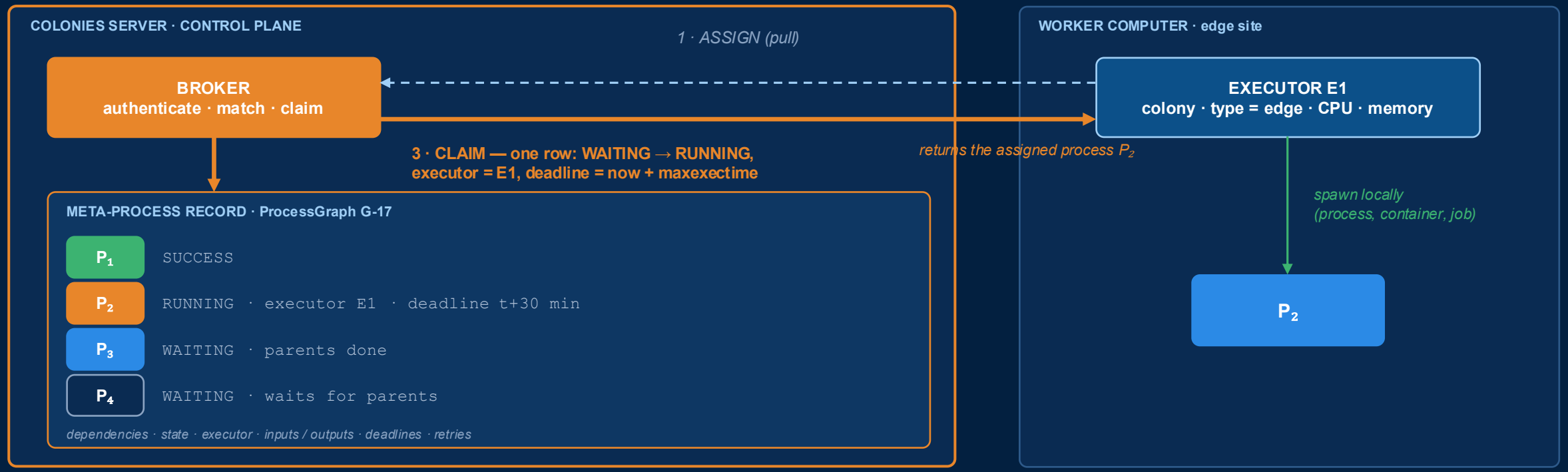


2 · MATCH · the server decides

Authenticate the executor. Among WAITING processes whose parents are done, keep those whose conditions the executor satisfies (the filter), ordered by priority and submission time (the queue policy).

Where the lecture argued it: station 1 (readiness) · station 2 (filter, then order)

Assignment begins at the executor

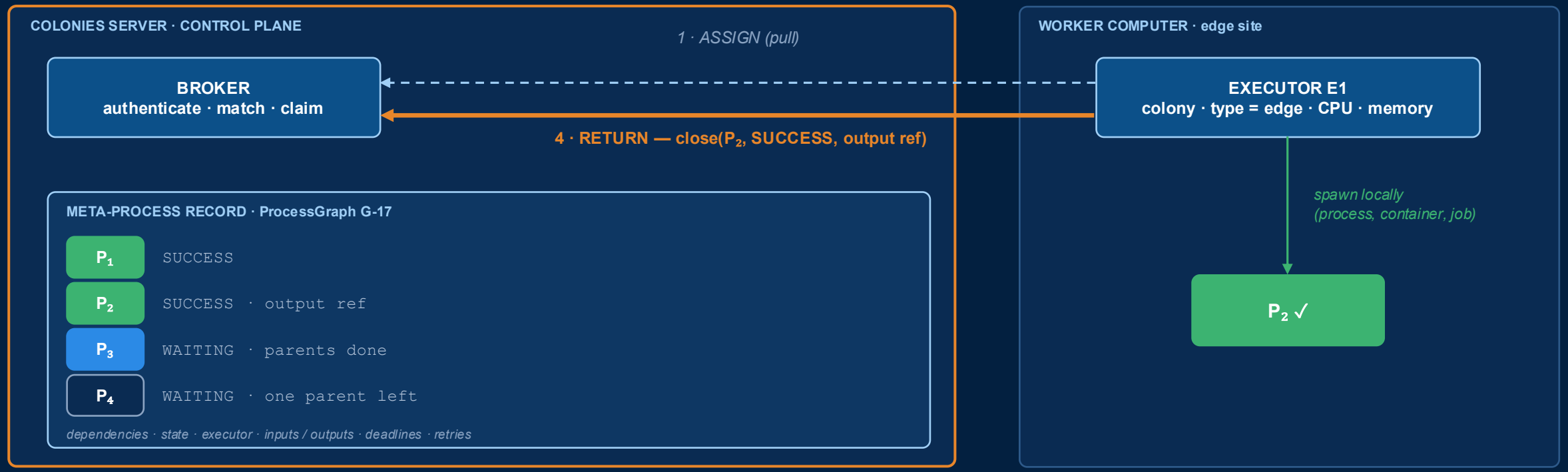


3 · CLAIM · one atomic transition

One conditional update, serialised by the database: WAITING → RUNNING, executor = E1, deadline = now + maxexectime. Two executors racing for P_2 get one winner and one "nothing for you".

Where the lecture argued it: station 4 (CAS; the deadline is the lease)

Assignment begins at the executor



4 · RETURN · an idempotent completion

The executor runs the function as platform-native work and closes the process with SUCCESS + output, or FAILED + errors. The record advances; P_2's children lose one parent to wait for. A close for a process no longer RUNNING is refused.

An HPC executor introduces a second queue — and a second clock

ColonyOS

READY work matched to the HPC executor

→

HPC executor

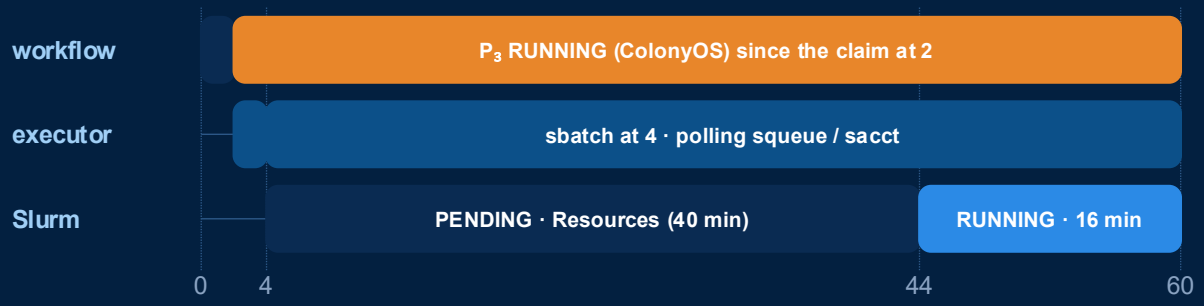
translate the FunctionSpec → sbatch; keep the Slurm job id in the record

→

Slurm

PENDING in its own queue under its own policy; then allocate and execute

P_3 = RUNNING (claimed by H1, lease running) while Slurm job 4812 = PENDING (Reason: Resources)



The lease must cover the native queue

maxexectime counts from the claim, and Slurm's queue is inside it. Set it from the expected wait plus the run time …

Follow a failure through the durable record

Record

Process P_3 · RUNNING · executor H1 · Slurm job 4812 · deadline t+120

Uncertainty

H1 stops answering. The native job may still be pending or running under Slurm — station 4: slow and dead look the same.

Recovery

The timeout loop: a WAITING process past maxwaittime is FAILED; a RUNNING process past maxexectime is unassigned and retried while retries remain, else FAILED.

Evidence

Process logs, the attempt history, the Slurm job id and squeue/sacct for 4812 — reconcile the native job before resubmitting it.

What a deadline proves

That the lease lapsed — nothing more. It is not proof that a launched Slurm job or an external side effect has stopped (station 4).

Reconcile, then retry

Before a second sbatch: is 4812 still queued or running? Cancel or adopt it. Otherwise the retry duplicates the native job — failure mode 7 with a 40-minute queue attached.

Retention of logs and instrumentation are deployment choices; the record's attempt history is what the design guarantees.

Recover from durable history; protect external work against duplicate effects. The record decides — after the evidence has been read.

The orchestrators landscape



dark = workflow engines that sit above a cluster scheduler and delegate placement to it

No system is the best point on this plane; each is the right point for who owns the network and who needs the information.